FAST Studio — Product Guide

Step-by-step walkthrough of every screen, from setup and video upload to live channels with SSAI/SGAI ads and SCTE-35 markers — with what happens behind the scenes, in plain language.

Prototype 0.1 · 20 October 2026 · Screenshots taken from the running product · All companies, people and money are fictional

FAST Studio — Complete Product Guide

Version: prototype 0.1 · 20 October 2026

Who this is for: anyone — directors, sales, content teams, new engineers. No TV or streaming background needed.

How to read it: every screenshot has **red numbered markers**. The numbered list under each picture explains what each marker means. Each chapter ends with a **"Behind the scenes"** box that explains, in plain words, what the system actually does when you click.

> All companies, people, ads and money in the screenshots are fictional demo data. Parts that are simulated are labelled as such in the product and in this guide.

Contents

1. FAST in two minutes
2. Setting it up
3. The big picture — how a channel goes from a file to a TV screen
4. Signing in and roles
5. Home — your launch checklist
6. How FAST works — the built-in course
7. Library — adding videos (generate or upload)
8. Rights — where and when content may play
9. Channels — creating a channel with the wizard
10. Schedule — building, checking and publishing the day
11. TV guide (EPG)
12. Watch live — the channel, the ads, and what's under the hood
13. Ads — campaigns and ad decisions
14. Distribution — sending channels to platforms
15. Monitoring and the audit log
16. Analytics
17. Team and permissions
18. Multiple companies (multitenancy) in action
19. Email — templates and sent emails
20. End-to-end checklist

21. Technology reference
22. What is real, what is simulated, known limits
23. Organization IDs, sign-in tokens and partner organizations
24. Partners — sharing channels with TV platforms (use case and example)

---

1. FAST in two minutes

FAST = Free Ad-Supported Streaming Television. These are TV channels that run over the internet. There's one continuous schedule, the same for everybody at the same moment. Viewers don't pay and don't pick each show: they tune in and something is already playing. **Ads pay for it.** You find FAST channels in the channel guides built into smart TVs and streaming apps.

Why it exists

- **Content owners** earn money from old films and series that would otherwise sit unused.
- **TV makers and platforms** get free channels to fill their guides, and share the ad money.
- **Viewers** get free, lean-back TV.

Why it's cheap: the channel isn't a live camera feed. A server stitches already-prepared video files together according to a schedule. This is called **virtual linear** playout, and it's exactly what FAST Studio does.

Model	Viewer pays?	Viewer picks each show?
SVOD (subscription)	Yes	Yes
AVOD (free with ads, on demand)	No	Yes
FAST	**No**	**No — it's a scheduled channel**

Three technical terms you'll meet in this guide:

- **HLS** — the streaming format. The "live stream" is really a small text file (a *playlist*) listing the next few 3-second pieces of video. The player re-reads it every few seconds.
- **SSAI / SGAI** — two ways of getting ads into the stream. In **SSAI**, the server stitches the ad video in. In **SGAI**, the server leaves a pointer and the player fetches the ads. Chapter 12 shows both live.
- **SCTE-35** — a tiny standard message saying "ad break starts here, it lasts 90 seconds, and it belongs to X". Every ad break gets one.

---

2. Setting it up

What you need

- **Java 21 or newer** (adoptium.net)
- **Node.js 20.19 or newer** (nodejs.org)

That's all. You don't install a database or video tools separately: by default the PostgreSQL database starts *inside* the app, and the video converter (ffmpeg) is downloaded automatically by npm.

Start and stop

Your computer	Start	Stop
Windows	Double-click **`start.bat`** (`start.cmd` does the same)	Double-click **`stop.bat`**
macOS / Linux	Run **`./start.sh`** in a terminal	Press Ctrl+C in that terminal

The launcher:

1. Installs the web app's libraries (first time only).
2. Builds the web app.
3. Starts the server on **http://localhost:8080**, and opens your browser once the server answers.

On Windows the server runs in the background. Its logs are in `data/runtime/app.log` and `data/runtime/app-error.log`. `stop.bat` shuts the server and its database down cleanly and keeps your data. To troubleshoot, run `start.bat -NoBrowser` from a terminal.

Optional Docker database: with Docker Desktop running, `start.bat -Docker` uses a PostgreSQL container (`faststudio-db`) instead of the built-in one. The two databases are separate. See the README for details.

First start takes a few minutes. The server:

1. Creates the database.
2. Creates two fictional companies and seven demo people.
3. **Generates about an hour of demo video** with ffmpeg: 14 programmes and 7 ads, each with its title drawn on the picture.

You can watch the Library fill up as each video turns from *Processing* to *Ready*.

Where things are stored

Folder	What's inside
`data/pg`	The built-in database (kept between restarts)
`data/runtime/`	Server logs and launcher state
`data/media/<company>/<video-id>/`	Converted video pieces (`seg_00000.ts` …), playlist and thumbnail
`data/uploads/<company-id>/<video-id>/`	Original files you upload
`data-pilot/`	The same for pilot mode, plus `pilot.env` (keep it private) and `FIRST-SIGN-IN.txt`
`backups/`	Copies made by `tools/backup.ps1`

To reset everything, run `stop.bat` (or stop the server) and delete the `data` folder. The next start rebuilds the demo from scratch.

Using it with your own data (pilot mode)

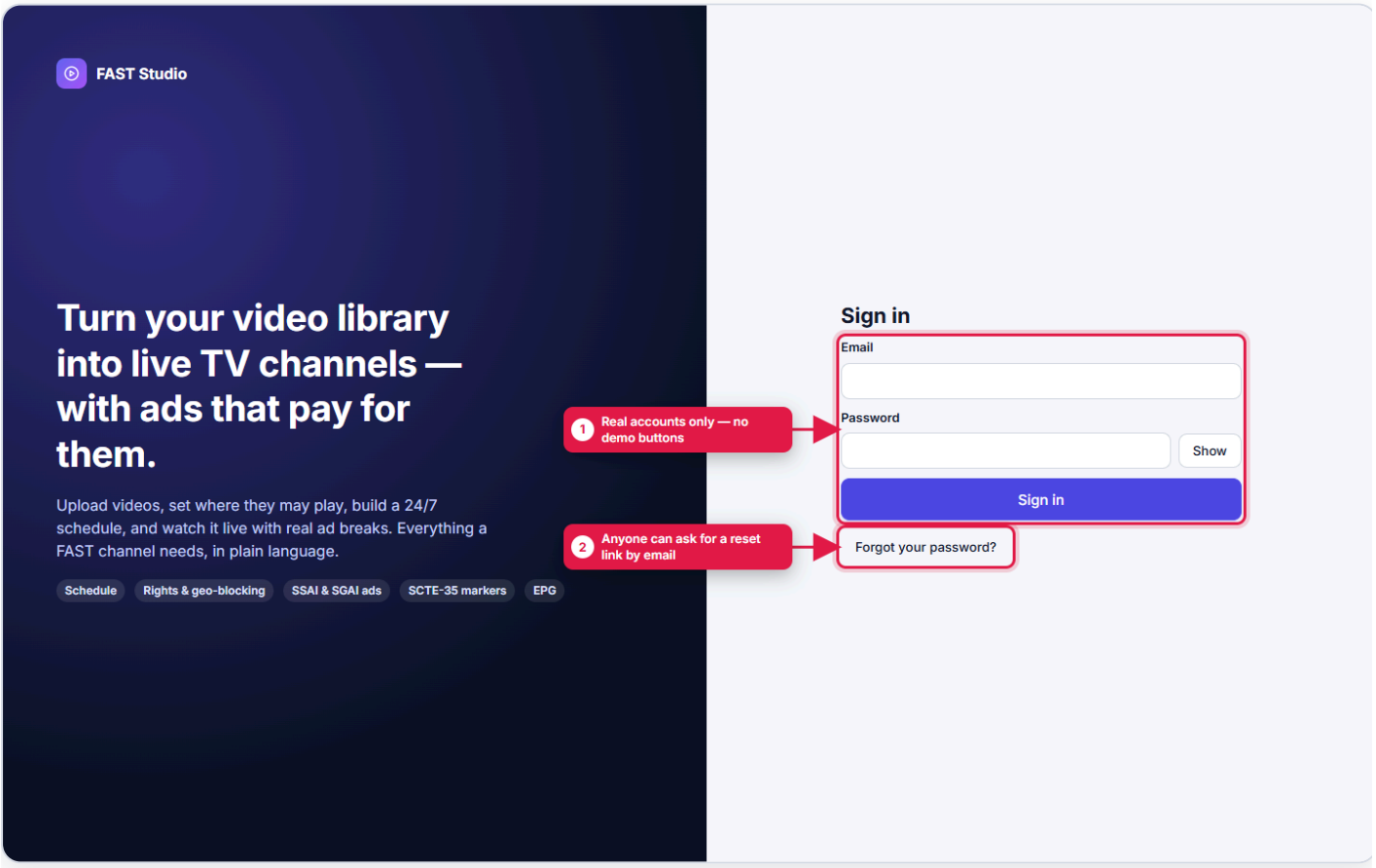
Everything above starts the **demo**: fictional companies, demo videos and one-click demo accounts. When you are ready to put in **your own companies, videos and people**, use **pilot mode**. It is the same product with three differences:

- **Nothing is invented.** There is no demo content and there are no demo accounts. You start with an empty platform.
- **Your data has its own folder,** `data-pilot/`. The demo in `data/` is never touched, and you can switch between the two (stop one, start the other).
- **Stricter safety settings.** Passwords must be strong, and temporary ones must be replaced. Video links are signed with a secret that stays the same across restarts, and the server refuses to start without that secret.

Your computer	Start pilot mode	Stop
Windows	`start.bat -Pilot` (from a terminal in the project folder)	`stop.bat`
macOS / Linux	`./start.sh --pilot`	Ctrl+C

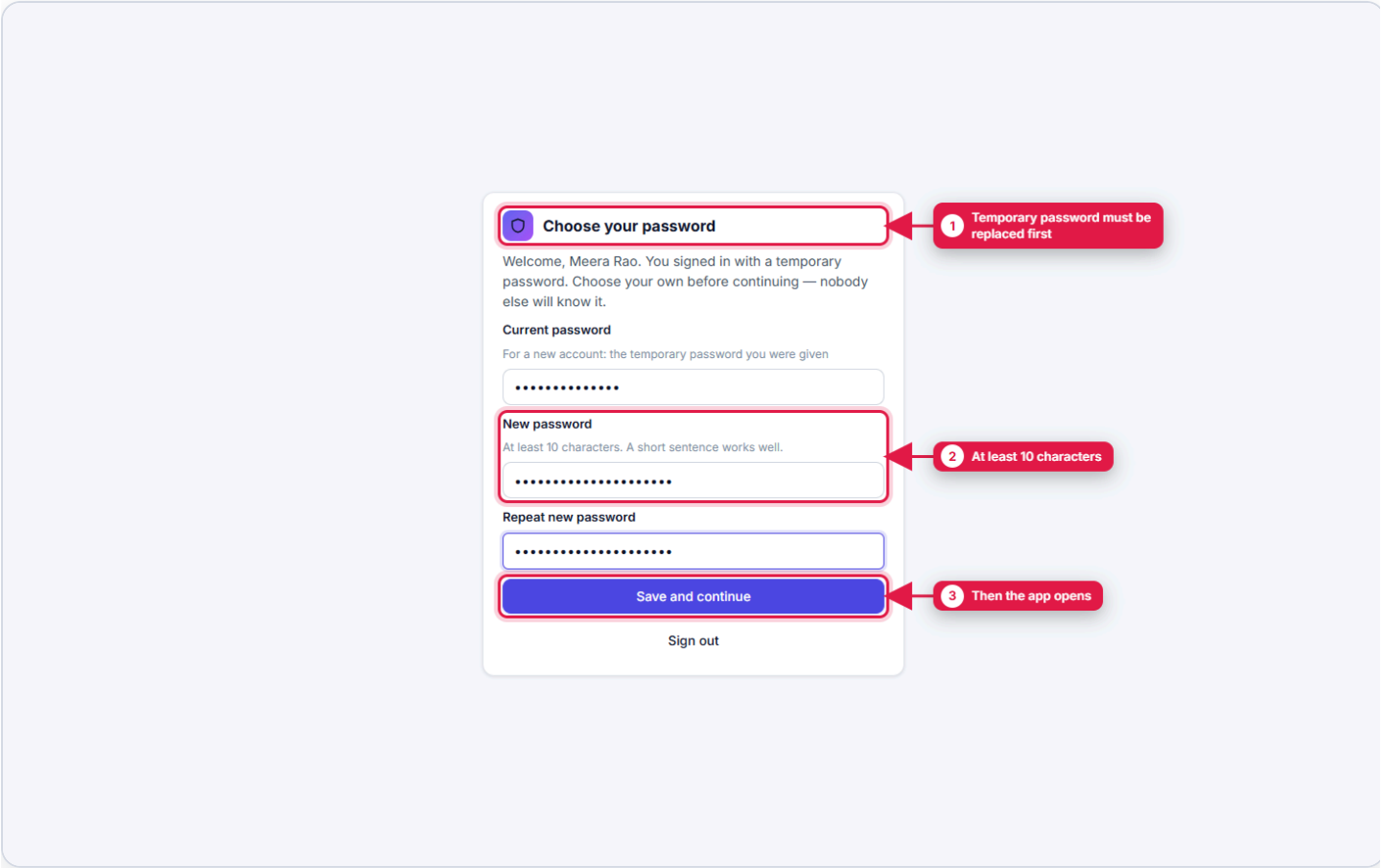
Step 1 — the very first start. The launcher creates `data-pilot/pilot.env`, which holds the video-link secret and the first **platform administrator**. If you set `FASTSTUDIO_ADMIN_EMAIL` and `FASTSTUDIO_ADMIN_PASSWORD` before starting, those are used. Otherwise the email is `admin@pilot.local`

and a random temporary password is generated. The details are printed in the terminal **once** and saved in `data-pilot/FIRST-SIGN-IN.txt`. To choose the administrator yourself, set `FASTSTUDIO_ADMIN_EMAIL` and `FASTSTUDIO_ADMIN_PASSWORD` in `.env` before this first start (see Step 6).



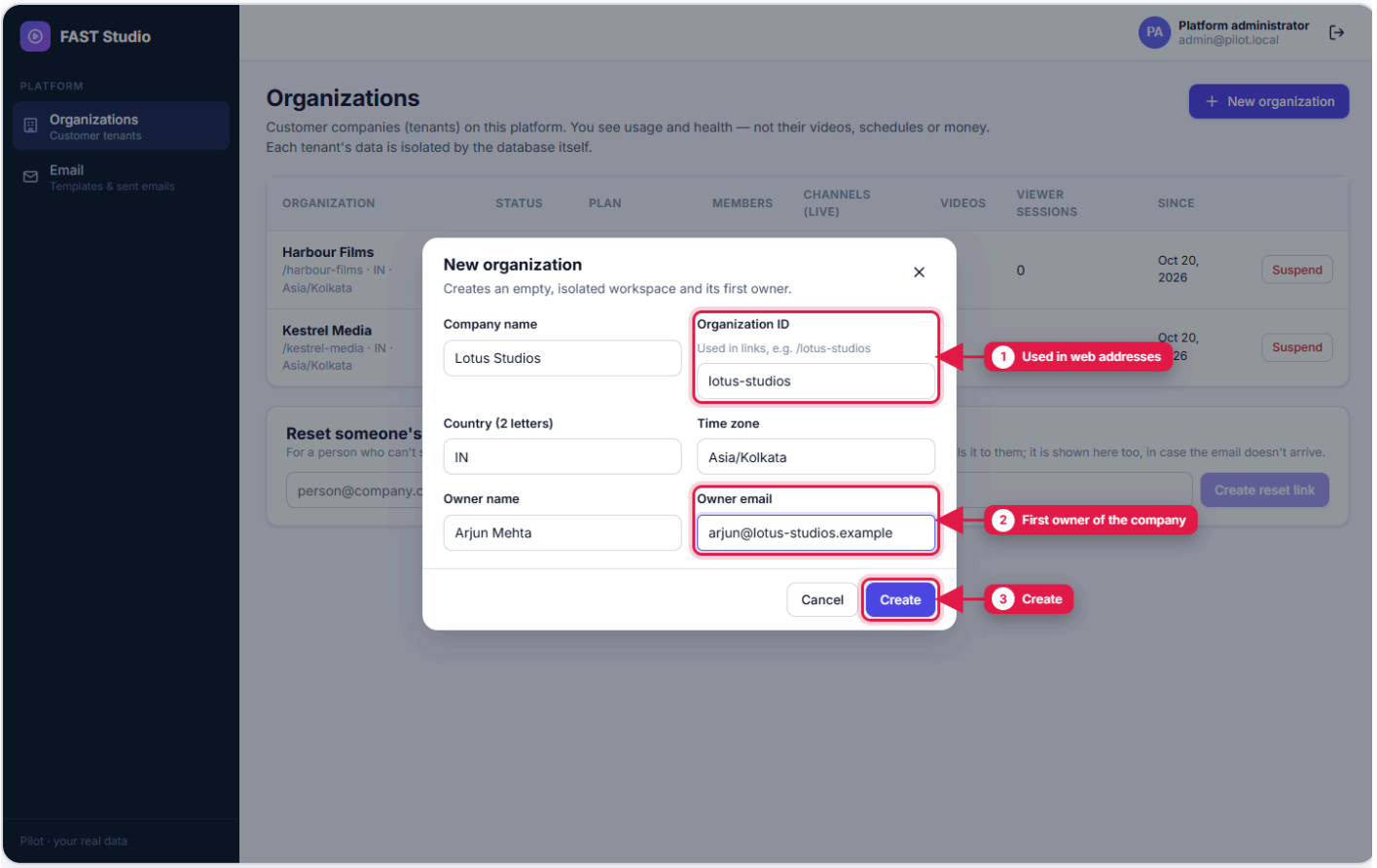
1. **Real accounts only.** In pilot mode the sign-in page has no demo buttons and no "fictional" wording.
2. **Forgot your password?** Anyone, whatever their role and administrators included, can ask for a reset link by email (chapter 4).

Step 2 — choose your own password. Anyone who signs in with a temporary password sees only this screen until they choose their own. That includes the first administrator, a new company owner and an invited colleague.



1. **Temporary password must be replaced first.** Until it is, the server refuses every other request, so the screen cannot be skipped.
2. **At least 10 characters.** The password also may not contain the name part of your email (for `meera@…`, not "meera"). Very simple ones such as `password1234` are refused.
3. **Then the app opens.** From now on the temporary password no longer works.

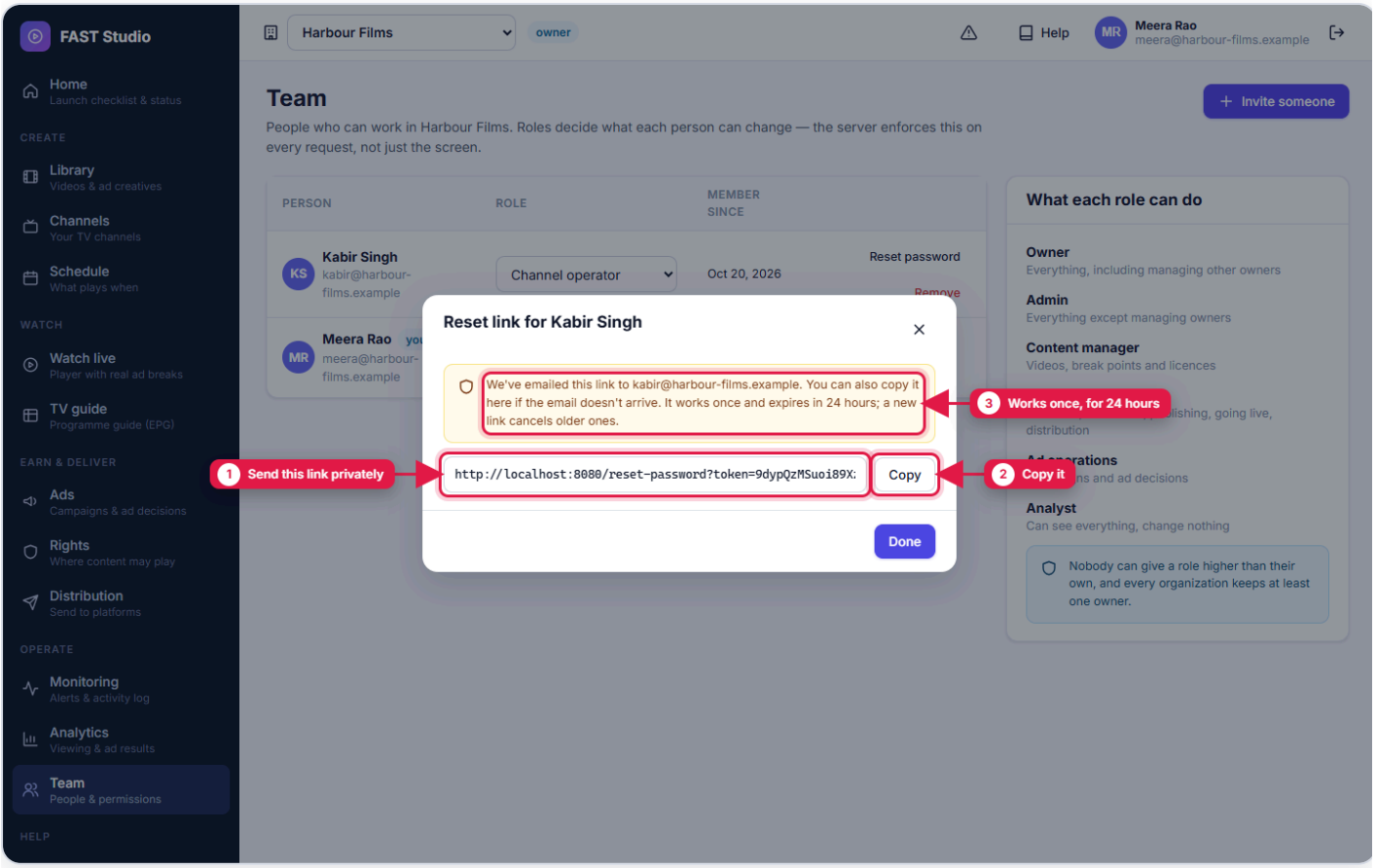
Step 3 — create your customer companies. Signed in as platform administrator, open **Organizations** → **New organization**.



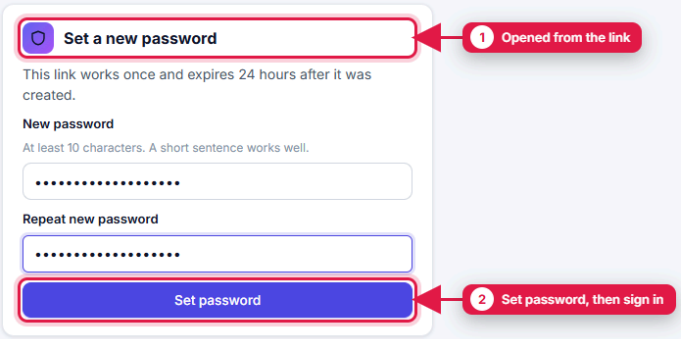
1. **Organization ID.** This short name appears in web addresses, e.g. `/lotus-studios`. It is filled in from the company name.
2. **Owner email.** This is the company's first owner. After **Create**, FAST Studio **emails** them their sign-in details (template *New organization — owner welcome*, chapter 19). The one-time password is also shown on screen in case the email doesn't arrive. If you pass it on yourself, do it privately and never in the same message as the web address.
3. **Create.** This makes a new workspace that is completely empty and isolated.

Step 4 — the owner builds their team. The owner signs in and replaces the temporary password (Step 2). From there they follow this guide from chapter 5 onwards with real videos: upload in the Library, set rights, create channels, schedule and go live. In **Team** → **Invite someone** they add colleagues, and each colleague gets an invitation email with a one-time password in the same way.

Step 5 — when someone forgets their password. People can help themselves with **Forgot your password?** on the sign-in page (chapter 4). An administrator can also help: on the **Team** page each person has a **Reset password** button. It emails the person a one-time link and also shows it to you:

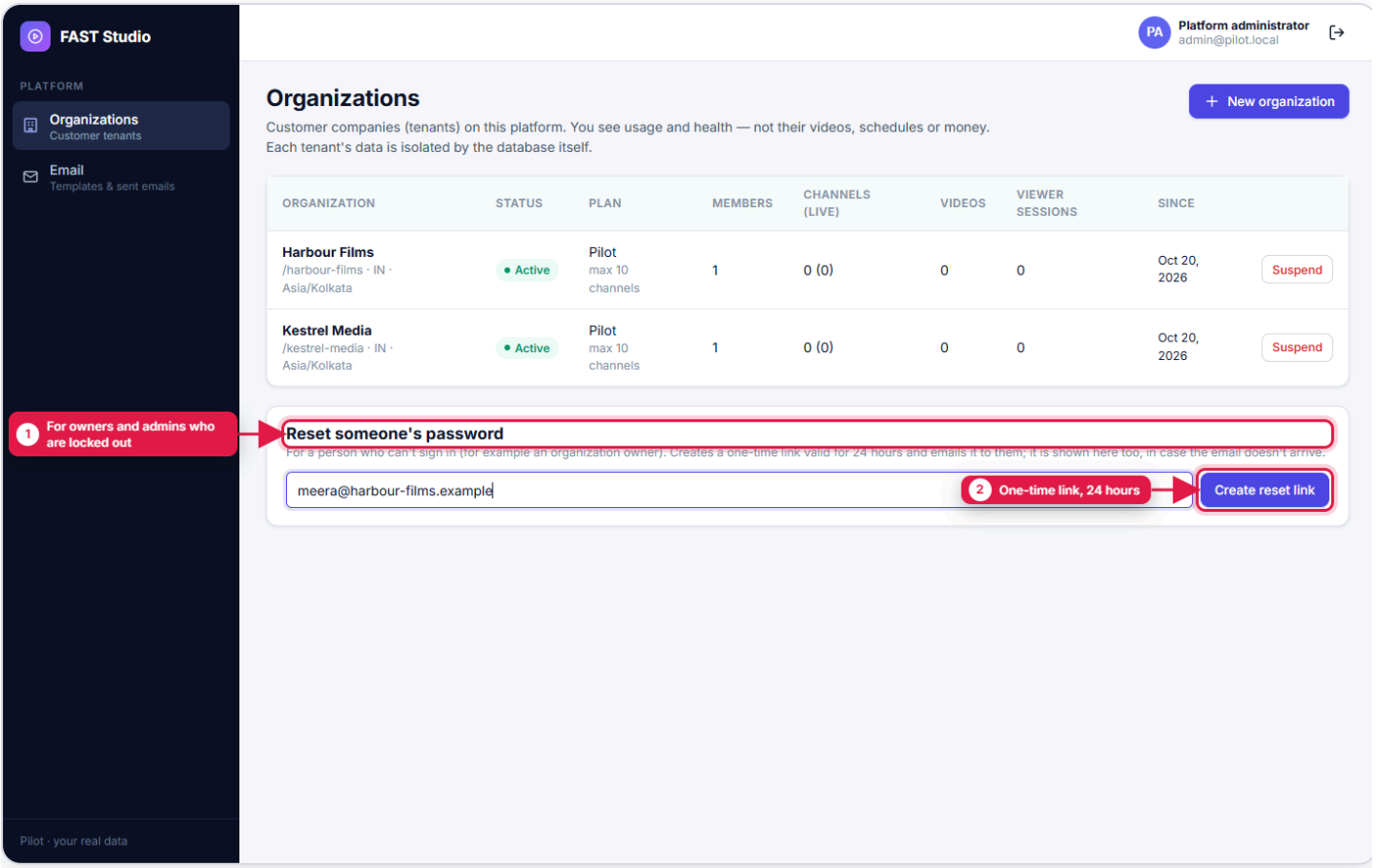


1. **The link.** It has already been emailed to the person (see the yellow note). Copy it and send it privately only if the email doesn't arrive.
2. **Copy it** with one click.
3. **Works once, for 24 hours.** Creating a new link cancels any older one.



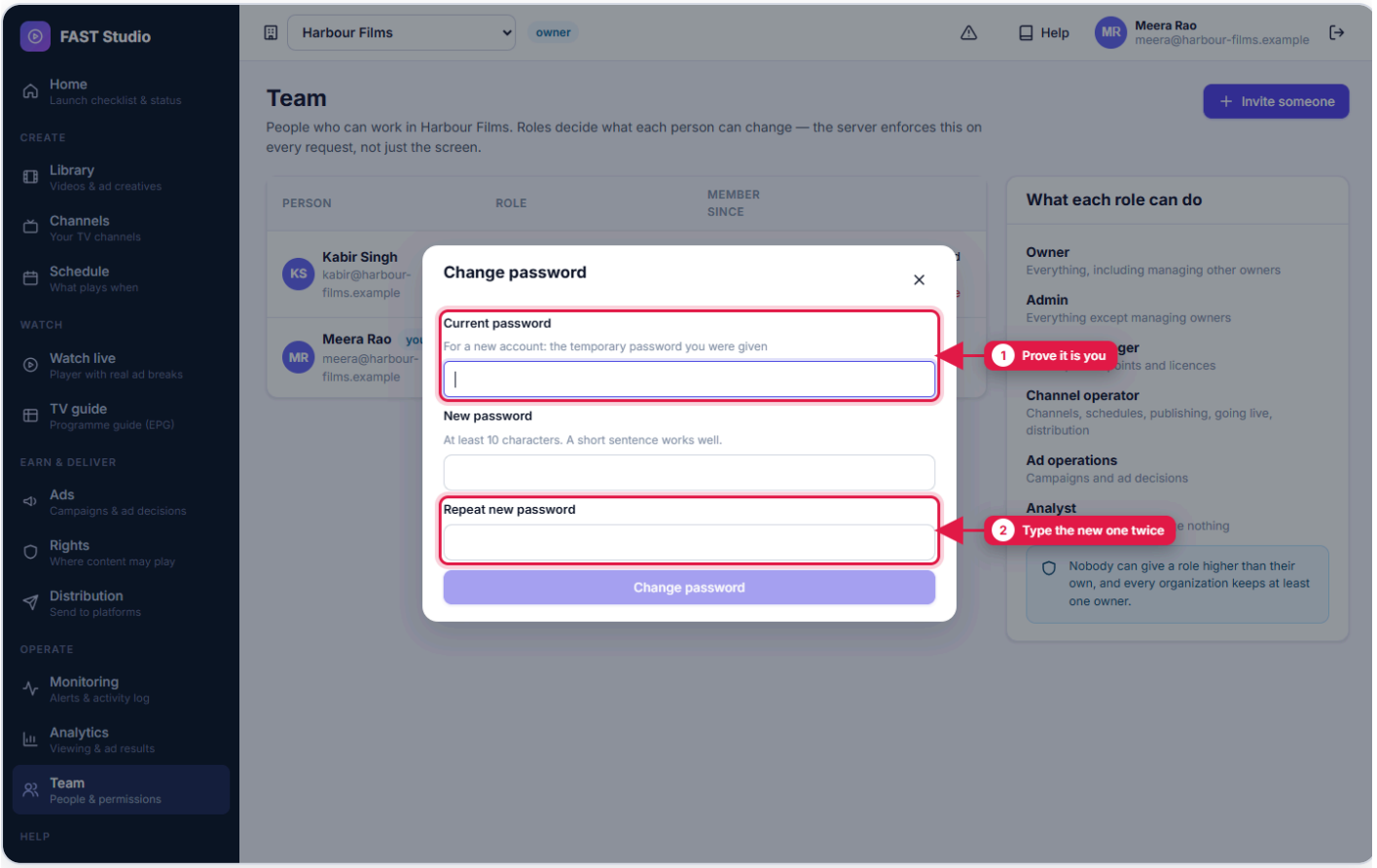
1. **Opened from the link.** No sign-in is needed: the link itself proves who it is for.
2. **Set password, then sign in** with it.

Owners and administrators who are locked out are reset by the **platform administrator**, as are people who work for more than one company. A company admin cannot reset them, because one password covers every company the person belongs to.



1. **Reset someone's password.** This card is at the bottom of **Organizations**.
2. **One-time link, 24 hours.** It works exactly like the Team reset link.

Changing your own password at any time: click your **initials** at the top right.



1. **Prove it is you.** Enter your current password.
2. **Type the new one twice.** The same rules as in Step 2 apply.

Step 6 — connect your mail server. FAST Studio sends its emails with **nodemailer**. Copy `.env.example` to `.env`; it explains, step by step with links, where to get the value for each email service (Mailtrap for testing, SendGrid, Amazon SES, Google, Microsoft 365). Then uncomment these lines in `.env`:

Setting	Example	Meaning
`FASTSTUDIO_SMTP_URL`	`smtps://user:password@smtp.example.com:465`	Your mail server. Microsoft 365, Google Workspace, Amazon SES, SendGrid and others all offer SMTP. If blank, emails are saved as files instead of being sent.
`FASTSTUDIO_MAIL_FROM`	`"FAST Studio <no-reply@yourcompany.com>"`	Sender name and address
`FASTSTUDIO_APP_URL`	`https://studio.yourcompany.com`	The address people use to open FAST Studio. Links inside emails start with it.

Restart (`stop.bat` , then `start.bat -Pilot`) and use **Email** → **Send a test email** (chapter 19) to check that it arrives.

Backups (pilot mode)

Your pilot holds real work, so keep copies. In a terminal in the project folder:

```
powershell -File tools/backup.ps1 -Pilot
```

This copies the whole database and all videos into `backups/pilot-<date>-<time>/`. If the app is running, the script stops it for about half a minute, because a database should be copied while it is closed, and then starts it again.

To go back to a copy:

```
powershell -File tools/backup.ps1 -Action Restore -Pilot -From backups/pilot-20261020-160454
```

Before overwriting anything, restore saves the current data as `backups/pilot-before-restore-<date>-<time>/`. If a restore was a mistake, restore that folder to undo it. Copy the `backups` folder to another disk or a cloud drive regularly: a copy on the same disk does not survive that disk failing.

> **Behind the scenes — pilot mode**
>
> - **Profile:** `start.bat -Pilot` starts the same program with the Spring profile `pilot` (`application-pilot.yaml`). It switches demo content off and points storage at `data-pilot/` . The database uses its own port, so demo and pilot can never mix.
> - **First administrator:** on start-up, if no platform administrator exists yet, the server creates one from `FASTSTUDIO_ADMIN_EMAIL` / `FASTSTUDIO_ADMIN_PASSWORD` . The account is marked *must change password*, and the password is never written to the log.
> - **Must-change rule:** a server-side check runs before every API call except sign-in and sign-out. It answers `403 PASSWORD_CHANGE_REQUIRED` until the password is changed; the screen is just the friendly face of that rule.
> - **Reset links:** each link is 32 random bytes. The database stores only its **SHA-256 fingerprint**, so even someone reading the database cannot use a link. A link expires after 24 hours, is marked used the first time it works, and is cancelled when a newer one is made. Every password change and reset is written to the audit log.
> - **Video-link secret:** video pieces are served from signed addresses that expire (see chapter 12). Pilot mode refuses to start without a fixed secret of at least 32 characters, so links keep working after a restart.
> - **Email:** see chapter 19. One-time passwords and links are hidden in the sent-email log.
> - **Backups:** the built-in database is copied *cold* (stopped), the simplest way to guarantee a consistent copy. On a managed PostgreSQL server you would use that server's own backups instead.

Settings: application.yaml and .env

All settings, each with a description, are in two files:

File	What it is
`backend/src/main/resources/application.yaml`	Every setting with its default, used for development and the demo
`backend/src/main/resources/application-pilot.yaml`	The few values that differ for real data (`start.bat -Pilot`)
`.env.example`	Every environment variable, documented with examples (mail servers too). Copy it to `.env` and uncomment what you need.
`.env` (you create it, git-ignored)	Your own values and secrets. `start.bat` / `start.sh` load it in demo and pilot mode.
`data-pilot/pilot.env` (generated, git-ignored)	Pilot secrets created on the first pilot start: media signing key and first administrator. Loaded after `.env` and wins.

How a value travels: `.env` → environment → `application.yaml` (`${NAME:default}`) → the server. For email, the server passes `FASTSTUDIO_SMTP_URL` on to nodemailer (`mailer/send.mjs`) as `SMTP_URL` . Any other setting can also be given as an environment variable: upper-case, with dots and dashes as underscores, e.g. `FASTSTUDIO_PARTNERS_RATE_LIMIT_PER_MINUTE=300` .

Developer mode (instant reload while editing code)

```
cd backend && ./mvnw spring-boot:run
```

```
cd frontend && npm run dev
```

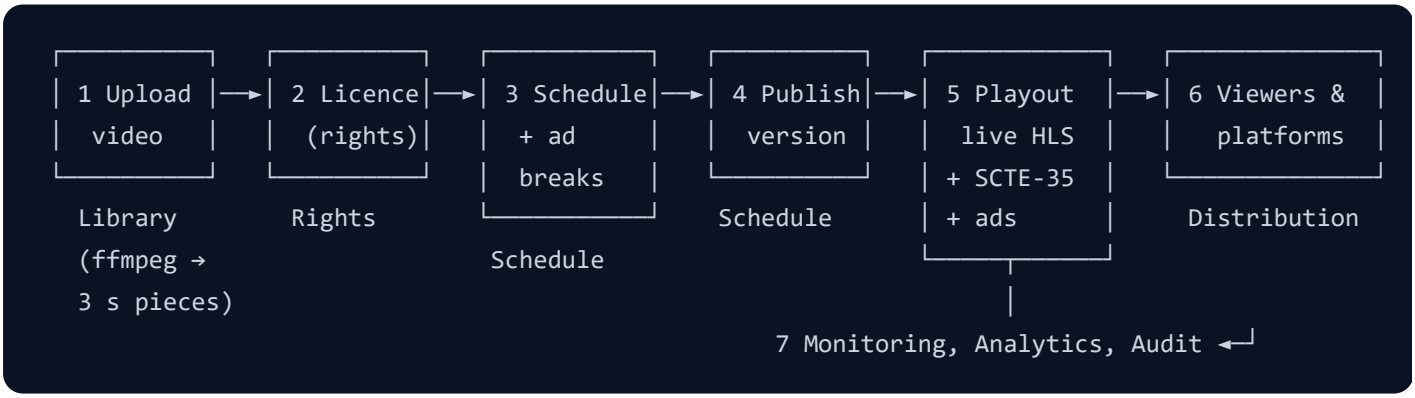
Run each in its own terminal, then open http://localhost:5173.

Troubleshooting

Symptom	Fix
Port 8080 already in use	Run `stop.bat`. If another program owns the port, the launcher leaves it alone, so stop that program first
Something went wrong at start	Look in `data/runtime/app-error.log`, or run `start.bat -NoBrowser` in a terminal to see messages
Videos stay "Failed"	Run `npm install` in `frontend/` (it downloads ffmpeg), then press **Retry** on the video
Channel shows "We'll be right back"	Nothing is scheduled right now. Open Schedule → Auto-fill → Publish
Player says "Playback problem"	The player retries by itself. If the channel was stopped, press Go live again

---

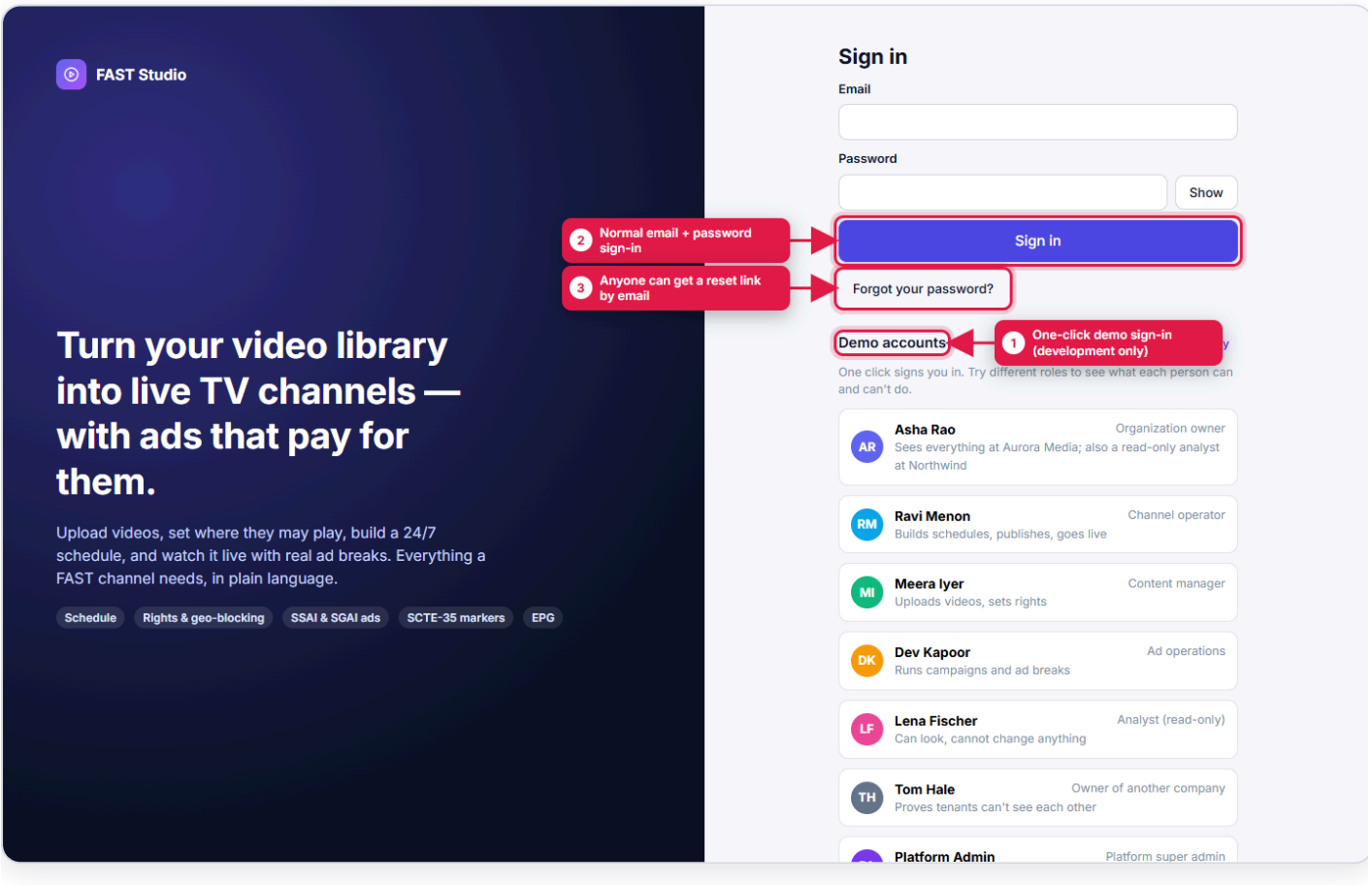
3. The big picture



1. **Upload:** a video file is converted into identical 3-second pieces.
2. **Licence:** you record where and when it may play.
3. **Schedule:** you arrange videos on a timeline, with ad breaks.
4. **Publish:** you freeze that timeline as a numbered version that viewers get.
5. **Playout:** every few seconds the server works out what should be on air now and writes the live playlist. That includes ad-break markers and, depending on the viewer's app, the ads.
6. **Delivery:** the stream and its TV guide go to your web player and to partner platforms.
7. **Operations:** alerts, analytics and the audit log watch everything.

---

4. Signing in and roles

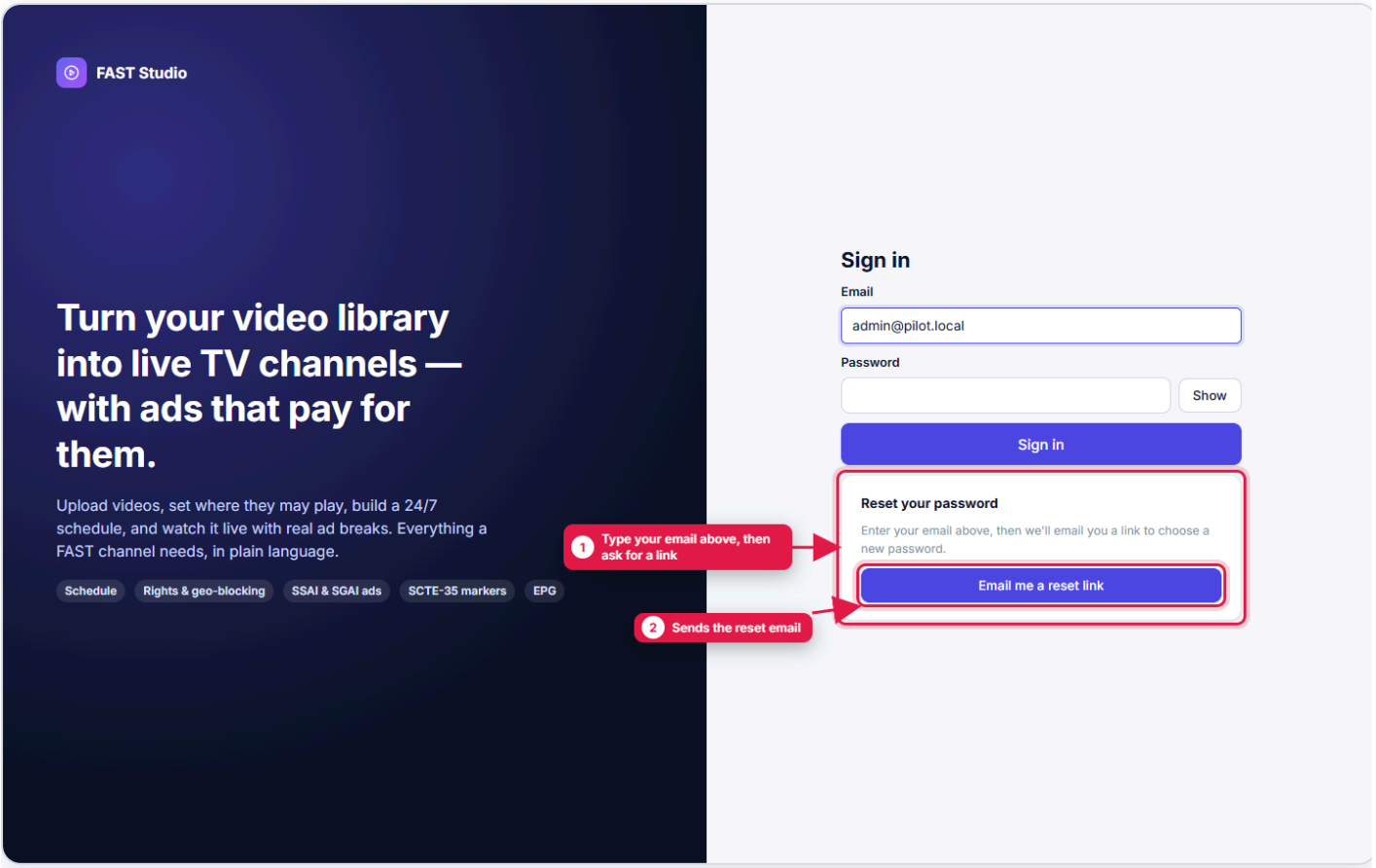


1. **Demo accounts** — one click signs you in as a fictional person with a particular role. This list only appears in demo mode and is labelled *Development only*.
2. **Normal sign-in** — email and password, with a show/hide button. Wrong details give a neutral message ("Email or password is incorrect"), so nobody can find out which emails exist.
3. **Forgot your password?** — available to everyone: owners, staff, partners later, and platform administrators. See below.

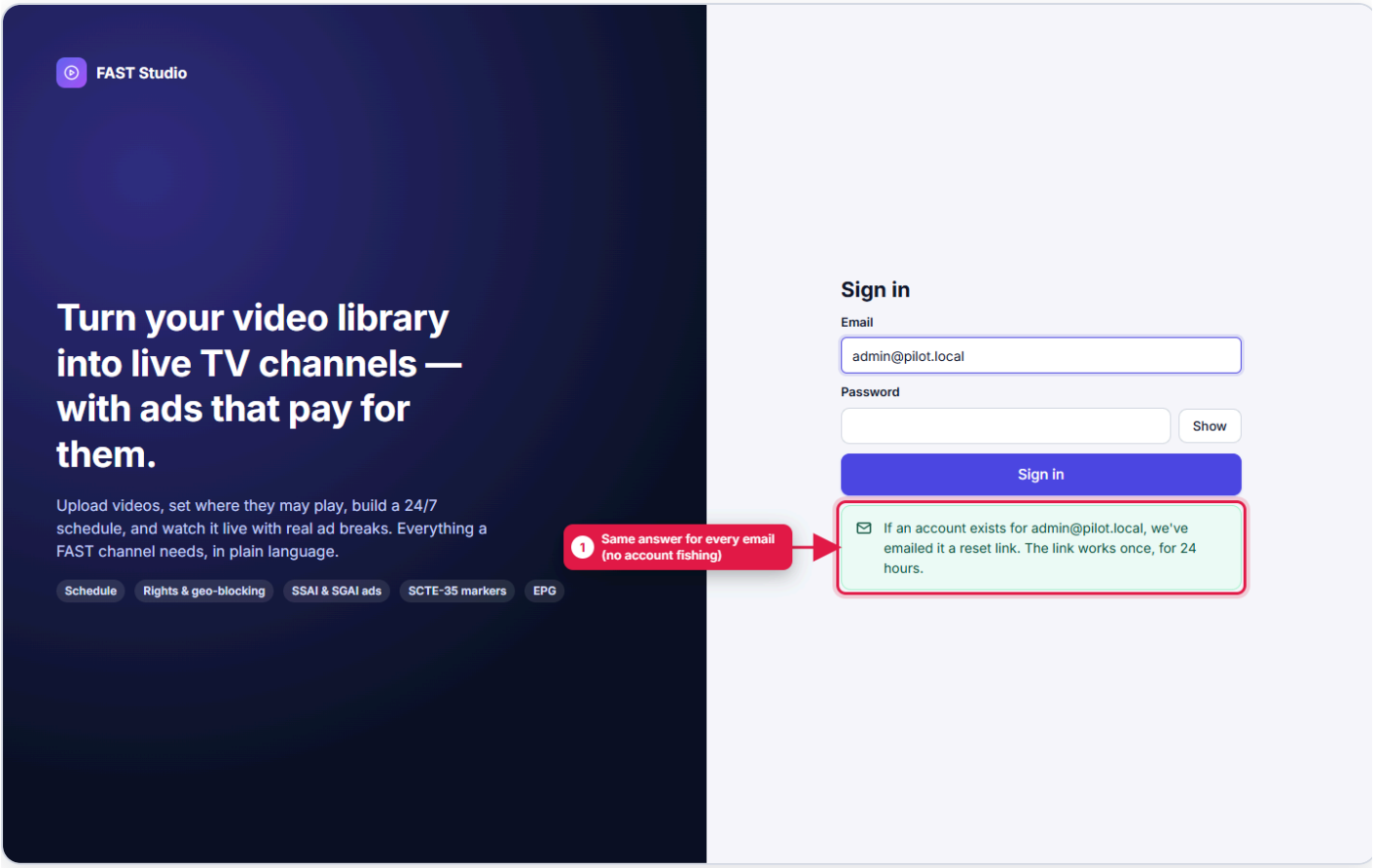
All demo passwords are `demo1234`.

Person	Role	Good for showing
Asha Rao — owner@aurora.test	Owner	Everything
Ravi Menon — operator@aurora.test	Channel operator	Schedules, publishing, going live
Meera Iyer — content@aurora.test	Content manager	Videos, licences
Dev Kapoor — ads@aurora.test	Ad operations	Campaigns
Lena Fischer — analyst@aurora.test	Analyst	Read-only view
Tom Hale — owner@northwind.test	Owner of *another* company	Data isolation
Platform Admin — admin@platform.test	Platform admin	Managing customer companies
Sam Lee — partner@partnertv.test	Partner admin at *Partner TV*, a fictional smart-TV app	What a partner sees: only the 2 channels Aurora shares (chapter 24)

Forgot your password?



1. **Type your email above, then ask for a link.** Click *Forgot your password?* under *Sign in*.
2. **Sends the reset email.** It contains a *Choose a new password* button that opens *Set a new password* (chapter 2, Step 5).



1. **Same answer for every email.** The message is identical whether or not an account exists, so nobody can use this to find out who has an account.

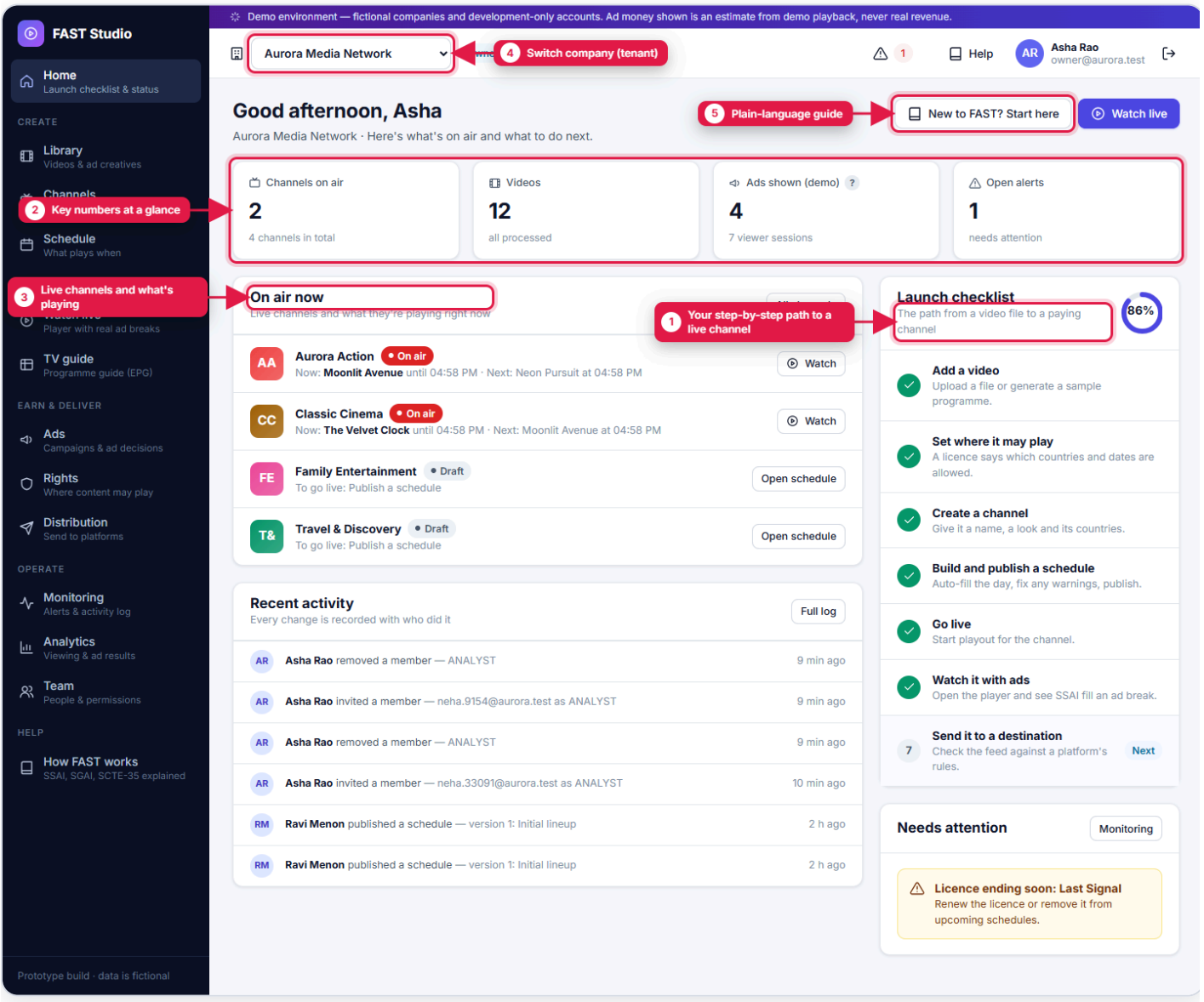
Each account gets at most 3 links per hour, and a new link cancels older ones. After the reset, a *Password changed* security email follows, so the person notices if it wasn't them. Without a mail server (for example in the demo), the email is saved in the outbox and the message says to ask an administrator to pass it on. The seven demo accounts can't be reset: their password always stays `demo1234`.

Behind the scenes — signing in

- **Passwords:** stored only as **bcrypt hashes**, never as readable text.
- **Sign-in token:** after sign-in the browser gets a signed **sign-in token** (JWT, 10 minutes) and a **refresh token**, both in cookies that page scripts can't read (HttpOnly). The app renews the token silently. Every action that changes something also sends a **CSRF token**, which stops other websites from acting on your behalf. What the token contains: chapter 23.
- **Throttling:** five wrong passwords in five minutes pause further attempts.
- **Reset links:** 32 random bytes, stored only as a fingerprint (SHA-256). Each works once and expires after 24 hours.
- **Production:** the login would move to the company's single sign-on (OIDC); nothing else changes.
- **Pilot mode:** there are no demo accounts. Temporary passwords must be replaced at first sign-in, and passwords need at least 10 characters (see chapter 2).

---

5. Home



1. **Launch checklist** — the seven steps from a video file to an earning channel. Each step ticks itself off from real data (for example, "Go live" ticks once a channel is actually on air). Click any step to jump to the right page.
2. **Key numbers** — channels on air, videos, ads shown in demo playback, open alerts.
3. **On air now** — every channel with what's playing *right now* and what's next, plus a **Watch** button.
4. **Company switcher** — people who work for several companies switch here. Each company's data is completely separate (chapter 18).
5. **Help** — opens the built-in course.

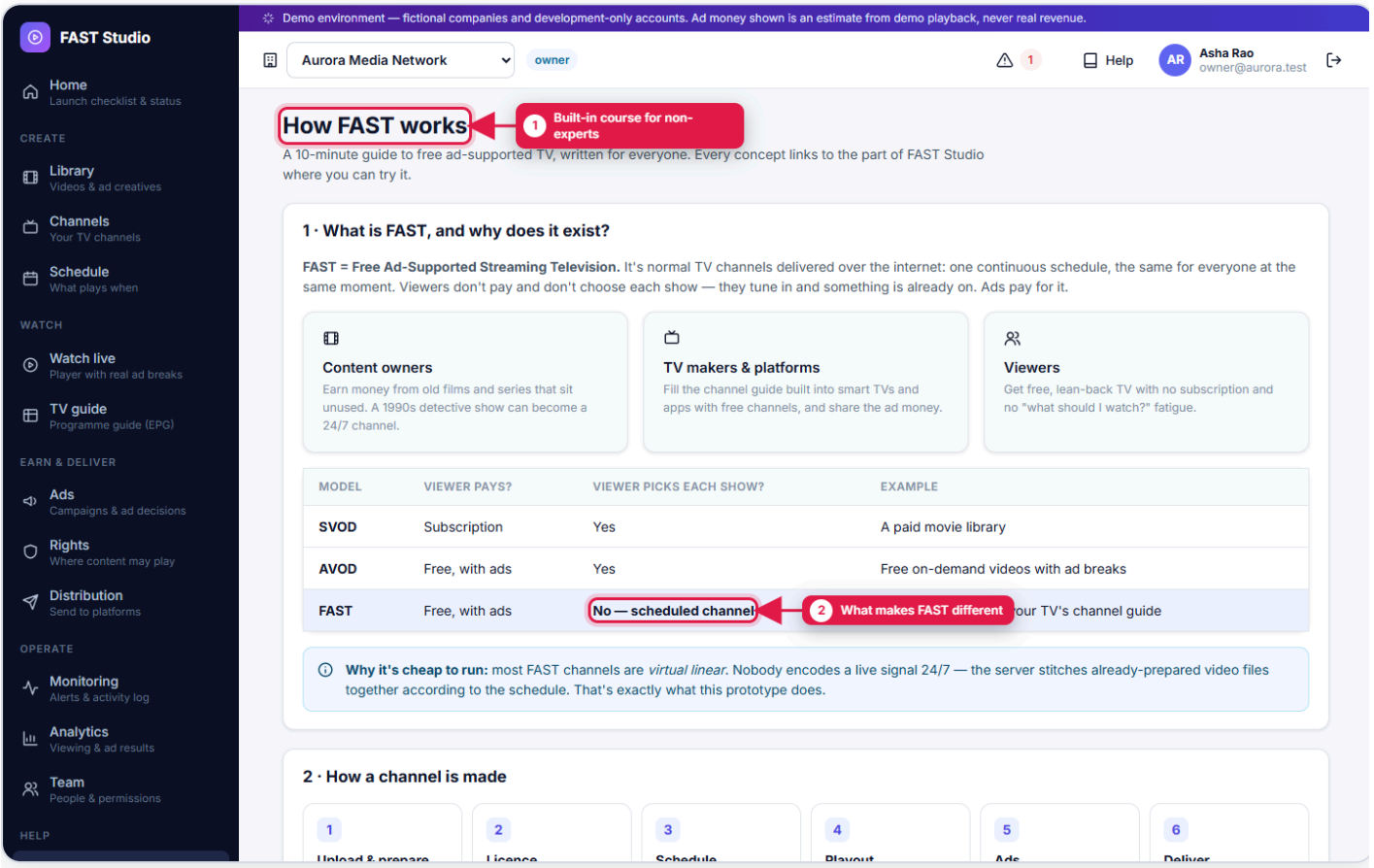
At the top, a purple ribbon reminds everyone this is a demo environment.

> **Behind the scenes:** the Home page asks the server for one summary of counts, checklist status, recent activity and open alerts. "On air now" comes from the same published schedule the player uses, so it can never disagree with what viewers see. The page refreshes itself every 15 seconds.

---

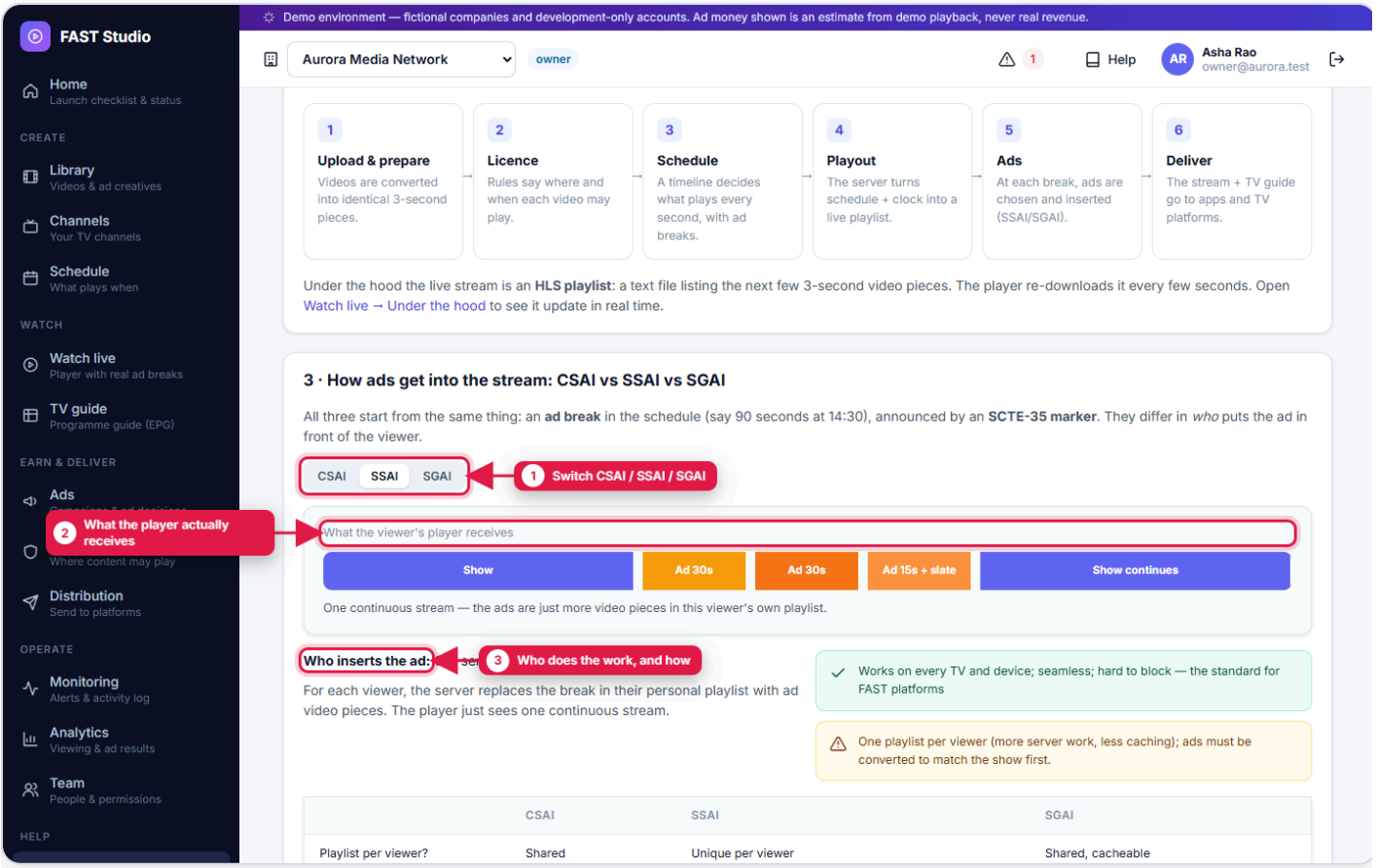
6. The built-in course

The **How FAST works** page (menu → Help) teaches the whole subject with interactive pieces.

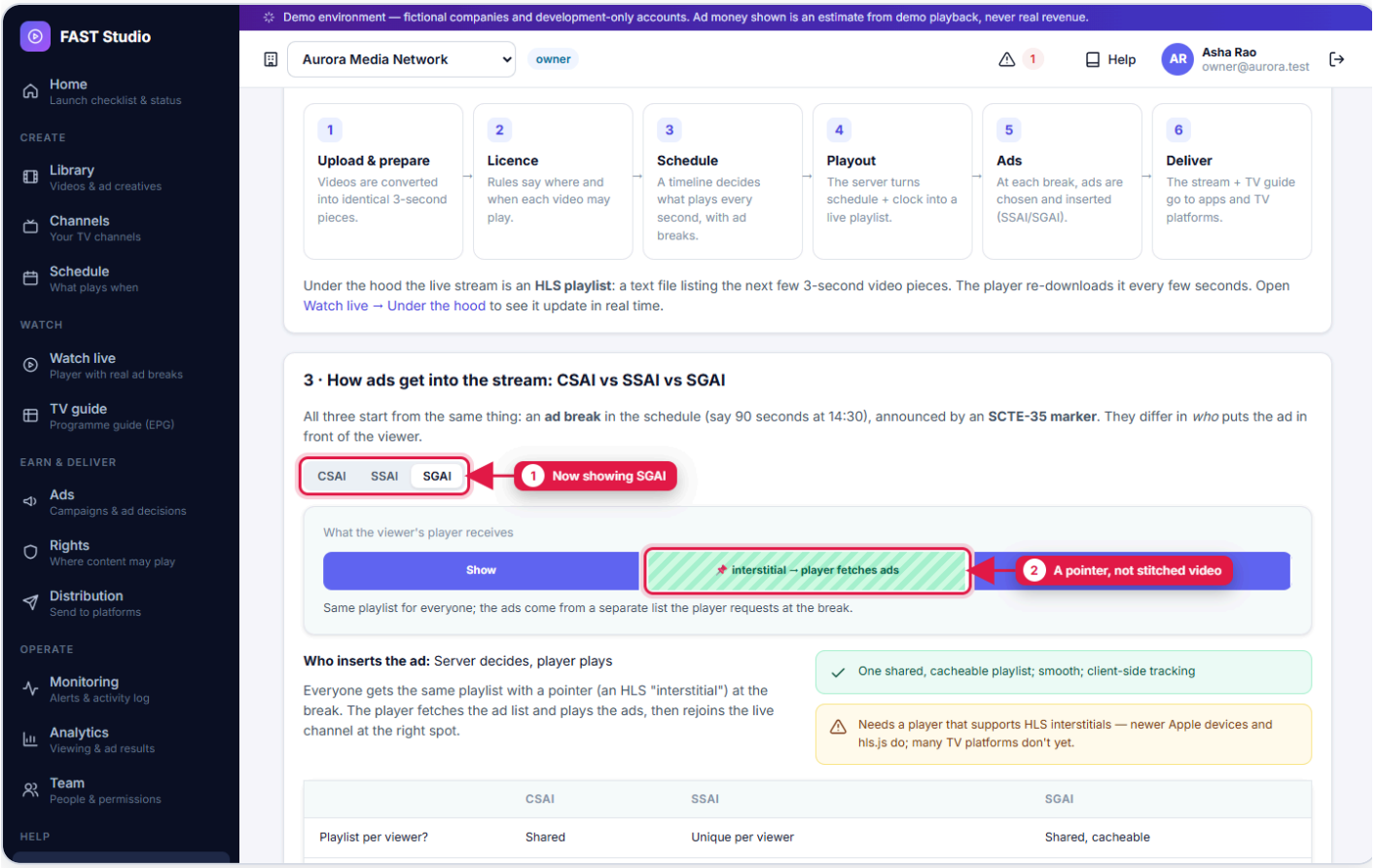


1. A short course aimed at people new to TV and streaming.
2. The comparison that matters: FAST is *scheduled*, not on-demand.

SSAI vs SGAI, interactively



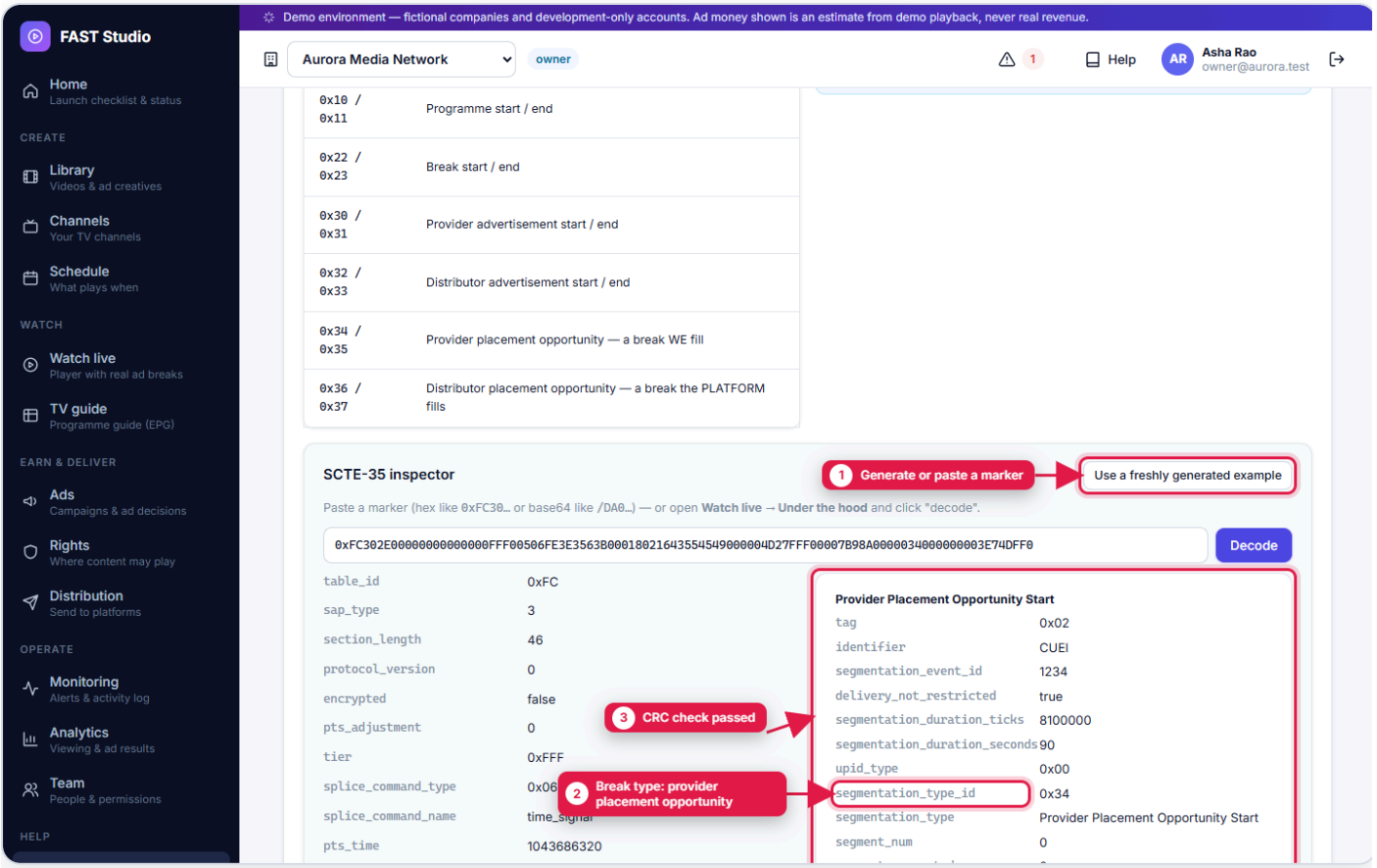
1. **Switch between CSAI, SSAI and SGAI** to see how each one works.
2. **What the player actually receives** — with SSAI, one continuous stream in which the ads are just more video pieces.
3. **Who does the work, and how** — plus the strengths and weaknesses of each approach.



1. **SGAI selected.**
2. Instead of stitched ad video, the stream contains a **pointer** (an "interstitial"). The player follows it to fetch and play the ads itself.

	CSAI (in the player)	**SSAI (server stitches)**	**SGAI (server guides)**
Who inserts the ad	Player	Server	Server decides, player plays
Same playlist for every viewer?	Yes	No — one per viewer	Yes (cache-friendly)
Ad blockers	Easily blocked	Very hard	Harder
Works on	Players with ad SDK	**Every HLS player** — why FAST platforms use it	Newer players (Apple, hls.js)

The SCTE-35 inspector



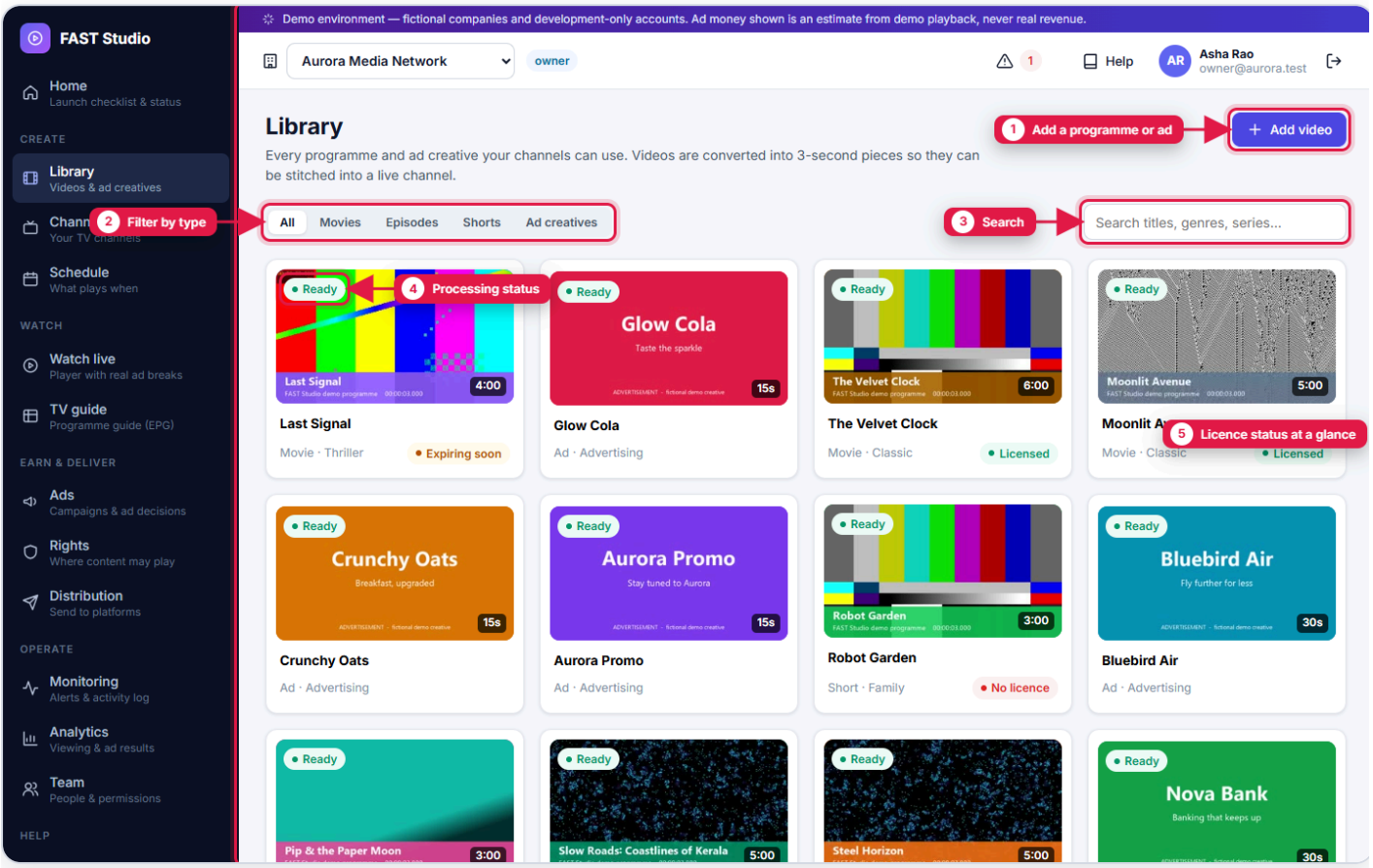
1. **Generate (or paste) a marker.** The same kind the live channel produces.
2. **The decoded break type.** `0x34` means "a break the *provider* (us) fills"; `0x36` means "a break the *platform* fills".
3. **Checksum valid** — partners reject markers with a bad checksum, so this matters.

> **Behind the scenes — what a marker is** A SCTE-35 marker is about 50 bytes of binary data. It starts with `0xFC` and contains:
>
> - *when* (in 90,000 ticks per second);
> - *what kind of moment*;
> - *how long*;
> - an *ID* that pairs the start with the end;
> - a 4-byte checksum (CRC-32/MPEG-2).
>
> FAST Studio builds these bytes itself (`Scte35.java`). The unit tests decode the official sample from the SCTE standard and check the checksum, so we know the bytes are right.

---

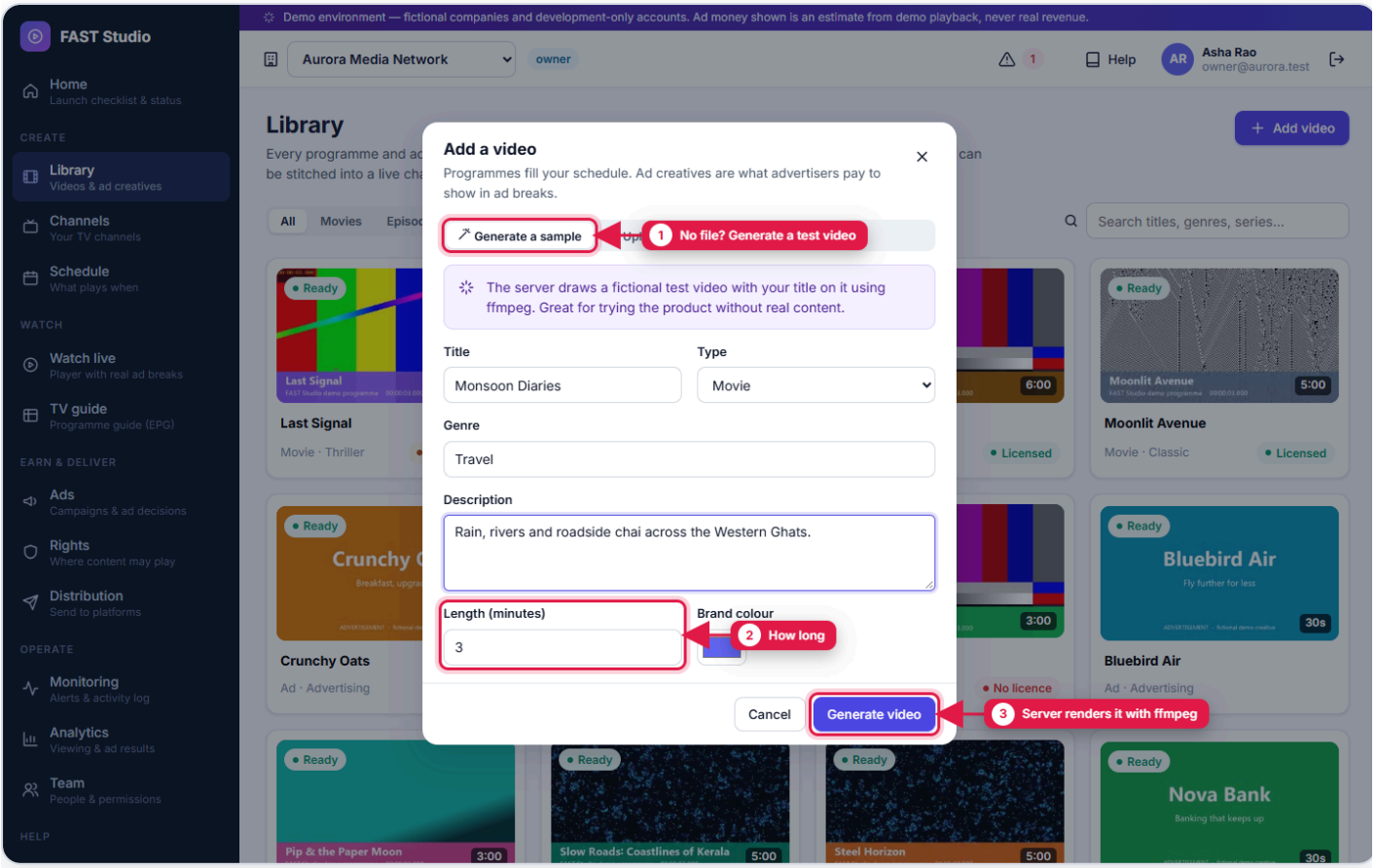
7. Library

The Library holds every **programme** (movies, episodes, shorts) and every **ad creative** (the ad videos advertisers supply).



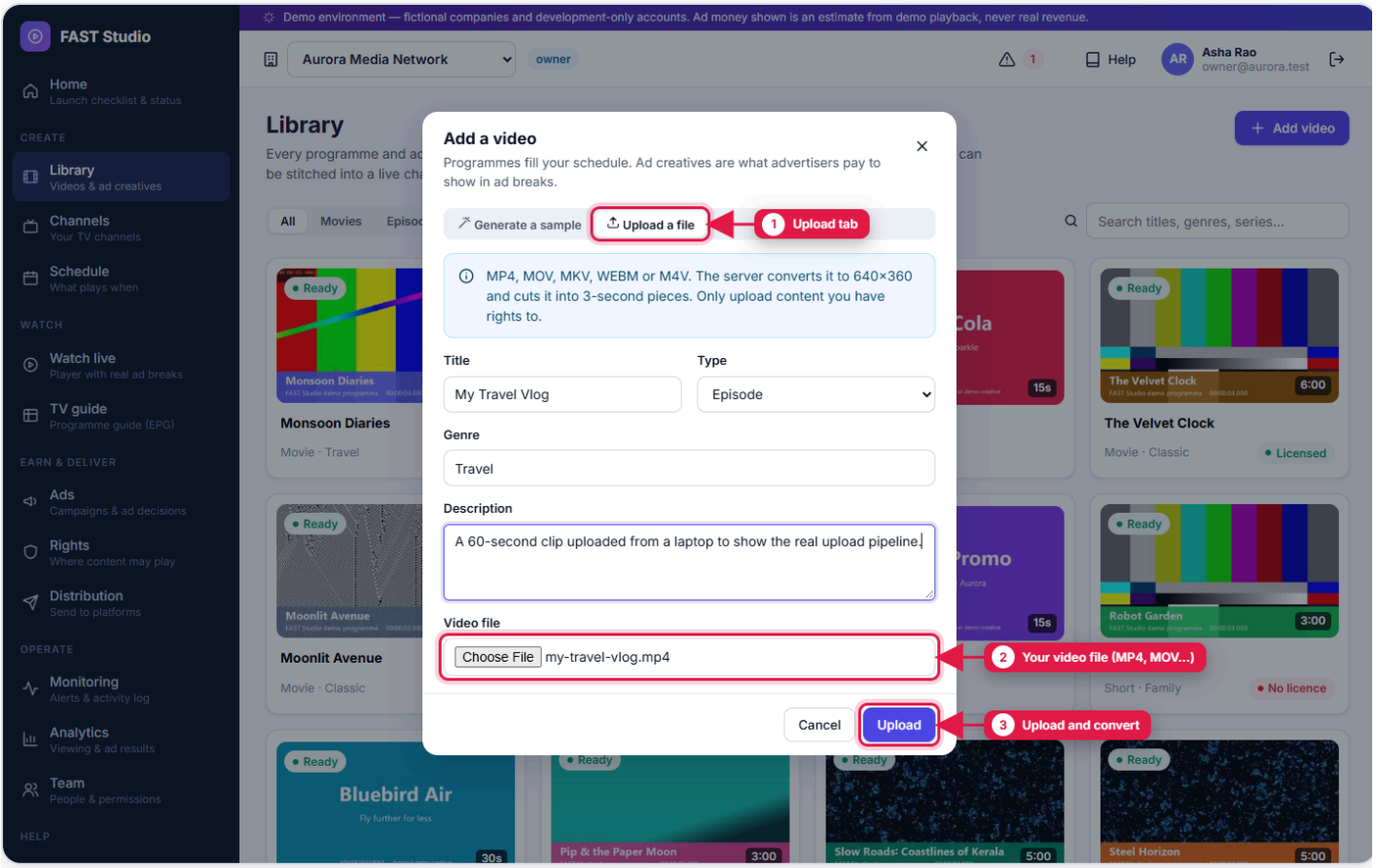
1. **Add video** — upload a file or generate a sample.
2. **Filter by type** — Movies, Episodes, Shorts, Ad creatives.
3. **Search** — title, genre or series.
4. **Processing status** — *Processing* (spinner), *Ready* or *Failed*.
5. **Licence status** — whether the video is allowed to play: *Licensed*, *Expiring soon*, *Expired*, *No licence*, *Incomplete licence*. Demo videos deliberately include each case.

Option A — generate a sample video (no file needed)

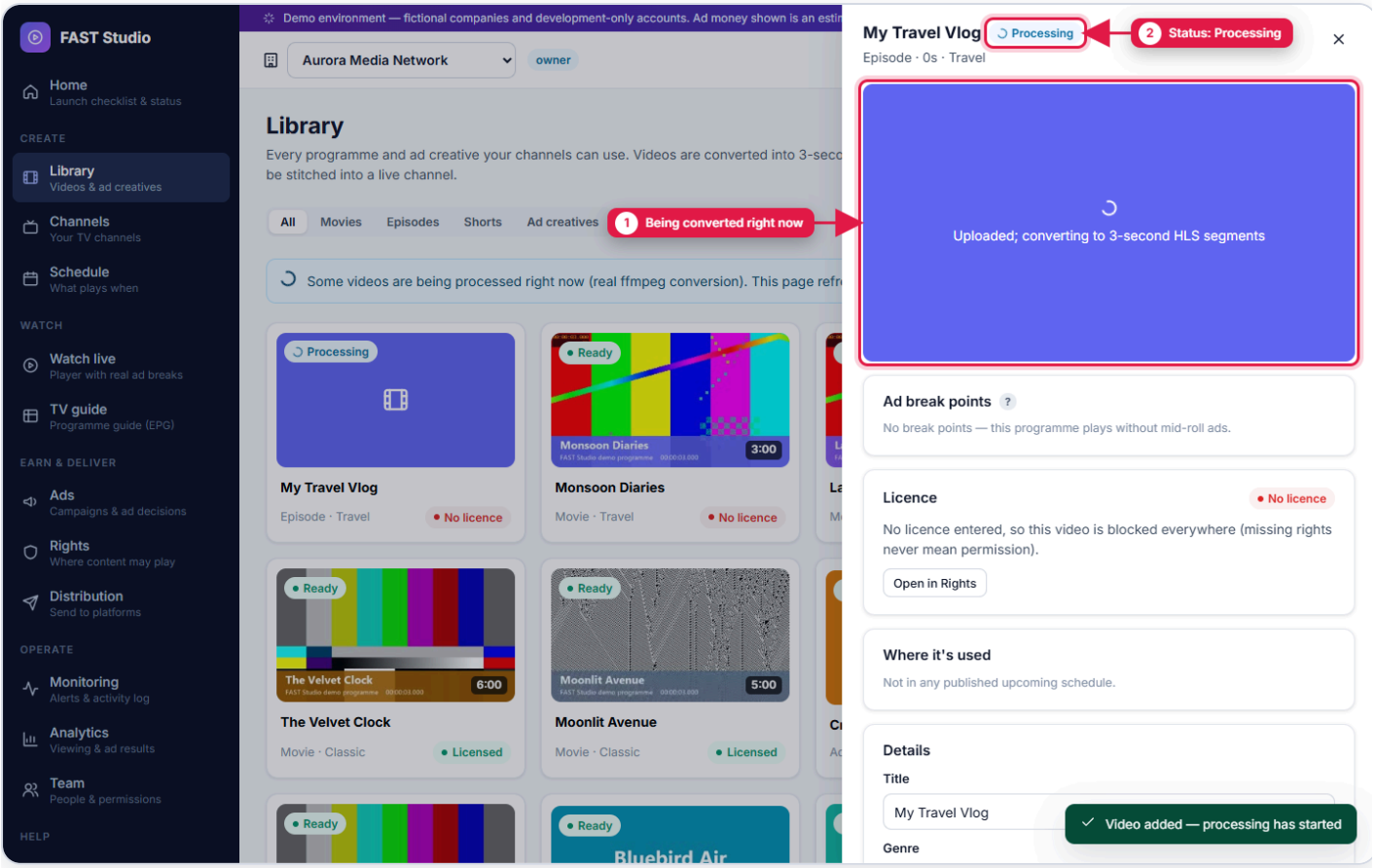


1. **Generate a sample** — for trying things out without real content.
2. **Length.**
3. **Generate video** — the server draws a fictional test video with your title on it.

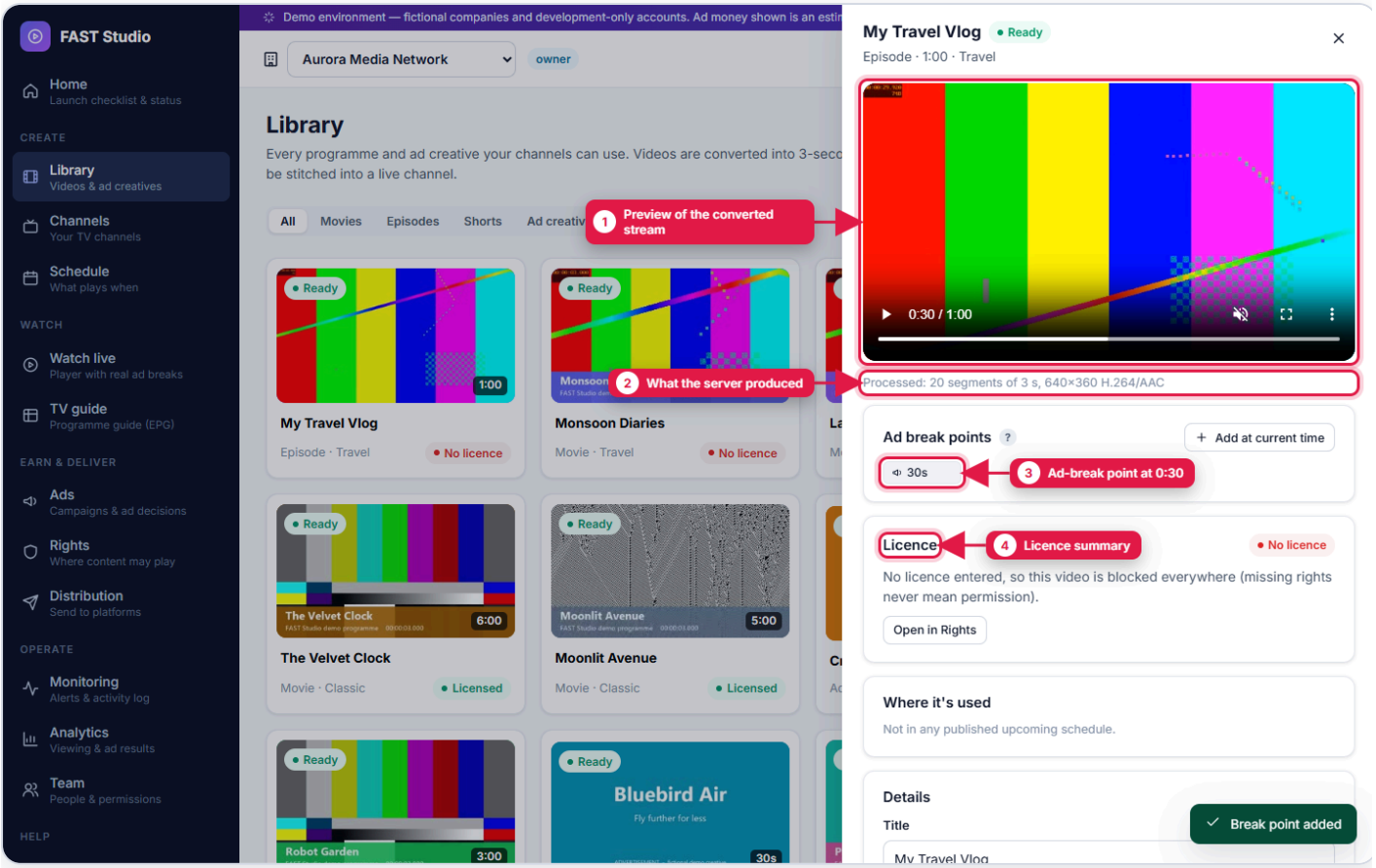
Option B — upload your own video



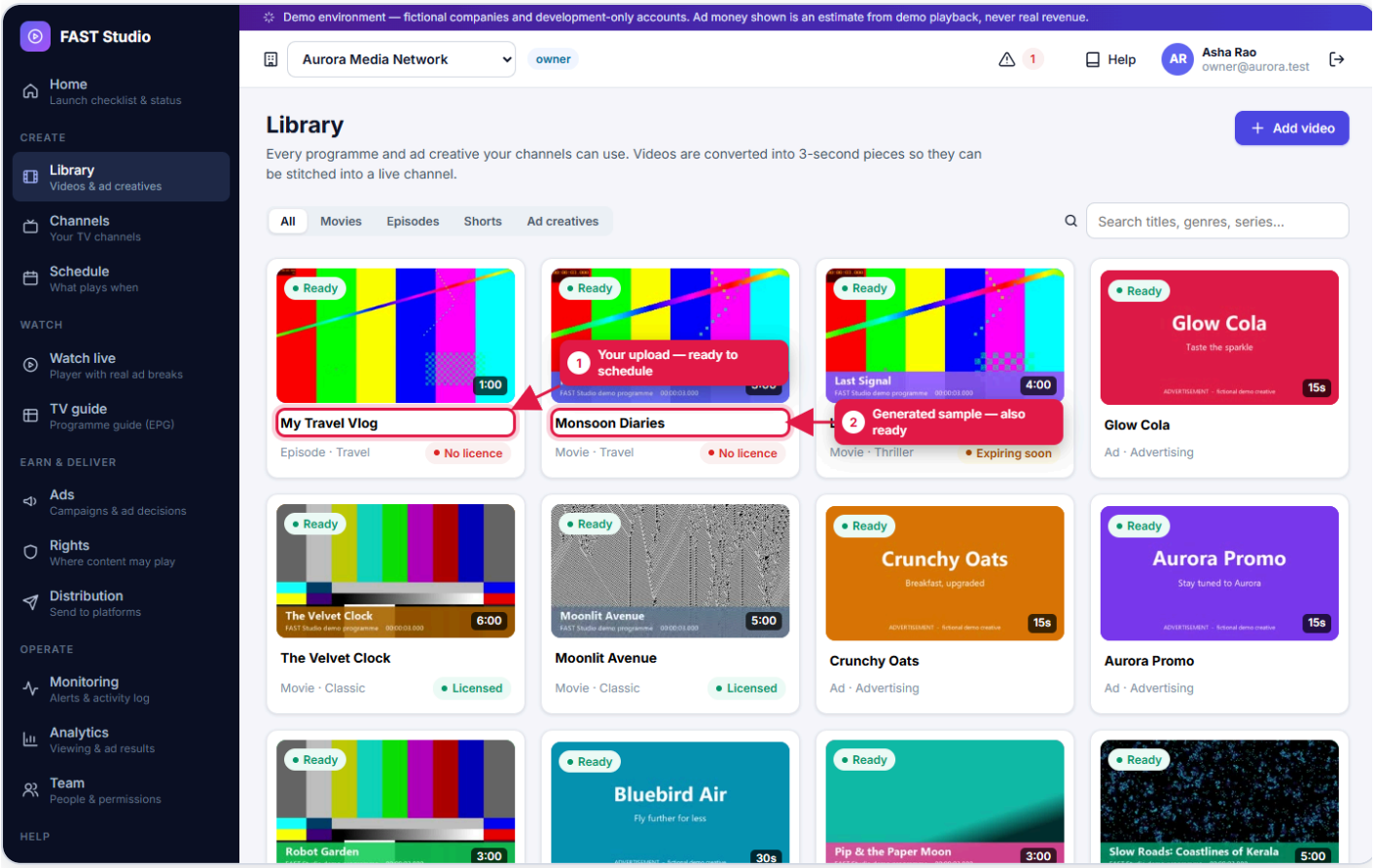
1. **Upload a file** tab.
2. **Your video file** — MP4, MOV, MKV, WEBM or M4V.
3. **Upload** — sends the file and starts conversion.



1. The video is being **converted right now**.
2. Status **Processing**. The page refreshes itself and turns *Ready* when done (a few seconds per minute of video).



1. **Preview** — you're watching the *converted* stream, exactly as the channel will use it.
2. **What the server produced**, for example "20 segments of 3 s in 3 qualities (360p, 540p, 720p)". (The screenshot shows an older single-quality result.)
3. **Ad-break point** — pause where an ad break would fit (for example a scene change) and press **Add at current time**. Mid-roll ad breaks will go here.
4. **Licence summary** — a new upload has *no licence*, so it is **blocked everywhere** until you add one (chapter 8).



1. Your uploaded video, ready to schedule.
2. The generated sample, also ready.

Behind the scenes — what happens to an uploaded video

1. **Safe storage.** The browser sends the file to the server, which checks the type and saves it under a **name the server chooses**, never your file name. That prevents path tricks. Files go in a folder belonging to your company.
2. **Conversion.** A background job runs **ffmpeg** once and produces **three qualities**: 360p, 540p and 720p, all H.264 video with AAC audio. This is an *ABR ladder*. Each quality is cut into **pieces of exactly 3 seconds**, with a fresh keyframe (a complete picture) at the start of each piece, and a *master playlist* lists the three. A viewer's player starts with a quality that suits its connection and can switch at any piece boundary, because every quality has identical timing.

Why so strict? A live channel is made by playing pieces from *different* files one after another: programme, ad, programme. That only works smoothly if every piece from every file has the same format and starts cleanly. The fixed 3-second grid also makes the live clock simple (chapter 12).

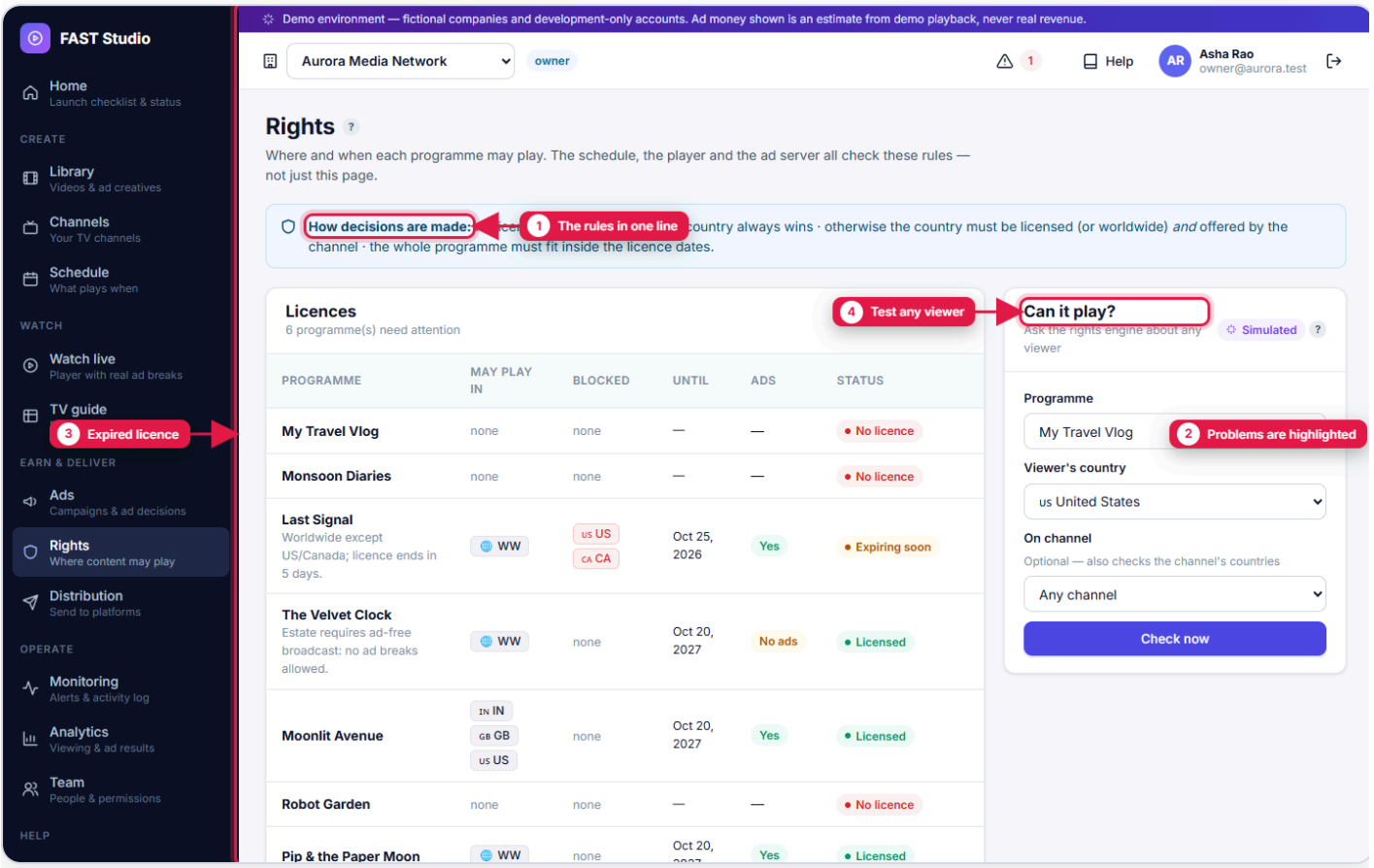
3. **Thumbnail.** A frame is saved as the thumbnail.
4. **Ready.** The video becomes *Ready* with its exact duration. If conversion fails, it becomes *Failed* with the reason, and you can press **Retry**.
5. **Restarts.** If the server restarts mid-conversion, the job starts again by itself.

Break points are rounded to the nearest 3-second boundary, because a stream can only switch between pieces, never in the middle of one.

---

8. Rights

A **licence** (rights policy) says **which countries** a video may play in, **during which dates**, and **whether ads may run during it**.

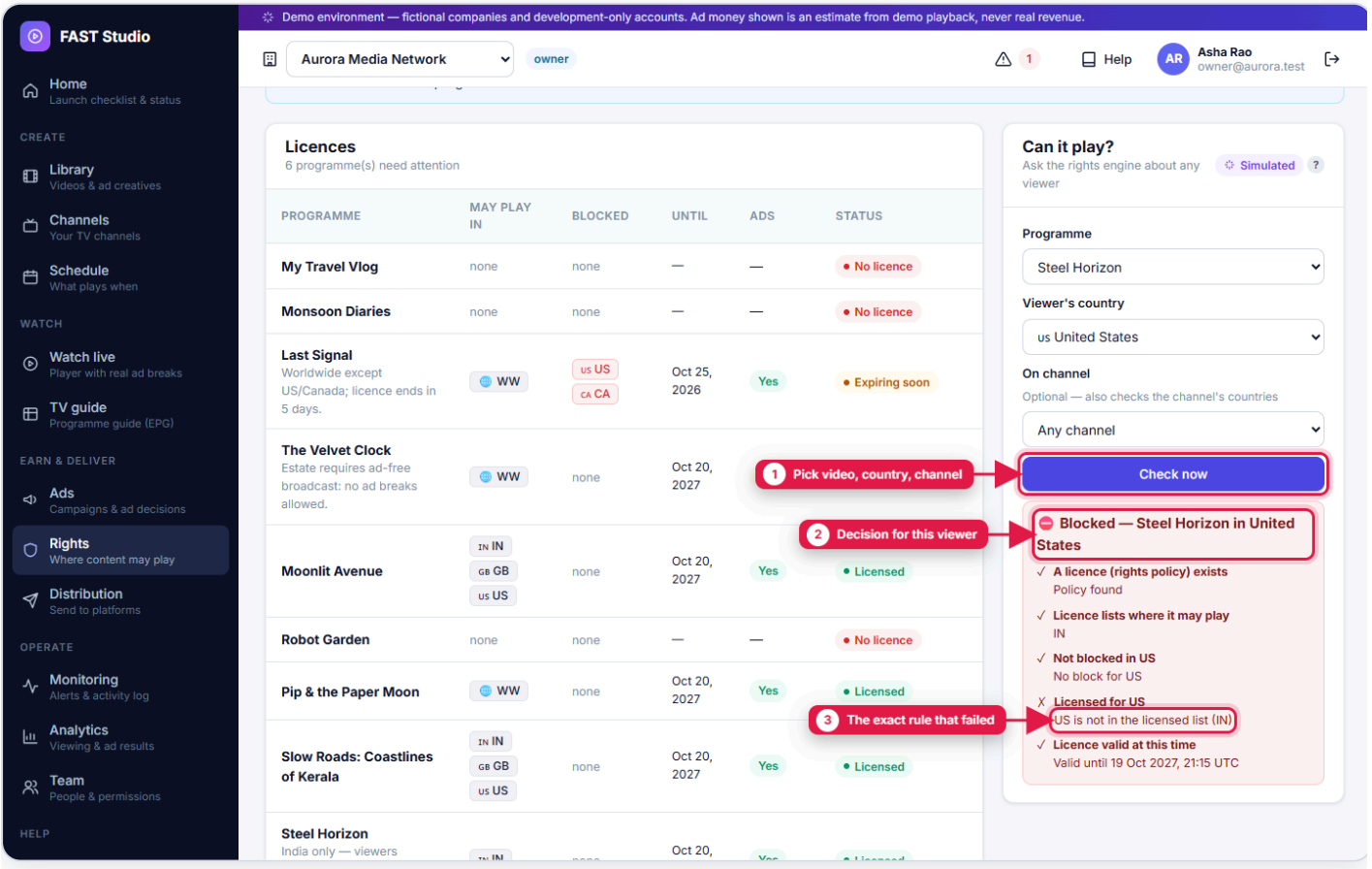


1. **The rules in one line** — see the box below.
2. **Problems highlighted** — for example *Incomplete licence*: the distributor hasn't confirmed countries yet, so the video is blocked.
3. **Expired licence** — can no longer be scheduled.
4. **Can it play?** — test any viewer.

The rules, in order:

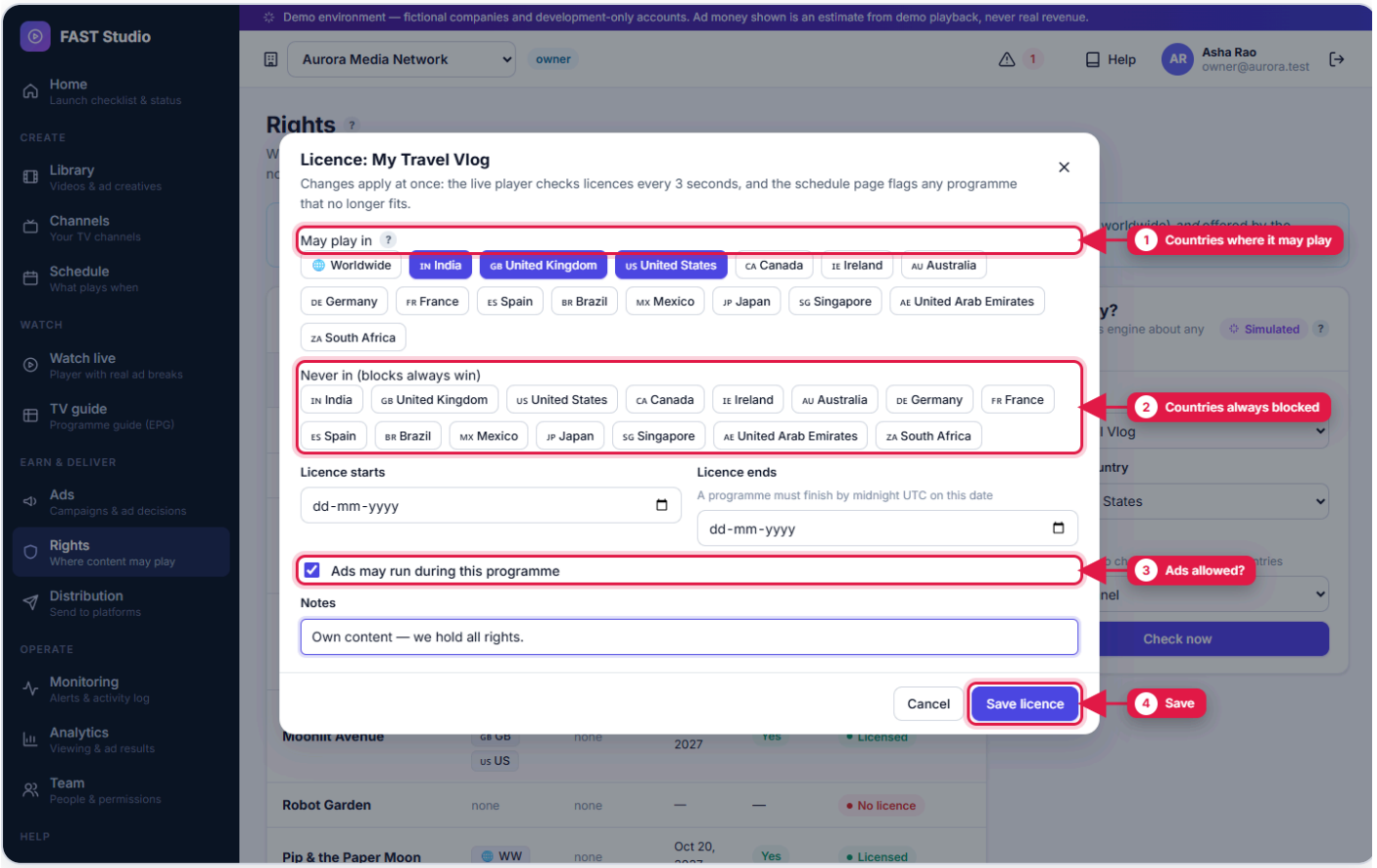
1. No licence → **blocked**. Missing information never means permission.
2. A **blocked country always wins**, even over "worldwide".
3. Otherwise the country must be **licensed** (or worldwide), **and** the channel must be **offered** in that country.
4. The **whole programme** must fit inside the licence dates. Ending exactly at the end date is fine; one second later is not.

"Can it play?"



1. Pick a video, a viewer's country, and optionally a channel.
2. The decision for this viewer: *Steel Horizon* in the United States is **blocked**.
3. The exact rule that failed: "US is not in the licensed list (IN)". Every rule is shown with ✓ or ✗.

Editing a licence



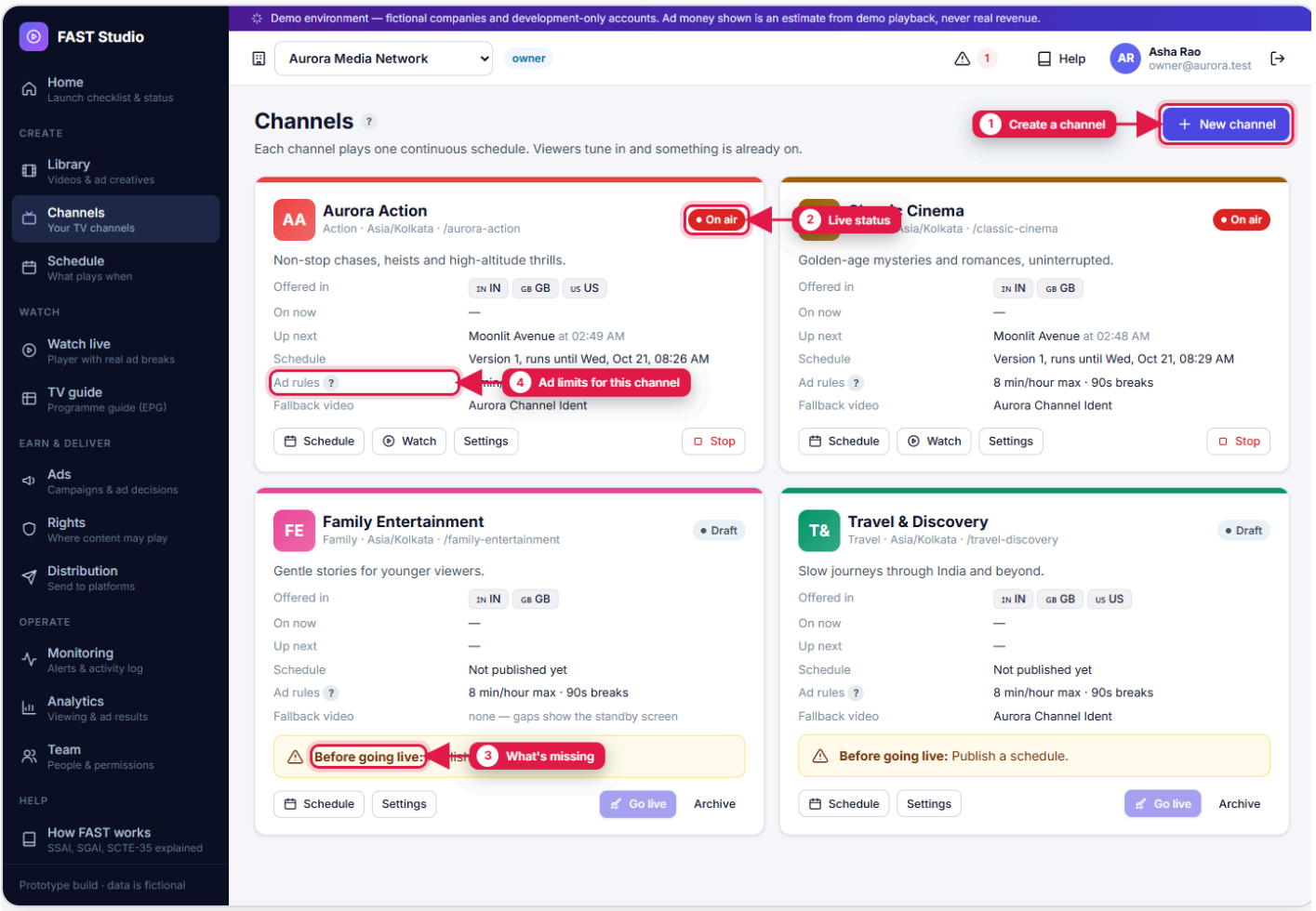
1. **Countries where it may play** — or *Worldwide*.
2. **Countries always blocked.**
3. **Ads allowed?** — some contracts require ad-free broadcast. Then FAST Studio won't put ad breaks inside that programme.
4. **Save.** The change takes effect immediately everywhere.

Behind the scenes — where licences are enforced (three places, not one)

1. **When scheduling:** the Schedule page checks every programme against every country the channel is offered in. Programmes blocked everywhere are a *must fix*. Those blocked in only some countries are flagged as regional blackouts.
2. **While streaming:** for every 3-second piece, the playout checks the licence *for that particular viewer's country*. A blocked viewer gets a "Not available in your region" screen instead (screenshot 32). It's enforced inside the stream itself, not by hiding a button.
3. **In ad decisions:** if a programme's licence forbids ads, its breaks stay empty.

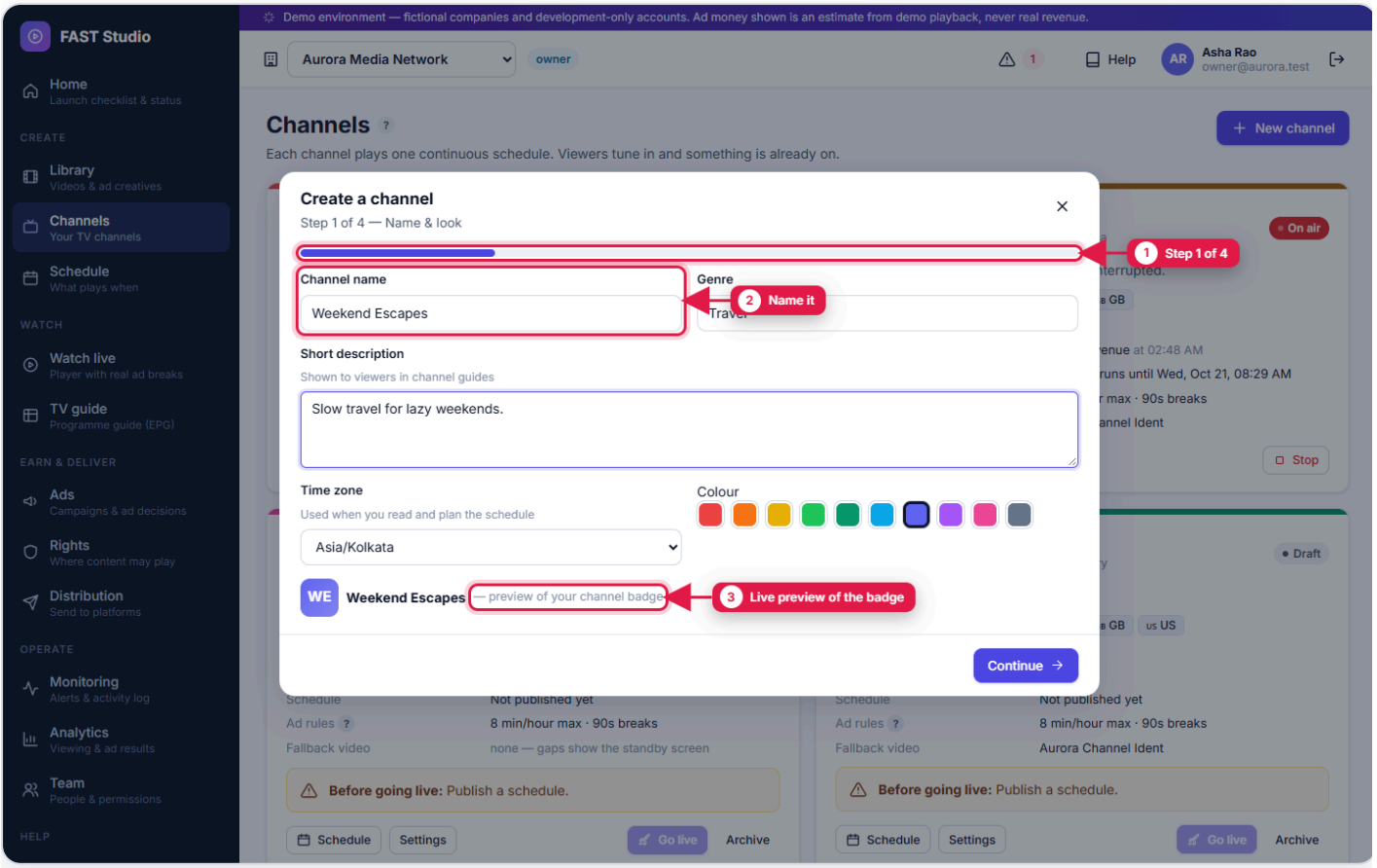
In this prototype you *choose* the viewer's country to demonstrate this. A real service reads it from the viewer's internet address at the network edge, and the stream links would be signed so they can't be shared.

9. Channels

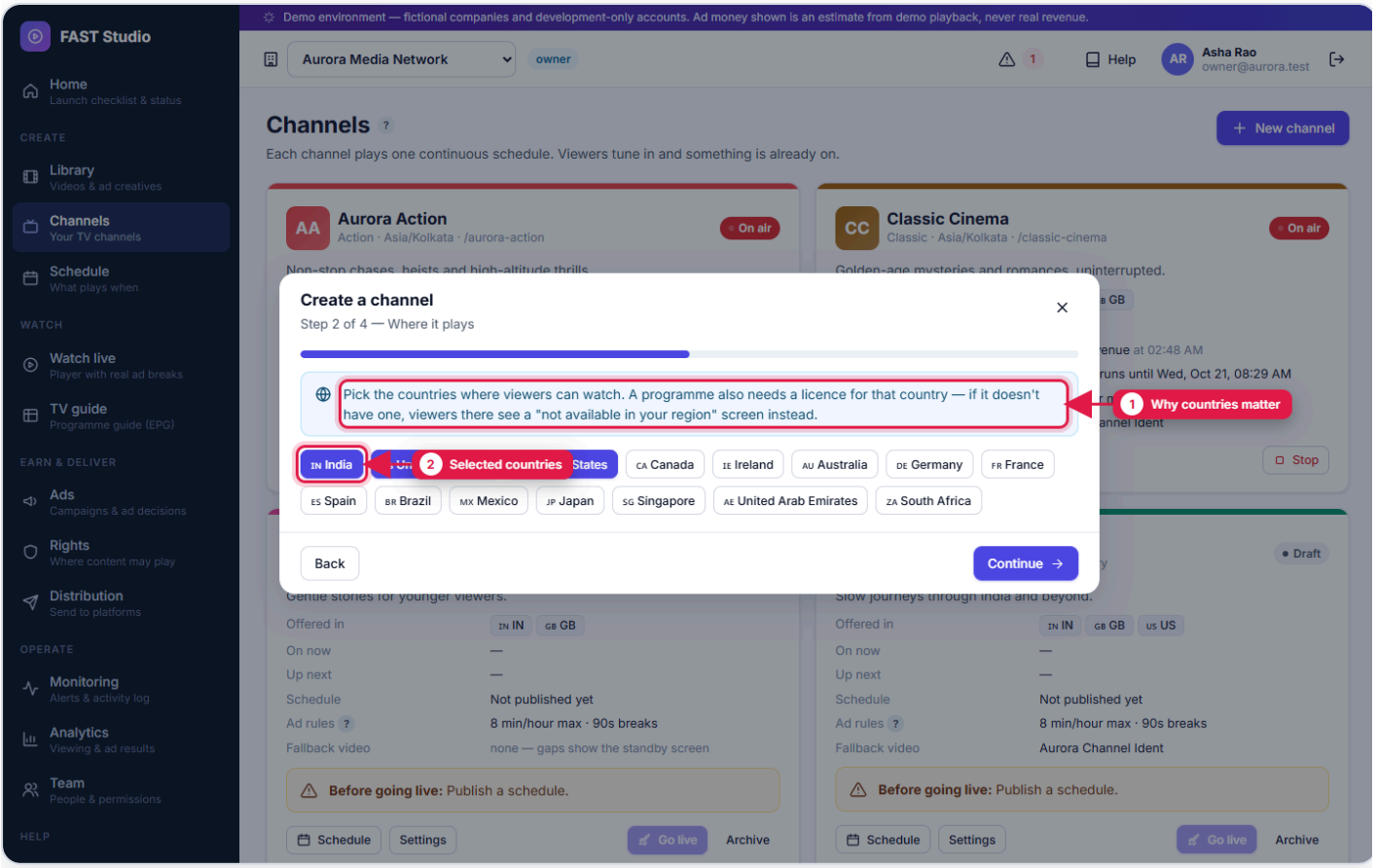


1. **New channel** — opens the wizard.
2. **Live status** — *On air*, *Draft* or *Stopped*.
3. **What's missing** before a channel can go live, in plain words.
4. **Ad rules** — maximum ad time per hour and the length of each break.

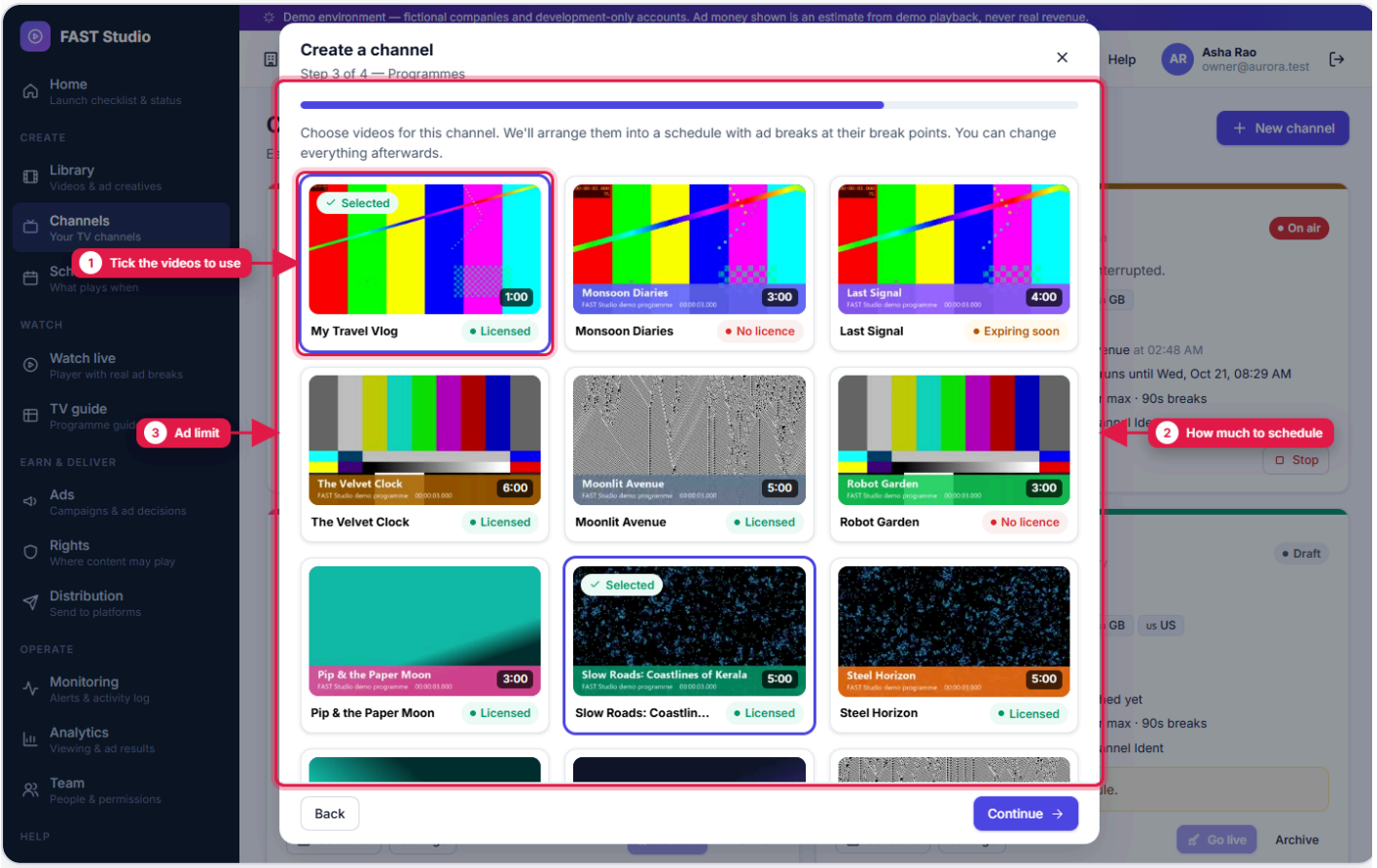
The four-step wizard



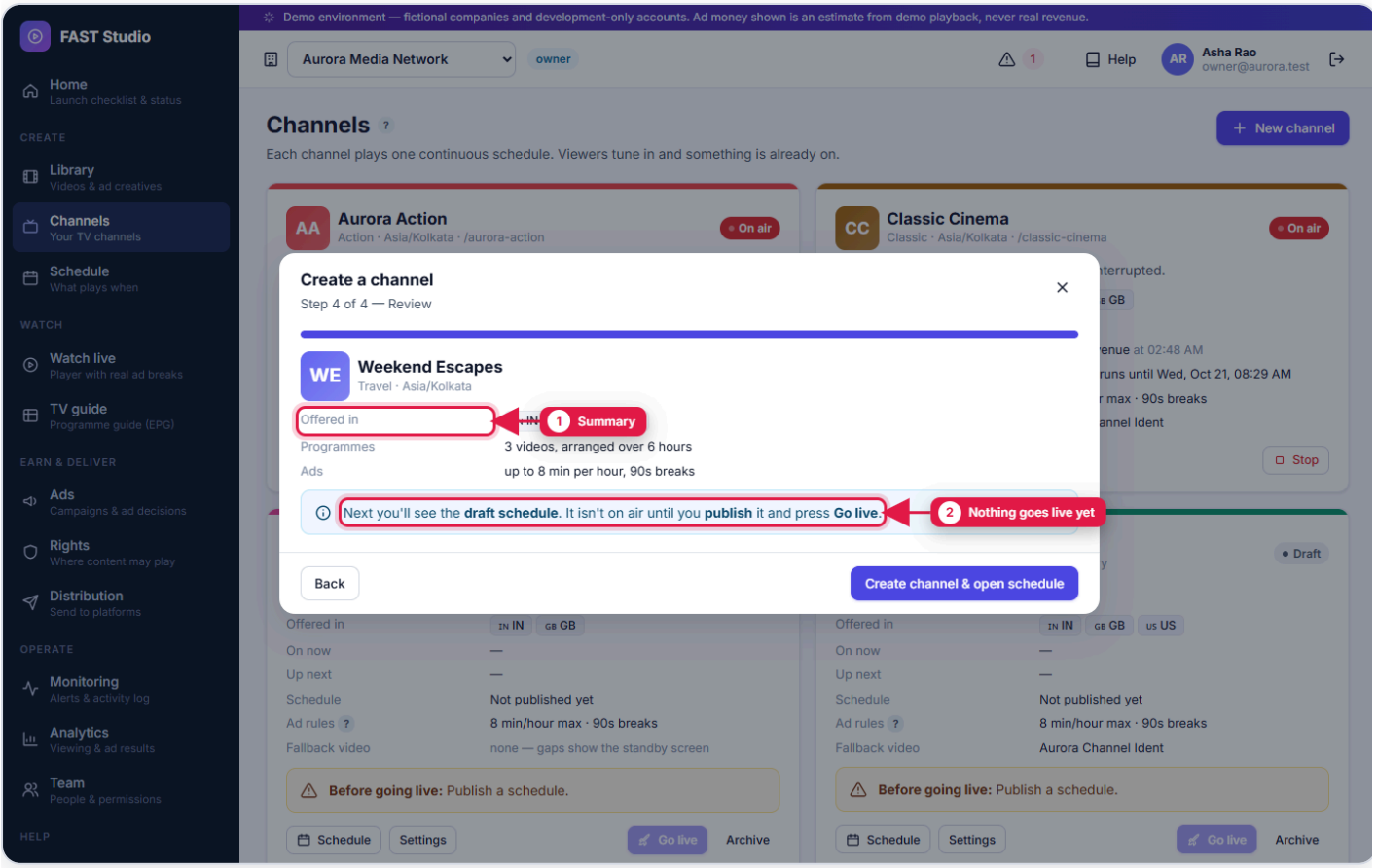
1. **Step 1 of 4.**
2. **Name it** (plus genre, description, time zone).
3. **Live preview** of the channel badge in your chosen colour.



1. **Why countries matter** — explained right on the page.
2. **Selected countries** — where viewers can watch this channel.



1. **Tick the videos** to use. Each card shows its licence status.
2. **How many hours** to fill straight away.
3. **Ad limit** per hour.



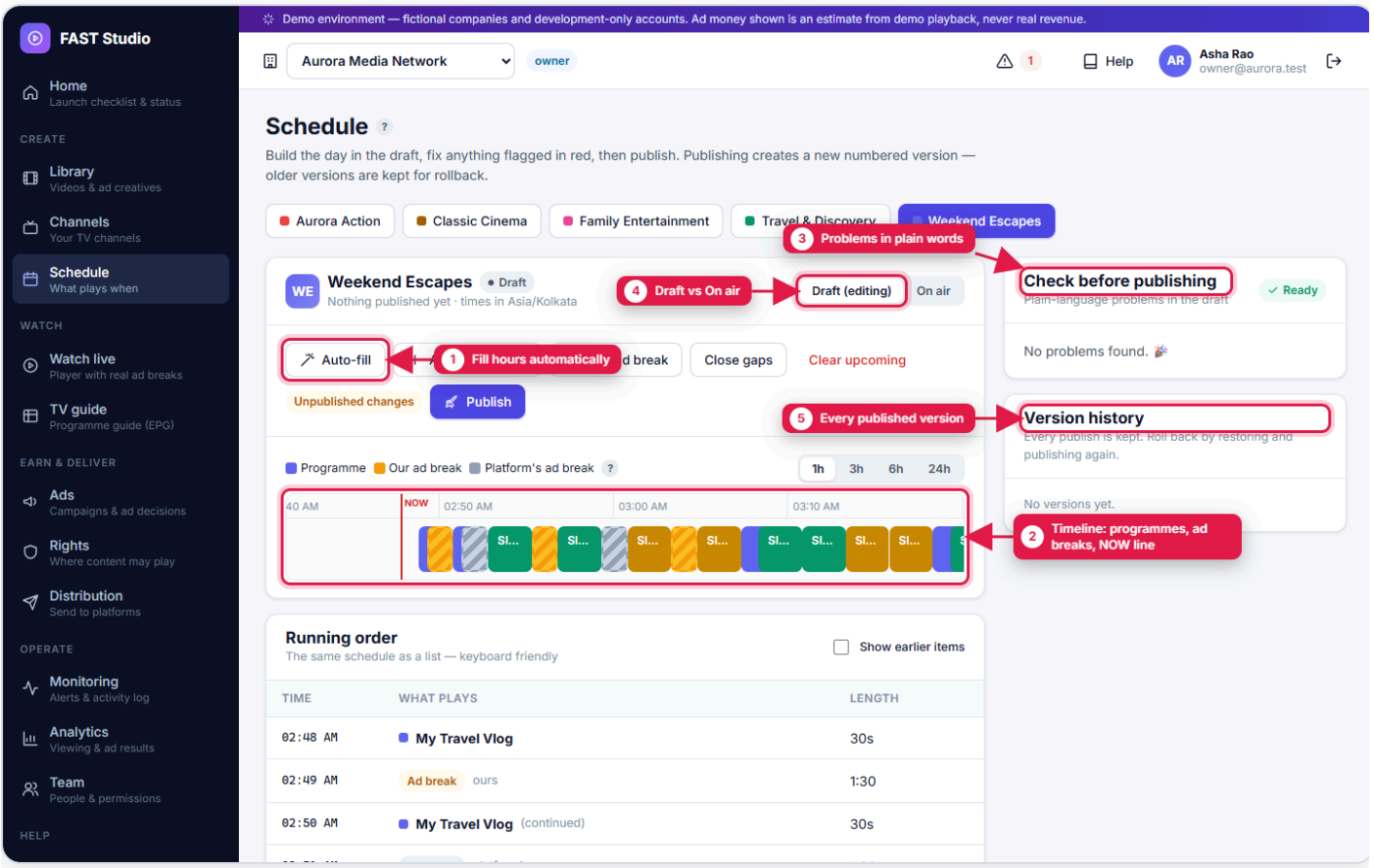
1. **Summary** of everything chosen.
2. A reminder that **nothing goes live yet**: you'll see a *draft* schedule first.

> **Behind the scenes:** creating a channel stores its settings and gives it a short web name (for example `weekend-escapes`), unique within your company. The server checks your plan's channel limit and creates an empty **draft** schedule. If you picked videos, it runs **auto-fill** (chapter 10) to build the first hours. Nothing reaches viewers until a schedule is **published** *and* the channel is **live**.

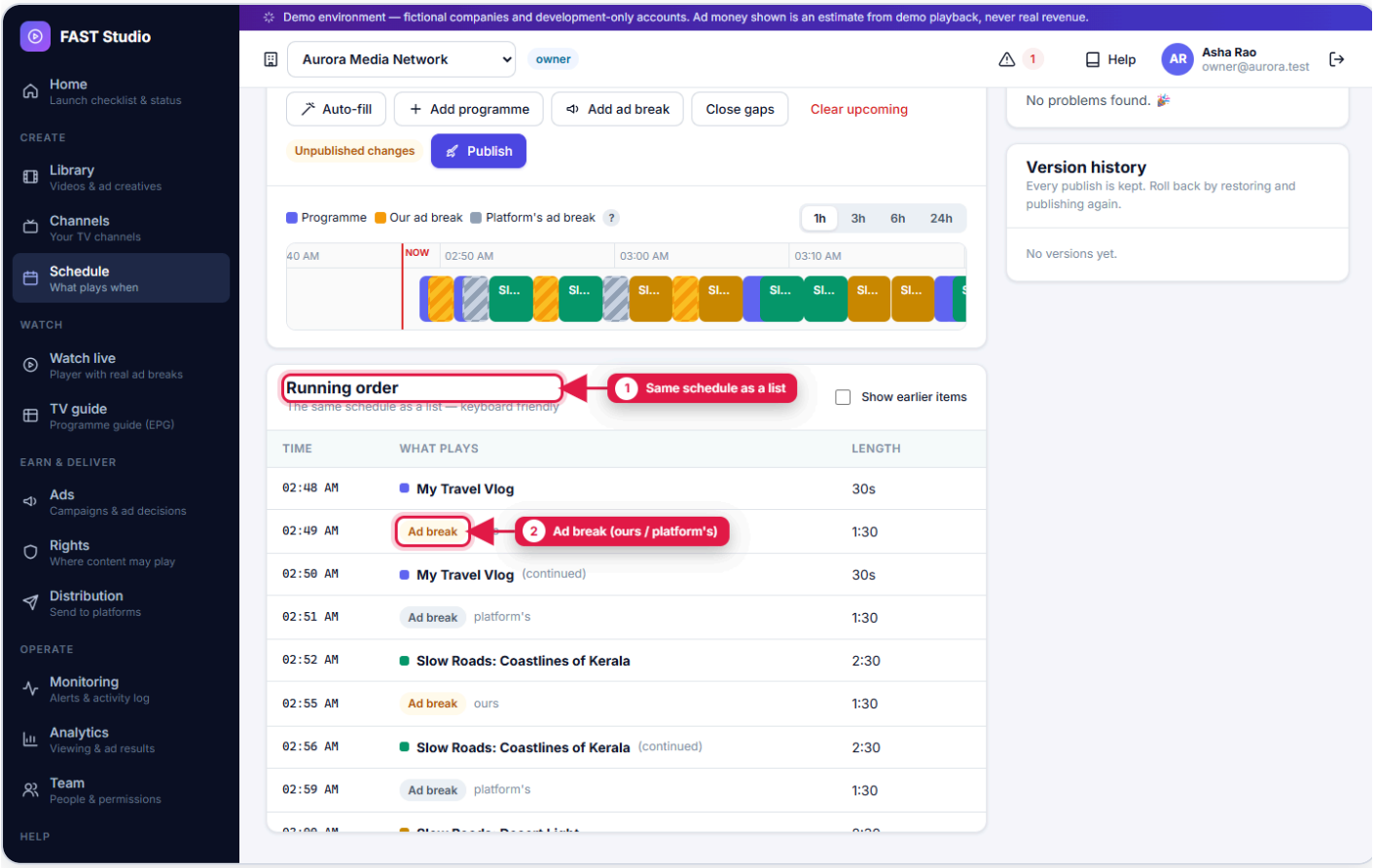
---

10. Schedule

The schedule decides what plays at every second. You edit a **draft**; viewers only ever get a **published version**.

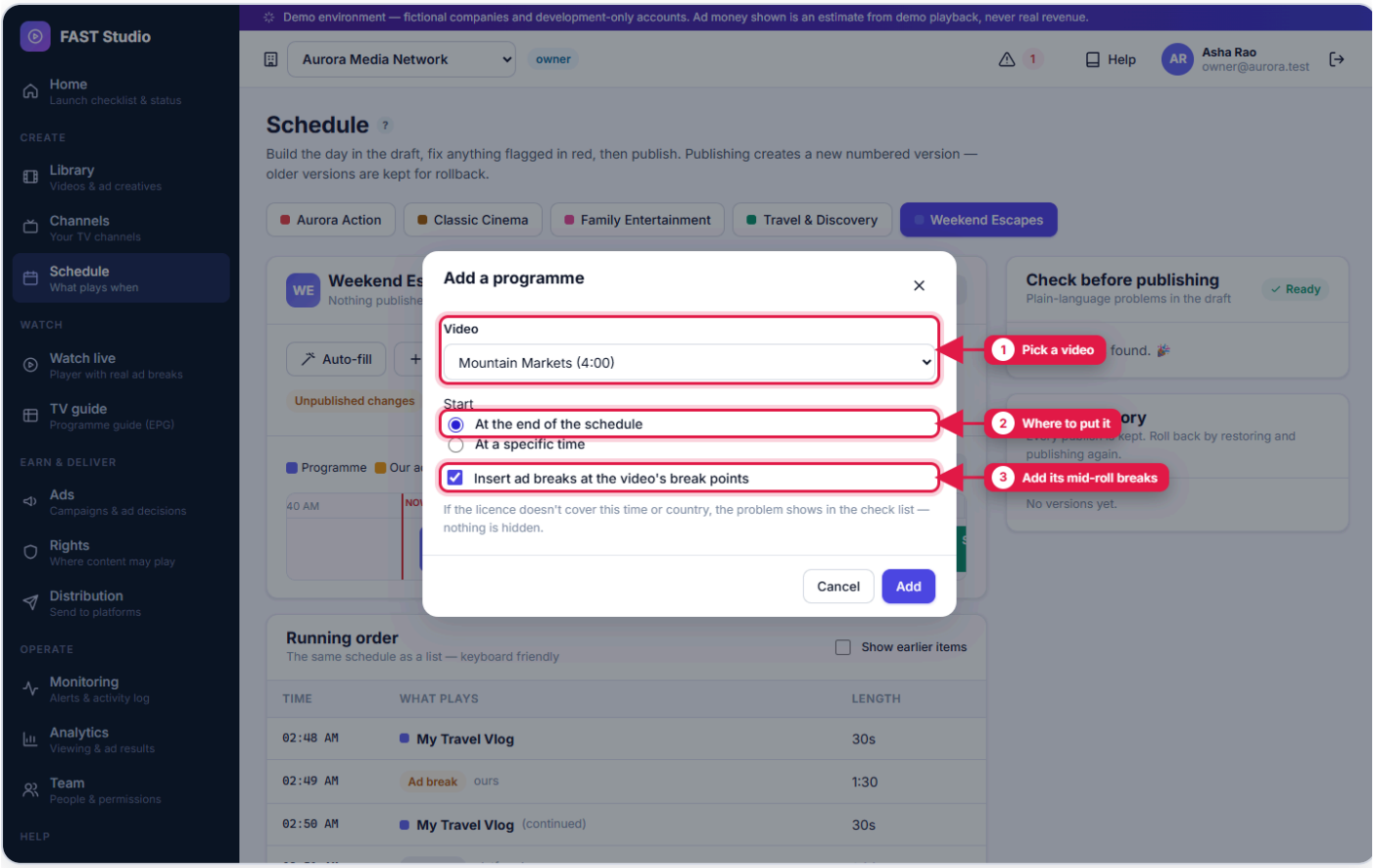


1. **Auto-fill** — builds hours of programming in one click.
2. **Timeline** — coloured blocks are programmes, **yellow striped** blocks are *your* ad breaks, **grey striped** blocks are the *platform's* ad breaks, and the red **NOW** line is the current time. Zoom with 1h / 3h / 6h / 24h.
3. **Check before publishing** — problems in plain words, with the fix.
4. **Draft / On air** — switch between what you're editing and what viewers get.
5. **Version history** — every published version is kept.

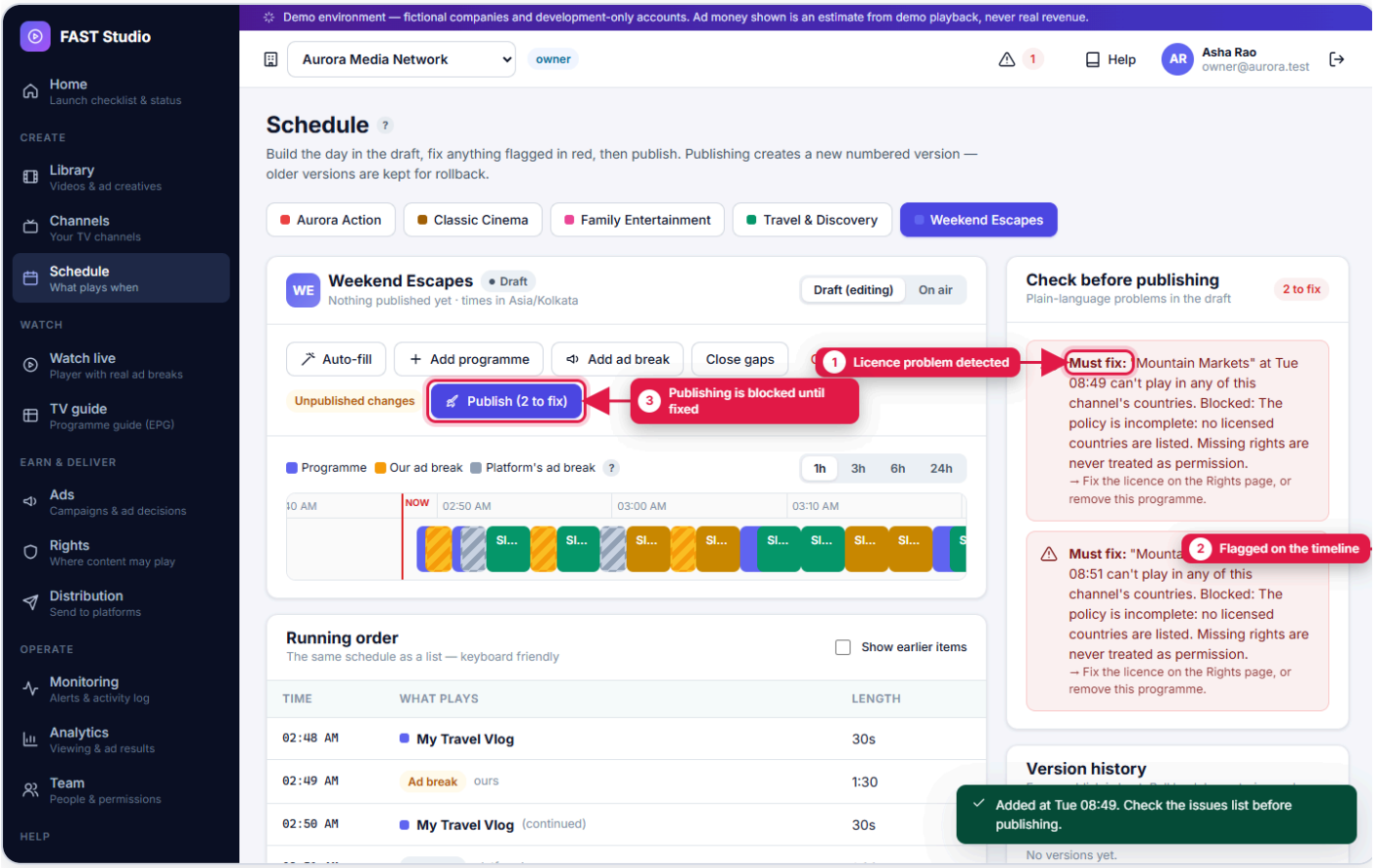


1. The **same schedule as a list**, usable from the keyboard.
2. **Ad breaks**, marked *ours* or *platform's*.

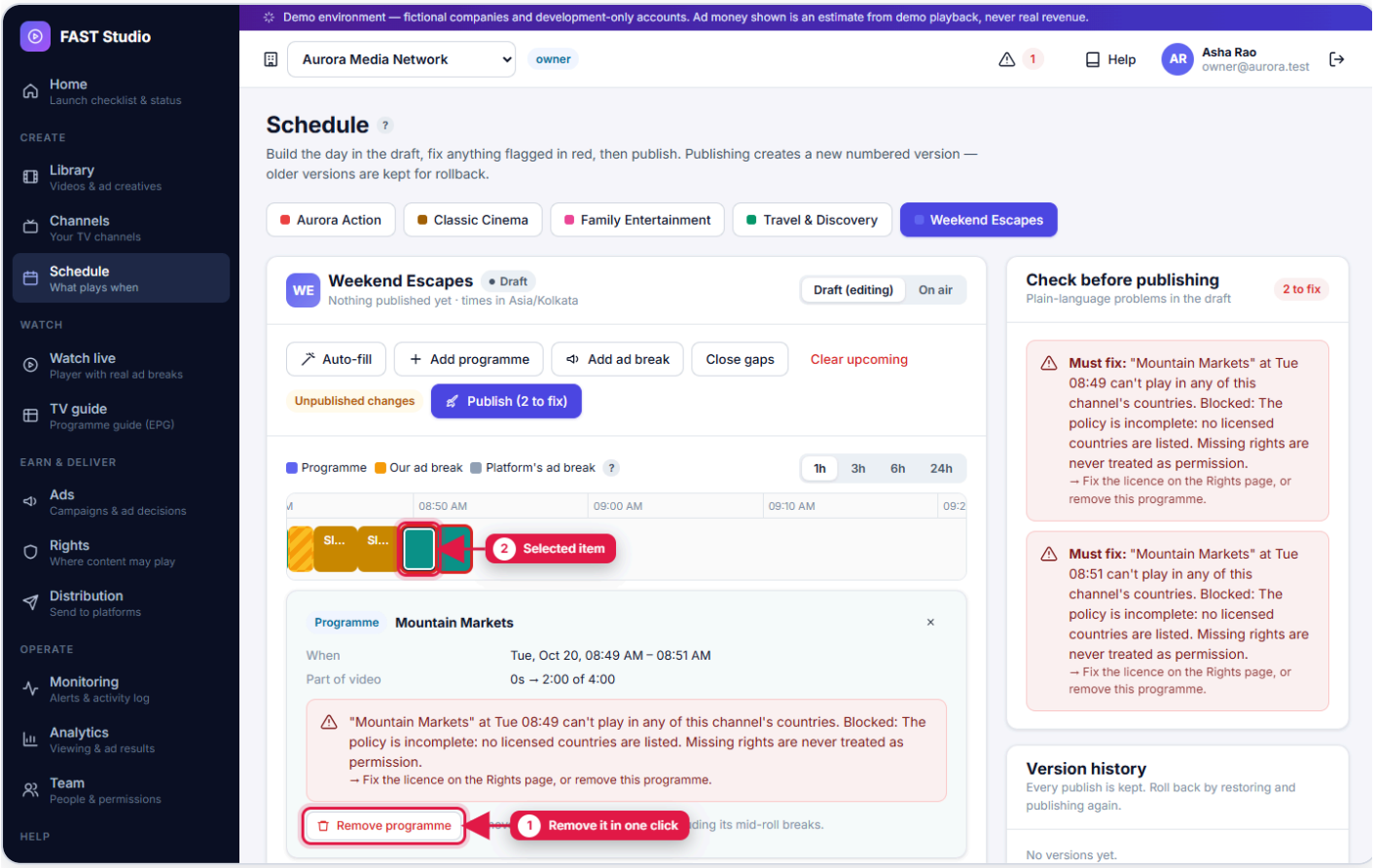
Adding a programme, and catching a problem



1. **Pick a video.** Here we deliberately pick *Mountain Markets*, whose licence is incomplete.
2. **Where to put it** — at the end, or at a specific time.
3. **Add its break points** as ad breaks automatically.

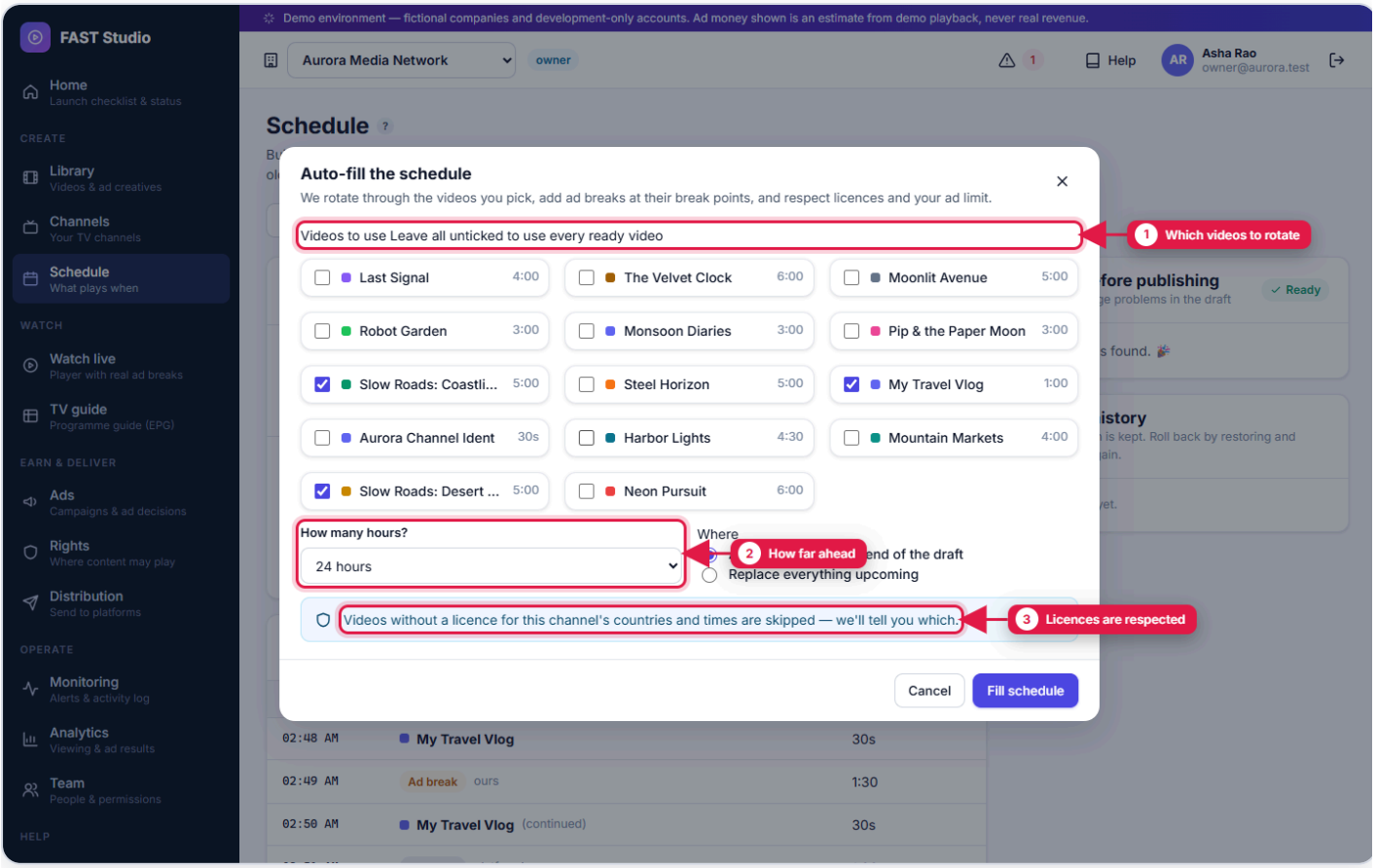


1. **Licence problem detected**, explained in plain words.
2. **Flagged on the timeline** with a red outline.
3. **Publishing is blocked** until it's fixed. A schedule with licence, overlap or ad-limit problems can never reach viewers.

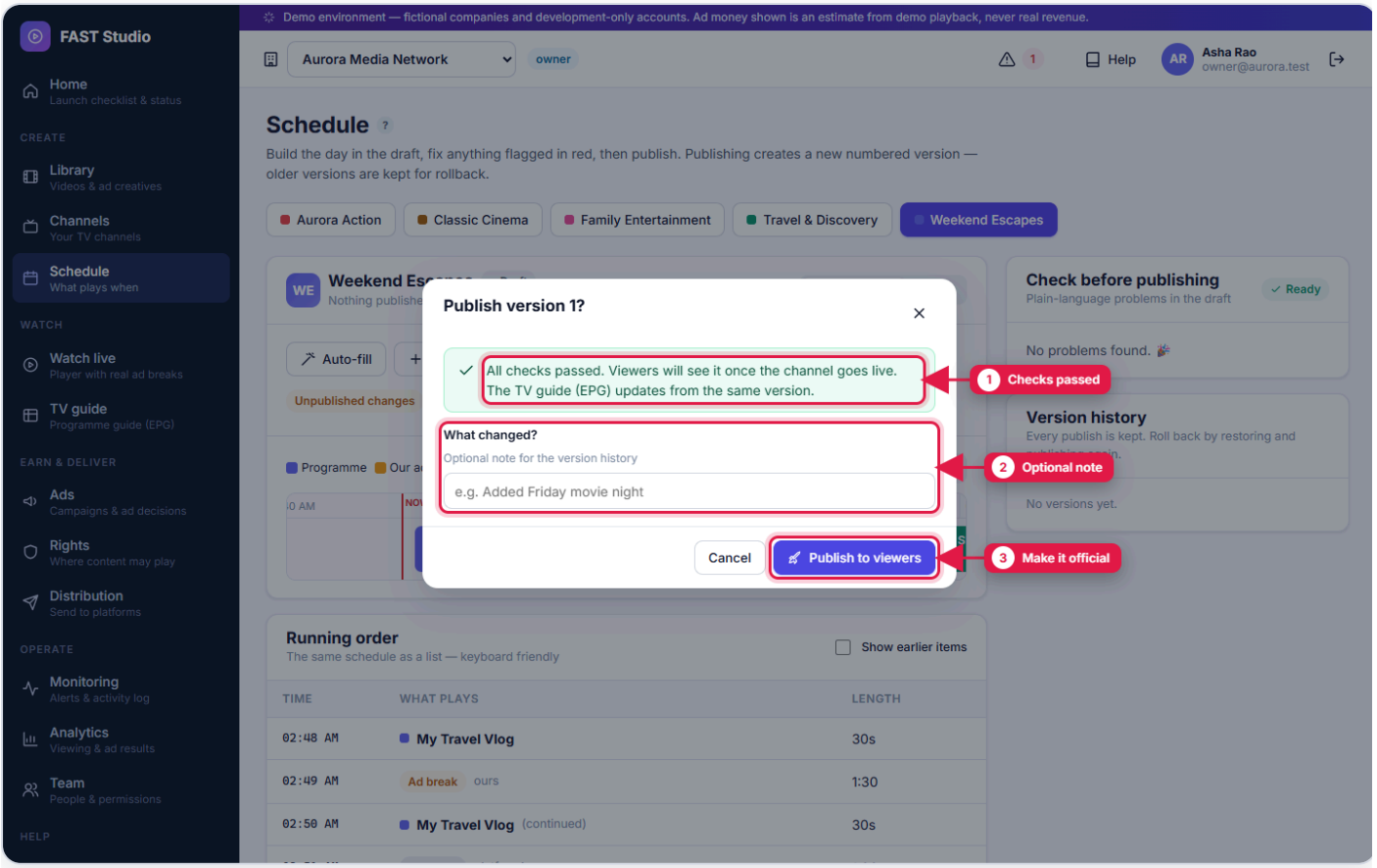


1. Clicking the problem selects the item. **Remove it** in one click (all its parts and its ad breaks go together).
2. The selected item on the timeline.

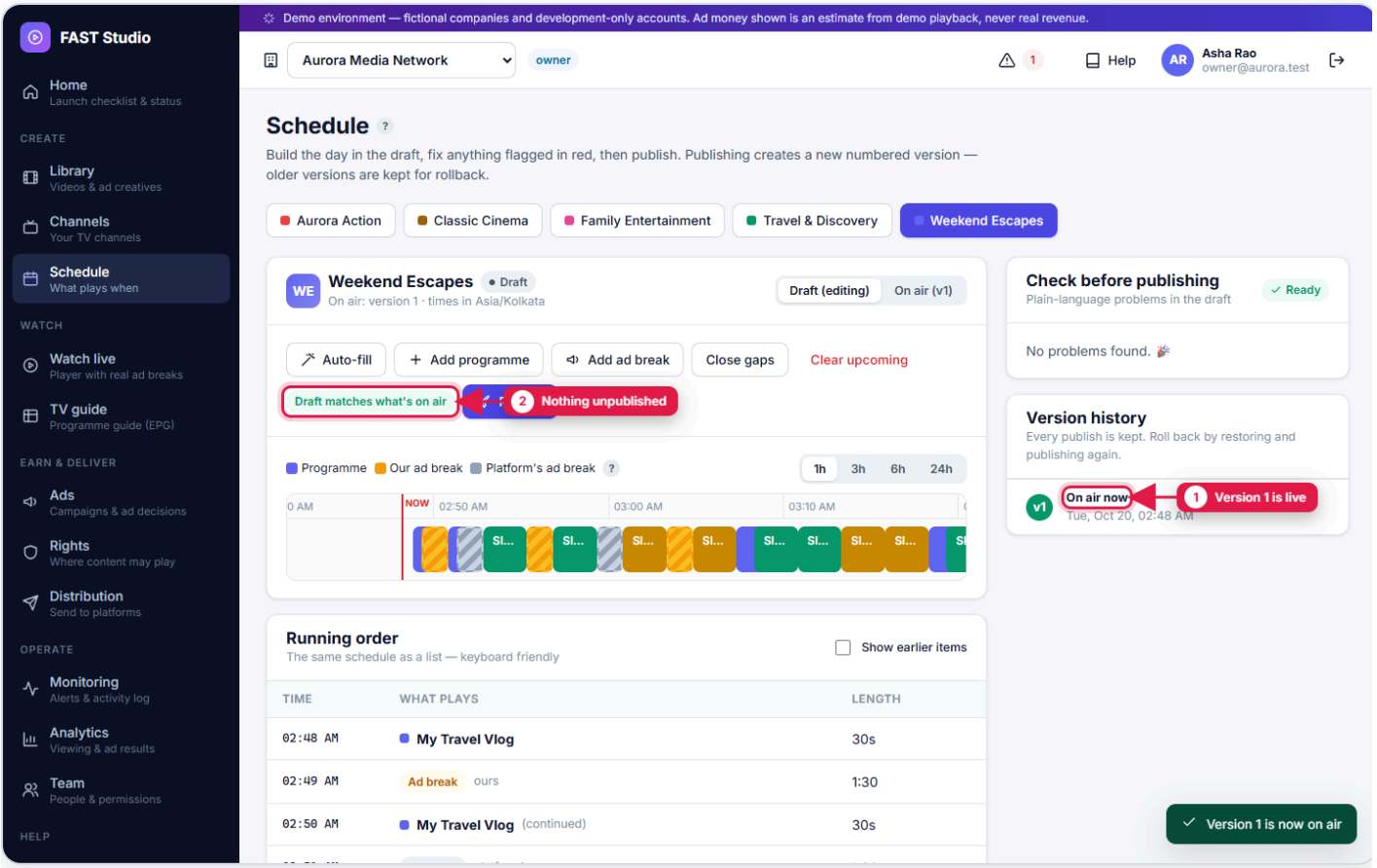
Auto-fill and publishing



1. **Which videos** to rotate.
2. **How far ahead** to fill.
3. **Licences are respected** — anything not allowed is skipped, and you're told which.



1. **All checks passed.**
2. An optional **note** for the history ("Added Friday movie night").
3. **Publish to viewers.**



1. **Version 1 is on air.**
2. The draft now matches what's on air.

Behind the scenes — the scheduling engine

Ad breaks are real timeline items. A 5-minute film with a break point at 2:30 becomes three items: *film part 1 (2:30) → ad break (90 s) → film part 2 (2:30)*. The ad break has a fixed length. Later, the ad system fills it. If the ads are shorter, the remaining seconds show a neutral "slate", so the channel never drifts off its timetable.

What the checks look for:

- Two items overlapping.
- Gaps (the standby screen would show).
- Videos not ready.
- Licence problems for each country the channel is offered in.
- Ad breaks inside a programme whose licence forbids ads.
- Too much advertising in any clock hour.
- The schedule running out within 6 hours.
- Items not on the 3-second grid.

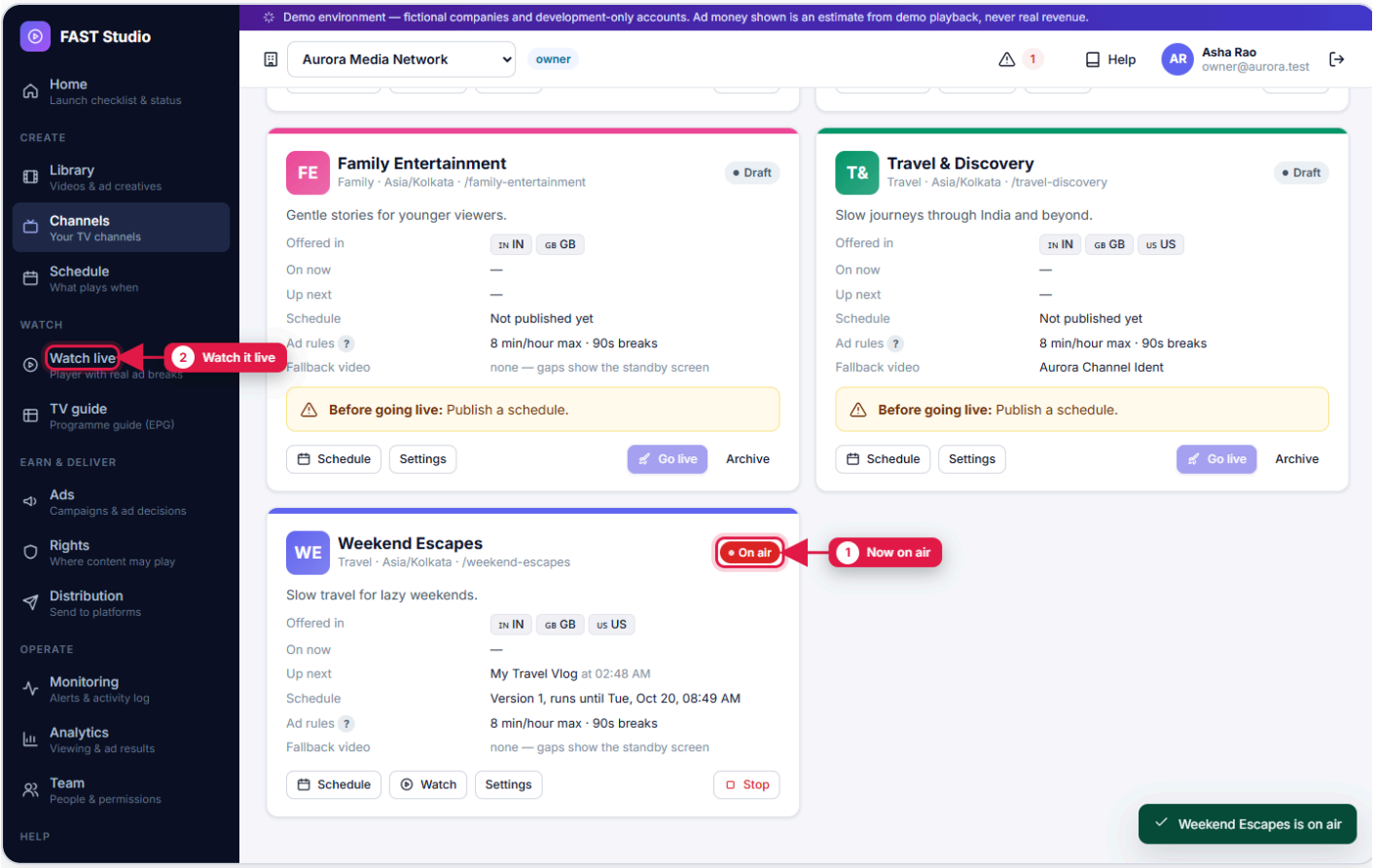
Problems are graded *Must fix* or *Heads up*.

How auto-fill works:

1. It takes the chosen videos in turn.
2. It skips any video whose licence doesn't cover the channel's countries for that exact time.
3. It splits each video at its break points and inserts a break there.
4. It adds a break between programmes.
5. It stops adding breaks in an hour once that hour's ad limit is reached.
6. Breaks alternate between *ours* and *the platform's*. This models the common "inventory split" deal, where you and the platform each sell part of the ad time.

Two people editing at once: every draft has a revision number. If someone else changed the schedule after you opened it, your change is refused with "Someone changed this schedule…", and the page reloads instead of silently overwriting their work.

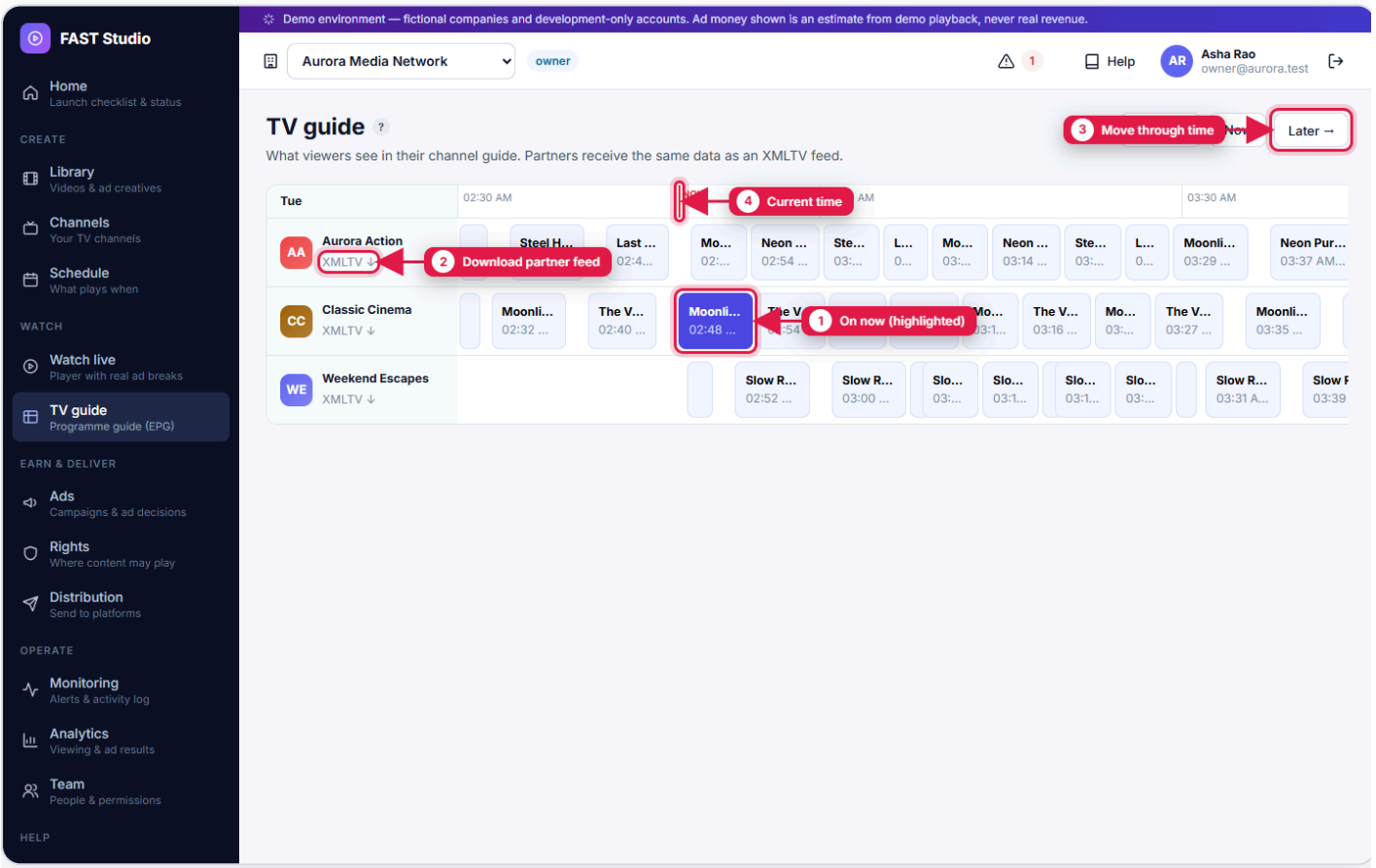
Publishing copies the draft into a new, **unchangeable** numbered version, inside a locked step so two publishes can't collide. **Rollback** means "restore an old version into the draft, then publish it again". It creates a new version, so history is never rewritten.



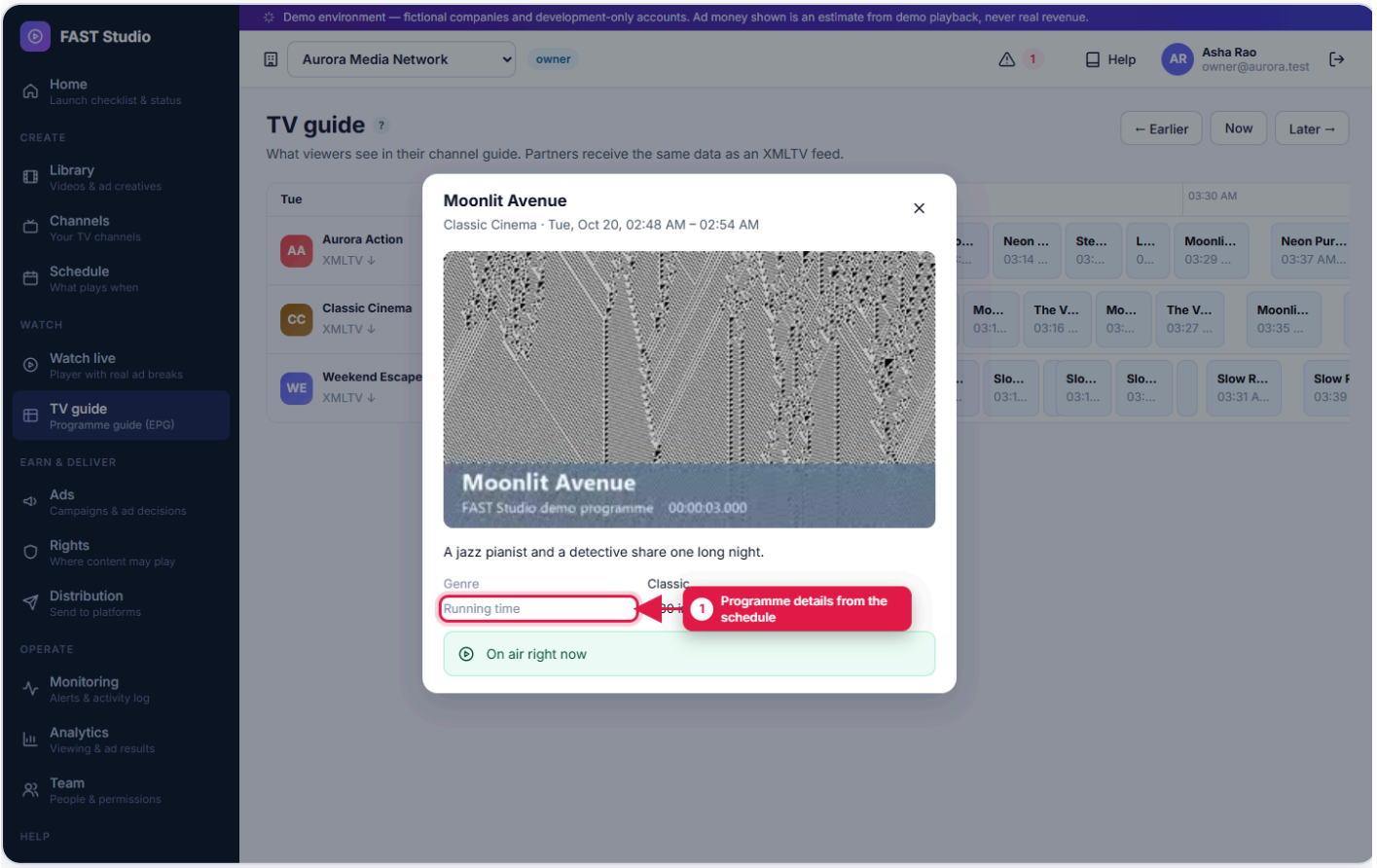
1. After publishing, **Go live** on the Channels page puts the channel **on air**.
2. **Watch** opens the live player.

---

11. TV guide



1. **On now**, highlighted, for every published channel.
2. **XMLTV** — download the guide in the standard file format partner platforms ingest.
3. **Move through time.**
4. **Current time** line.



1. Click any programme for details, including its running time *with* its ad breaks.

> **Behind the scenes:** the guide is **calculated from the published schedule**, never typed in separately, so it can't disagree with what plays. A film split into *part 1 → ad break → part 2* is shown as **one** programme from the start of part 1 to the end of part 2.

---

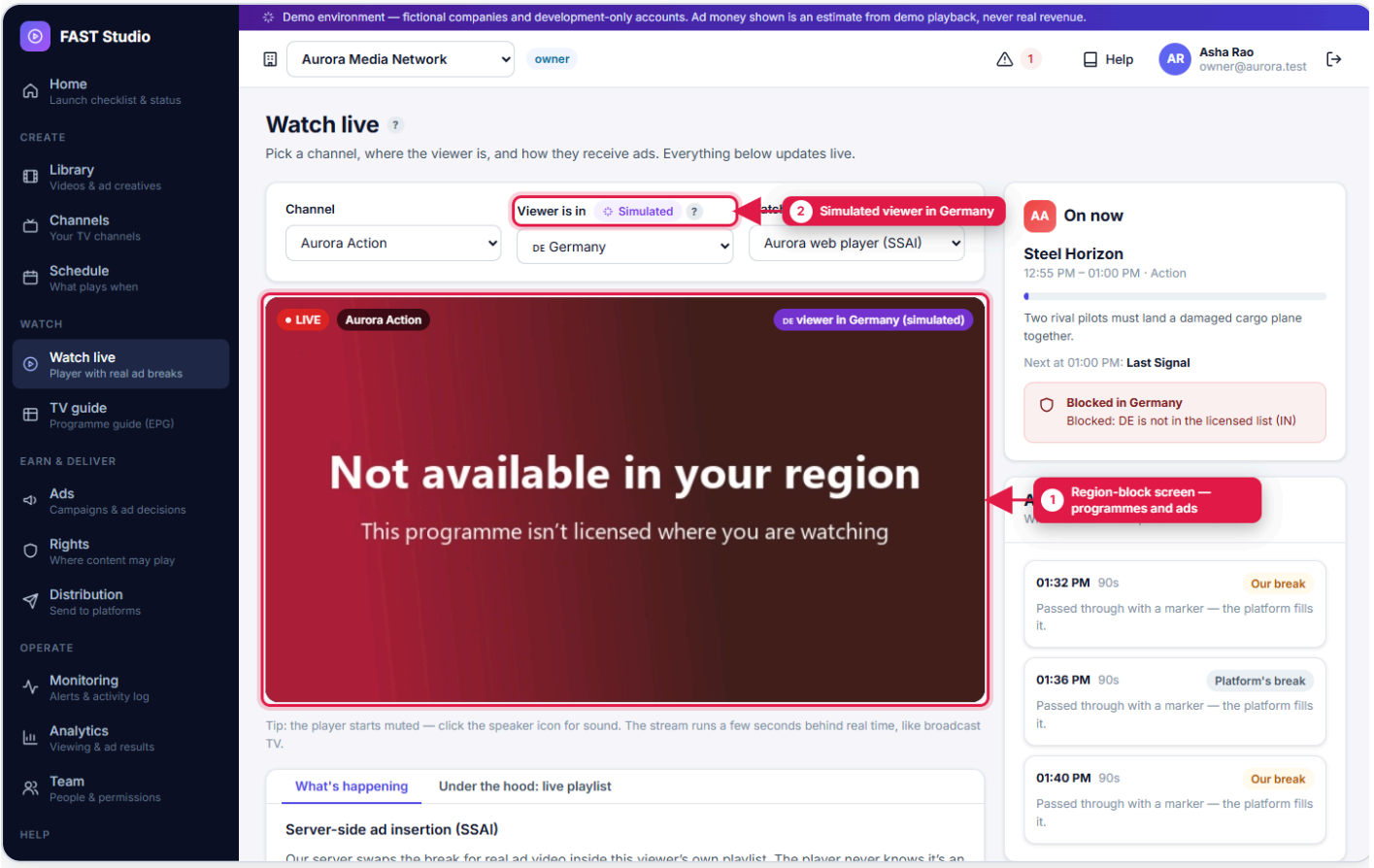
12. Watch live

This page plays the channel exactly as a viewer's app would. **It is a real live stream generated by FAST Studio**, not a recording.

The controls at the top:

- **Channel.**
- **Viewer is in** — a *simulated* country, to demonstrate rights.
- **Watching on** — which destination, and therefore which ad method:
  - **SSAI** web player
  - **SGAI** web player
  - **Partner TV** (markers only)

Geo-blocking, enforced in the stream

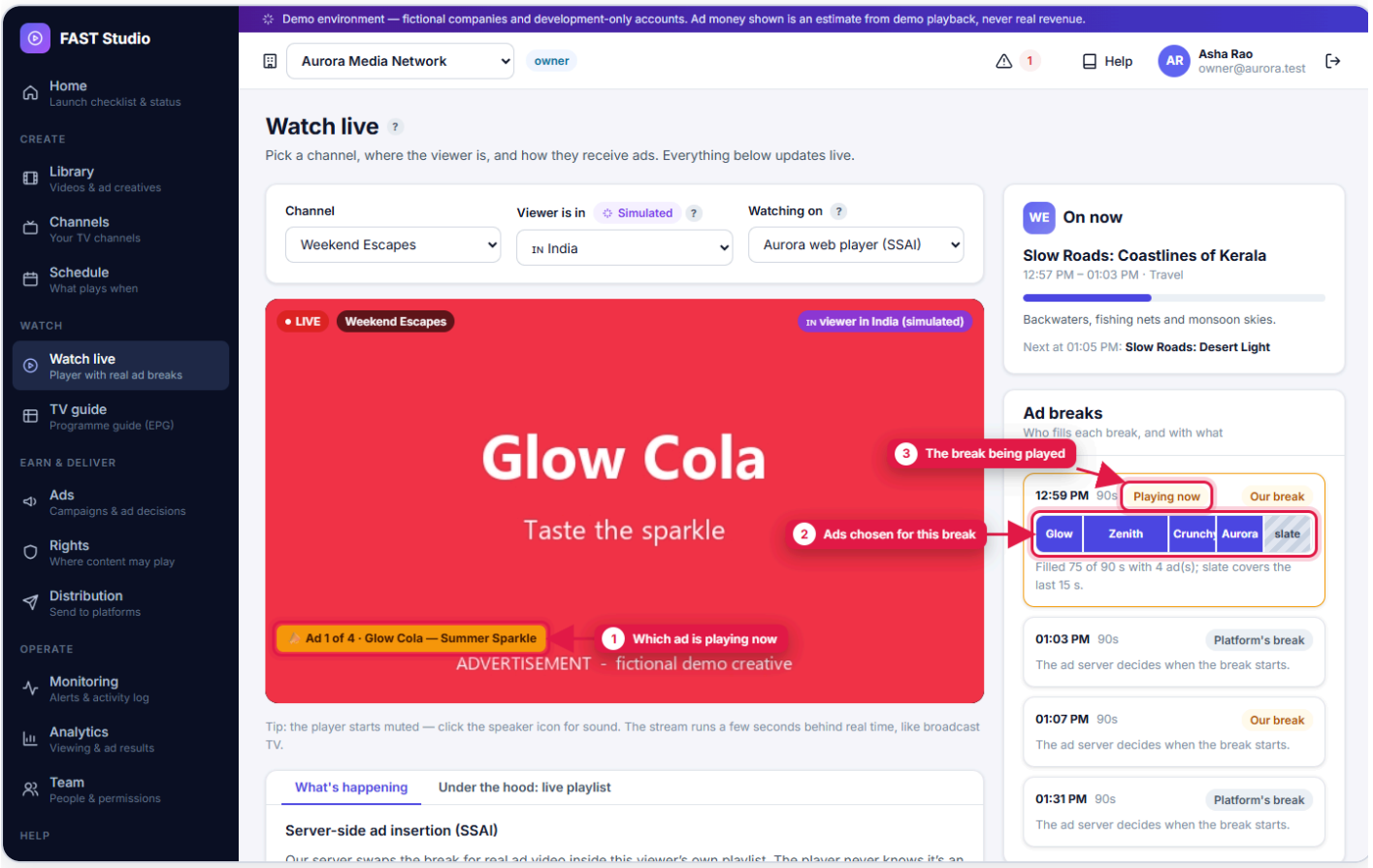


1. **Region-block screen.** Aurora Action isn't offered in Germany, so every piece of the stream is replaced with "Not available in your region": programmes **and** ad breaks. A blocked viewer gets no ads and no ad markers.

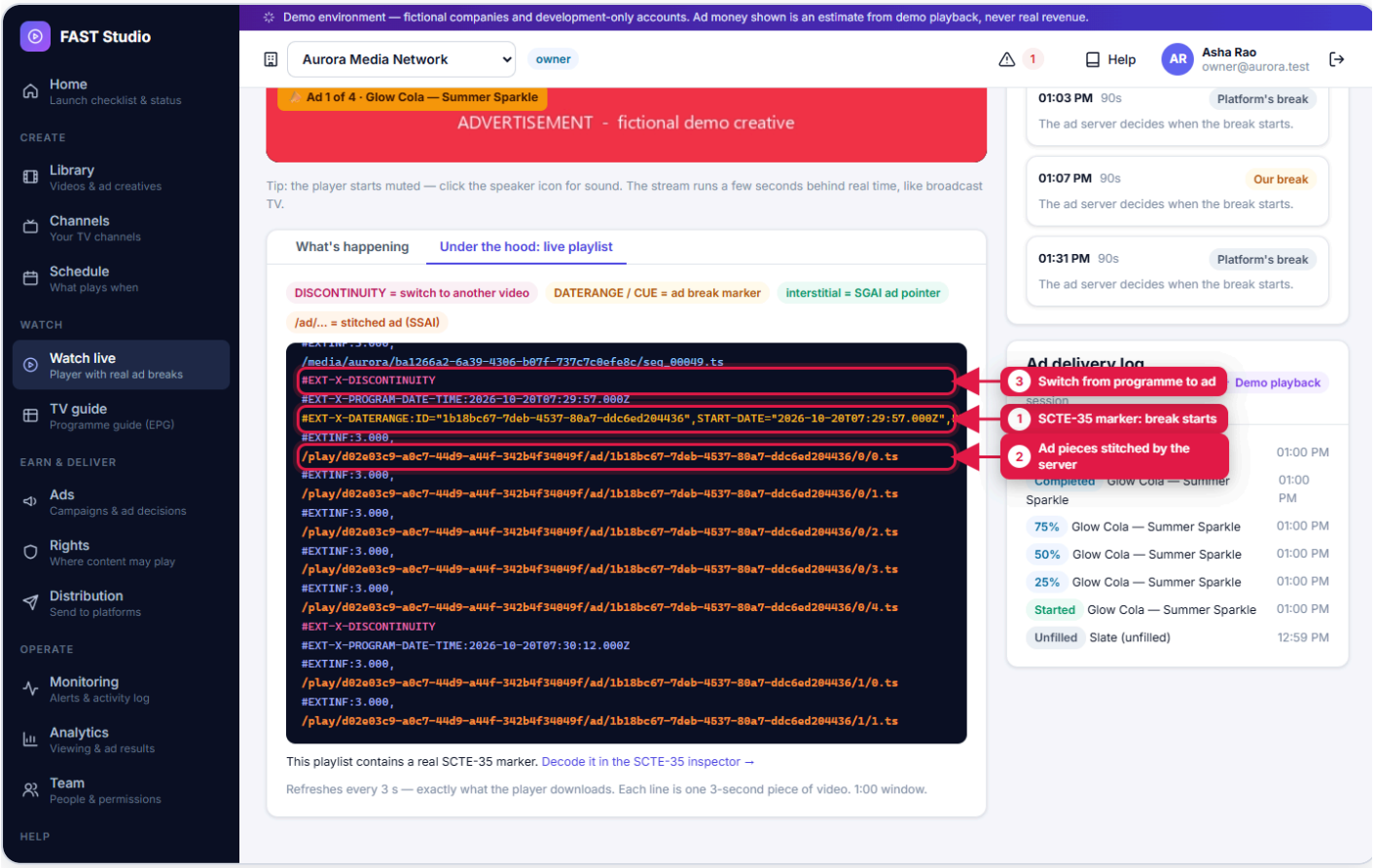
2. **Simulated viewer in Germany.**

If the channel *is* offered in the viewer's country but one programme isn't licensed there, only that programme is replaced. Ads and the rest of the channel play normally.

SSAI — ads stitched by the server

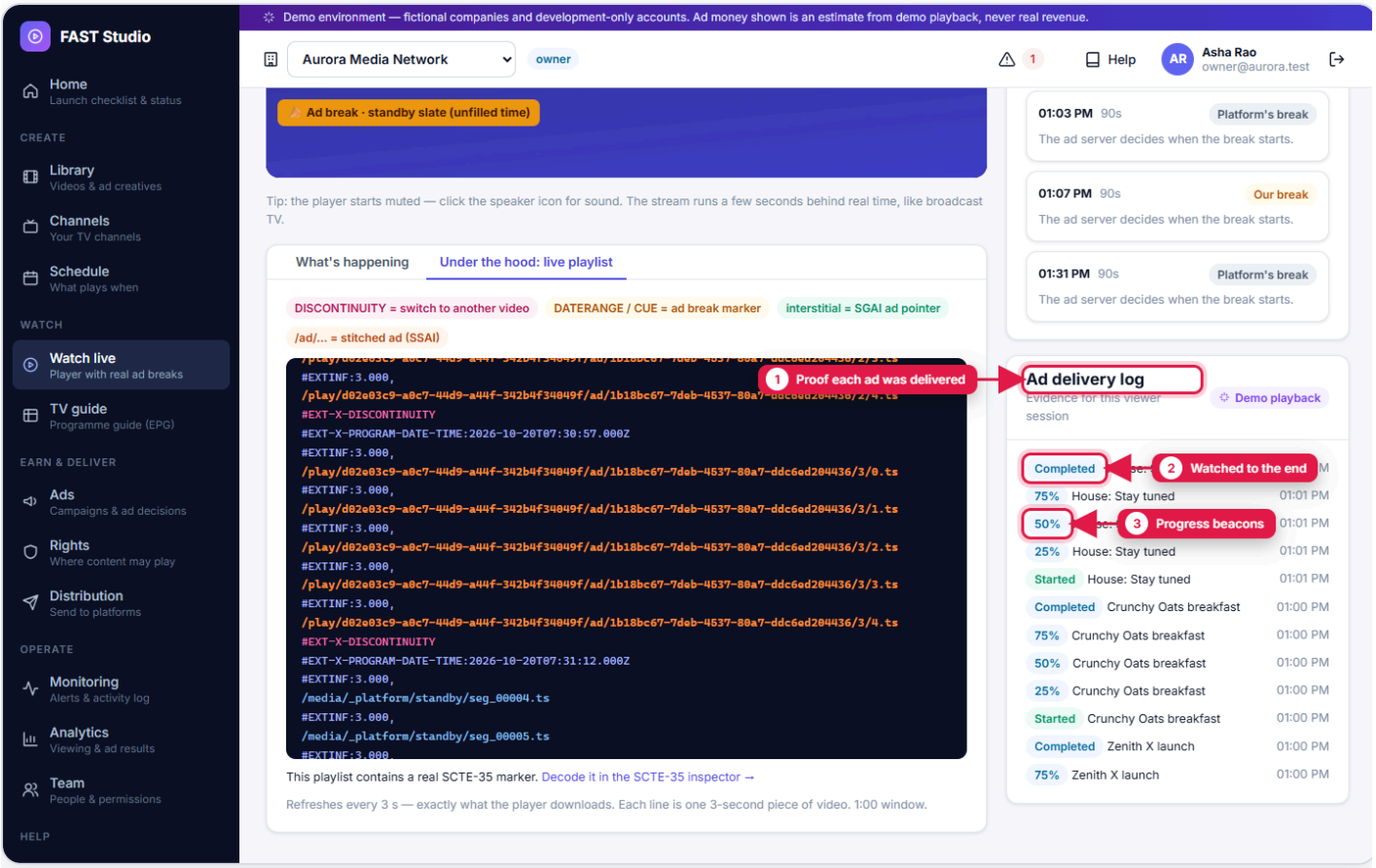


1. **Which ad is playing now**, for example "Ad 1 of 3 · Glow Cola".
2. **The ads chosen for this break**, as a bar: each ad's share of the break, plus slate for any leftover seconds.
3. **The break currently playing.** *Our break* means our campaigns fill it.



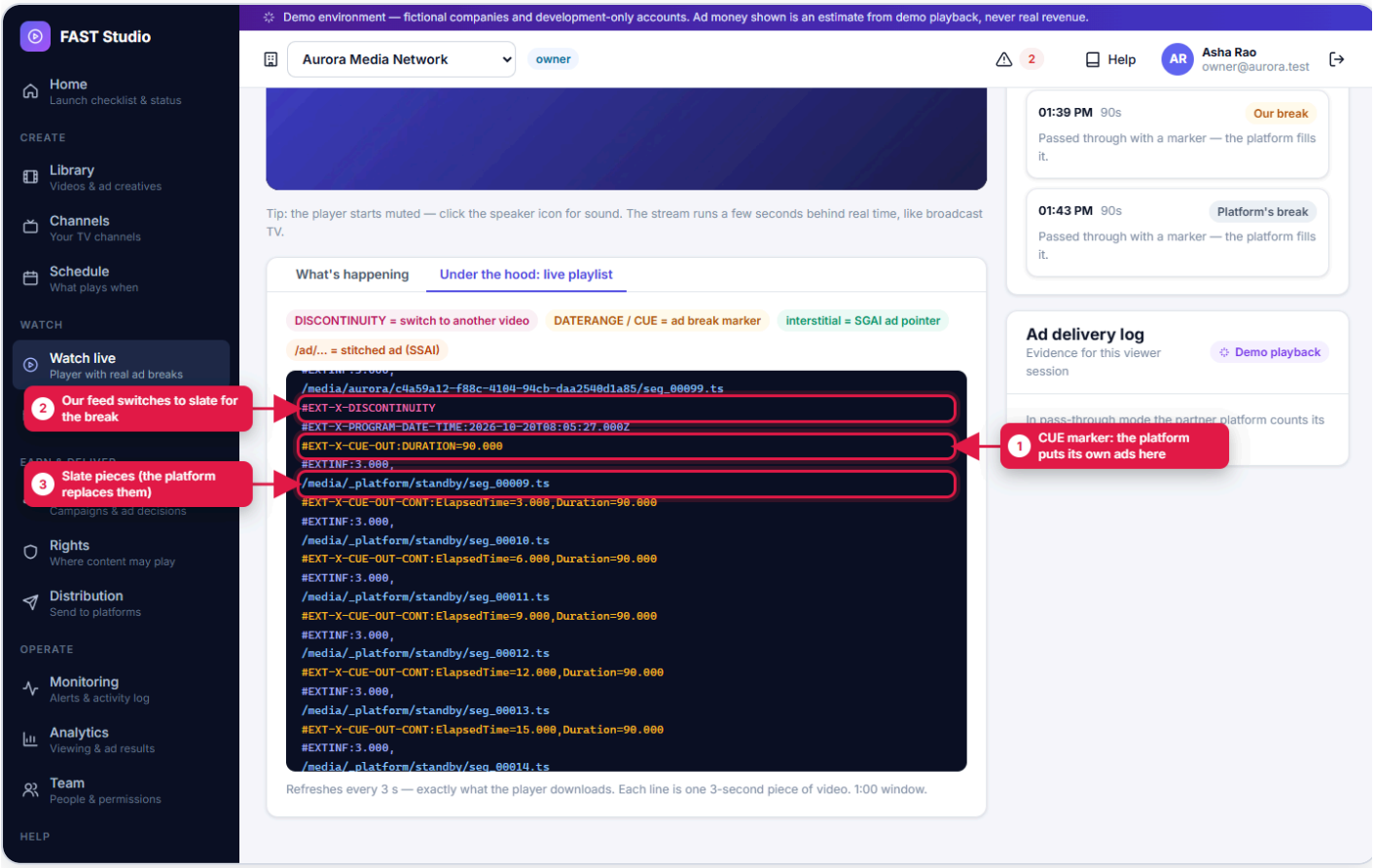
The **Under the hood** tab shows the live playlist: the actual text file the player downloads every 3 seconds, colour-coded.

1. **SCTE-35 marker**: `#EXT-X-DATERANGE … SCTE35-OUT=0xFC…` — "an ad break of 90 s starts here".
2. **Ad pieces stitched in by the server**: lines like `/play/…/ad/…/0/0.ts`. To the player they're just more video.
3. `#EXT-X-DISCONTINUITY` — tells the player "the next piece comes from a different video file, reset your timing".



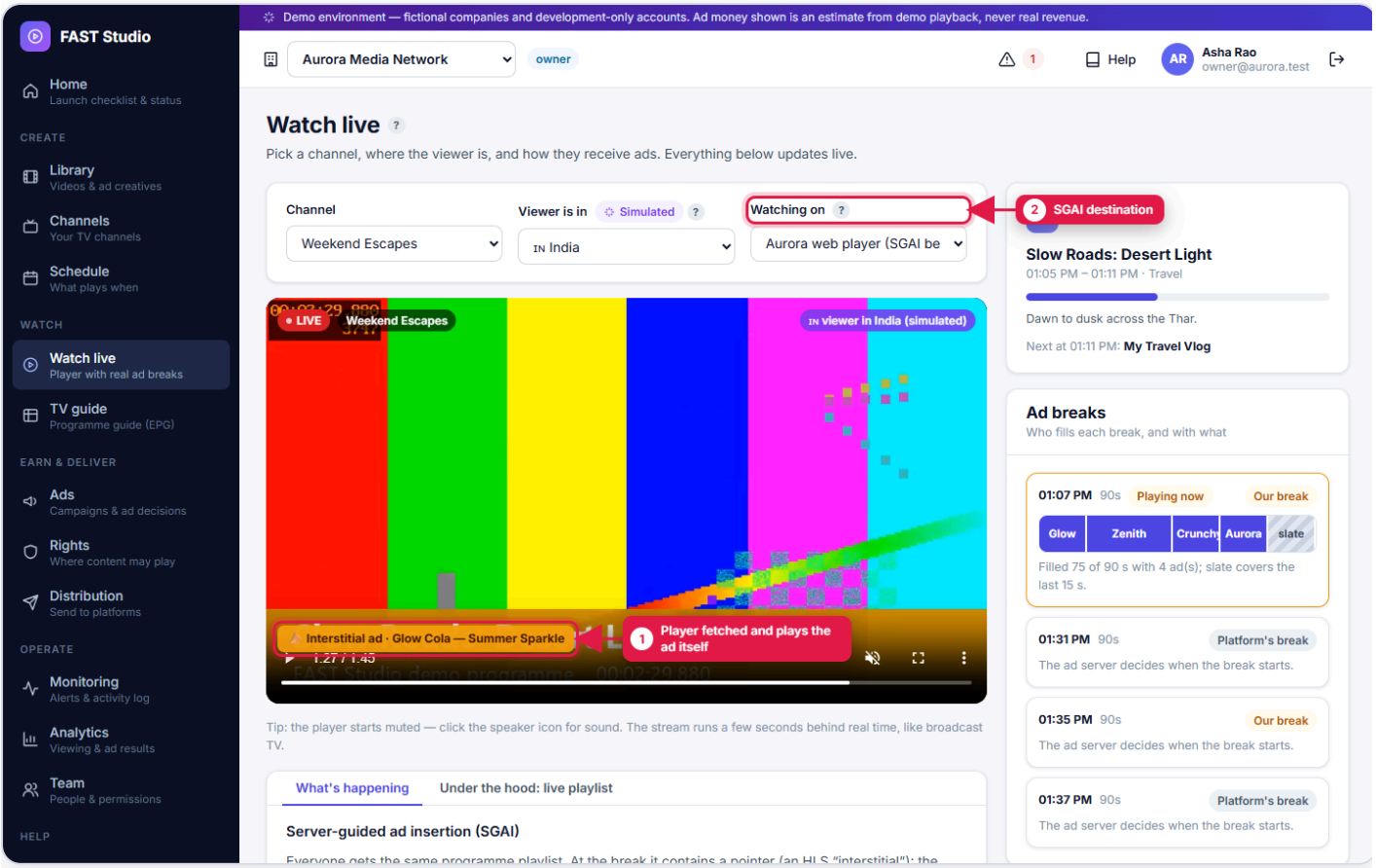
1. **Ad delivery log** — proof for this viewer that each ad was delivered.
2. **Completed** — the last piece of the ad was delivered.
3. **50%** and other progress points. Advertisers pay on these standard measures.

Partner TV — markers only

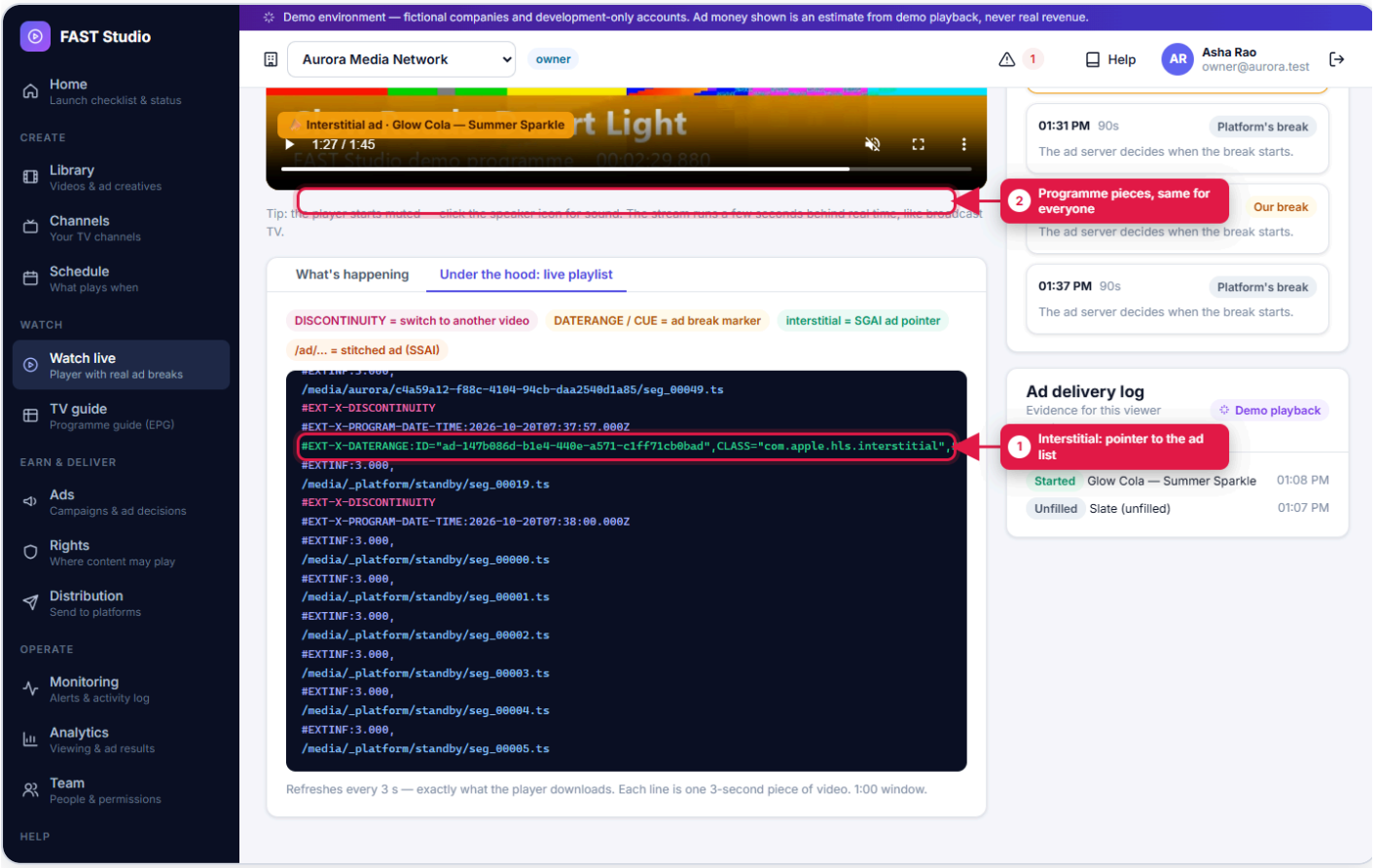


1. `#EXT-X-CUE-OUT:DURATION=90` — the older marker style some platforms require. It tells the platform where the break is and how long. `CUE-OUT-CONT` lines follow every few seconds ("30 of 90 s elapsed"), and `CUE-IN` marks the end.
2. `#EXT-X-DISCONTINUITY` — our feed switches from the programme to a slate for the break.
3. **Slate pieces** — the platform replaces these with its **own ads**. This is the "inventory split": these are breaks the platform sells.

SGAI — the player fetches the ads itself



1. The **player fetched and is playing the ad** as a separate video.
2. The **SGAI destination** is selected.



1. **Interstitial**: `#EXT-X-DATERANGE … CLASS="com.apple.hls.interstitial" … X-ASSET-LIST="…"` — a *pointer* to the list of ads, not ad video.
2. The programme pieces stay **the same for every viewer**, so the playlist can be cached by a content delivery network. That's SGAI's big advantage.

Behind the scenes — how a live channel is made, step by step

1. A clock made of 3-second slots. From a fixed starting moment (1 January 2026, 00:00 UTC), time is divided into numbered 3-second slots. Slot number *N* always means the same 3 seconds of real time, which is why the playlist's sequence number is always correct.

2. Every few seconds the player asks for the playlist. The server takes the last 20 slots (60 seconds). For each slot it looks up the **published** schedule: what's on at that moment, and how far into it?

- **Programme:** it points to the matching piece of that video (for example piece 41 of *Neon Pursuit*).
- **Gap:** it shows the channel's fallback video or the standby screen.
- **Viewer's country not licensed:** it shows the region-block screen.
- **Ad break:** see steps 4 and 5.

3. Joins are marked. Wherever the stream switches from one video to another, the server writes `#EXT-X-DISCONTINUITY`, and it keeps a running count so players never get confused. A wall-clock timestamp (`PROGRAM-DATE-TIME`) is added so guide times, ad reports and the stream all agree.

4. Ad breaks get a SCTE-35 marker. The server builds the binary marker from the break's start time, length and owner. Our breaks are marked type `0x34`, the platform's `0x36`. It writes the marker in the format the destination asked for: modern `DATERANGE` or older `CUE-OUT/IN`.

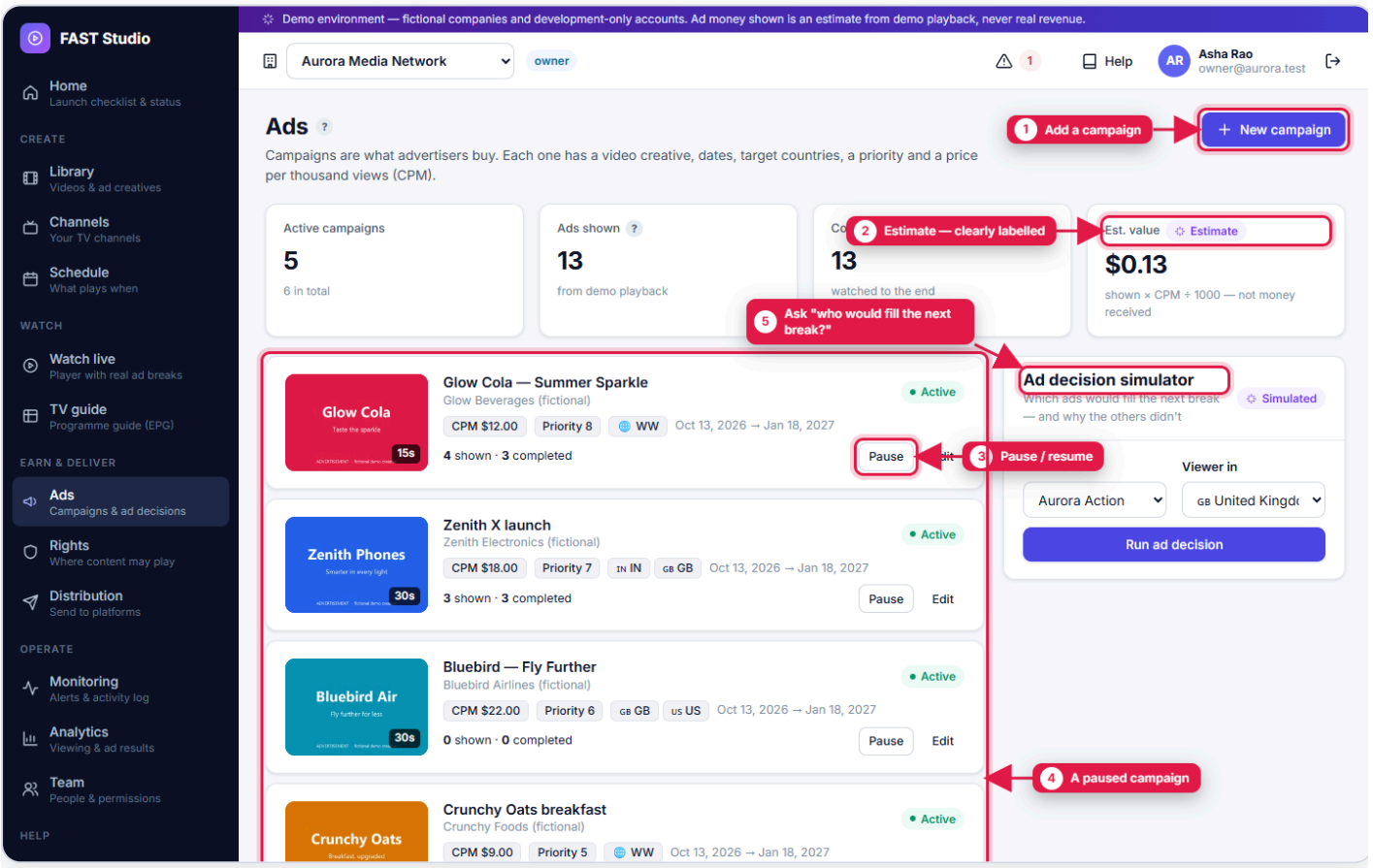
5. The break is filled according to the destination:

- **SSAI:** the server runs the **ad decision** for this viewer and break (chapter 13). It replaces the break's slots with ad pieces, and leftover time gets slate. Ad pieces are served through a special address, so each time the player downloads one, the server records a **delivery event**: *Started*, 25%, 50%, 75%, *Completed*.
- **SGAI:** the server writes an **interstitial** pointing to an *asset list*. It announces the interstitial **up to 30 seconds before the break**. Players need that warning: if the pointer first appears at the moment the break starts, the player stalls at the boundary without fetching the ads. A few seconds before the break, the player requests the list. The server runs the same ad decision and answers with the ads' own playlists, plus slate to make up the exact break length. The player plays them, rejoins the channel exactly at the break's end, and reports *Started*/*Completed* itself.
- **Pass-through (Partner TV):** only the markers. The platform sells and inserts its own ads.

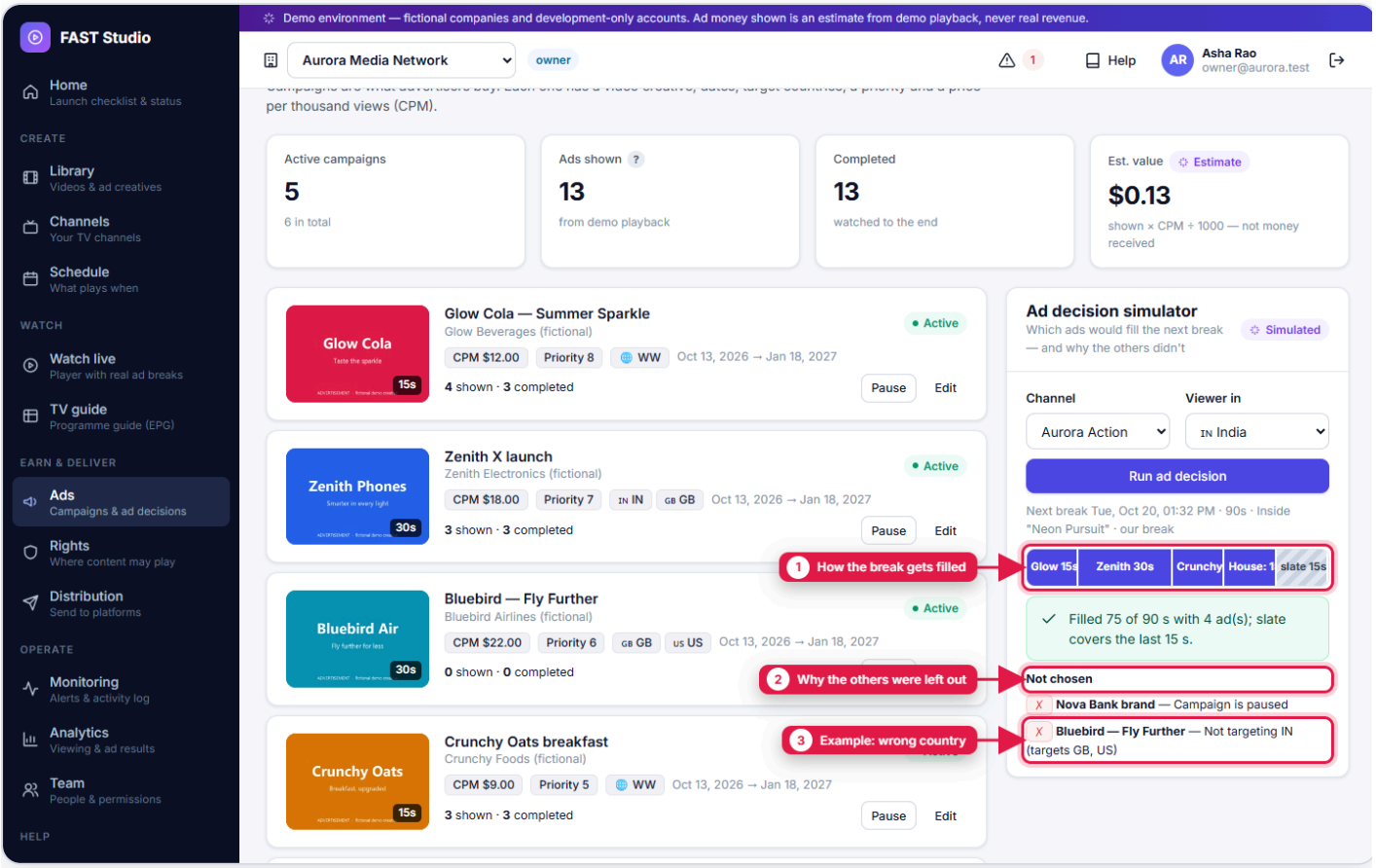
6. Each viewer gets a session. Opening the player creates a viewer session that remembers the destination and country, and keeps each viewer's ad choices stable while they watch.

---

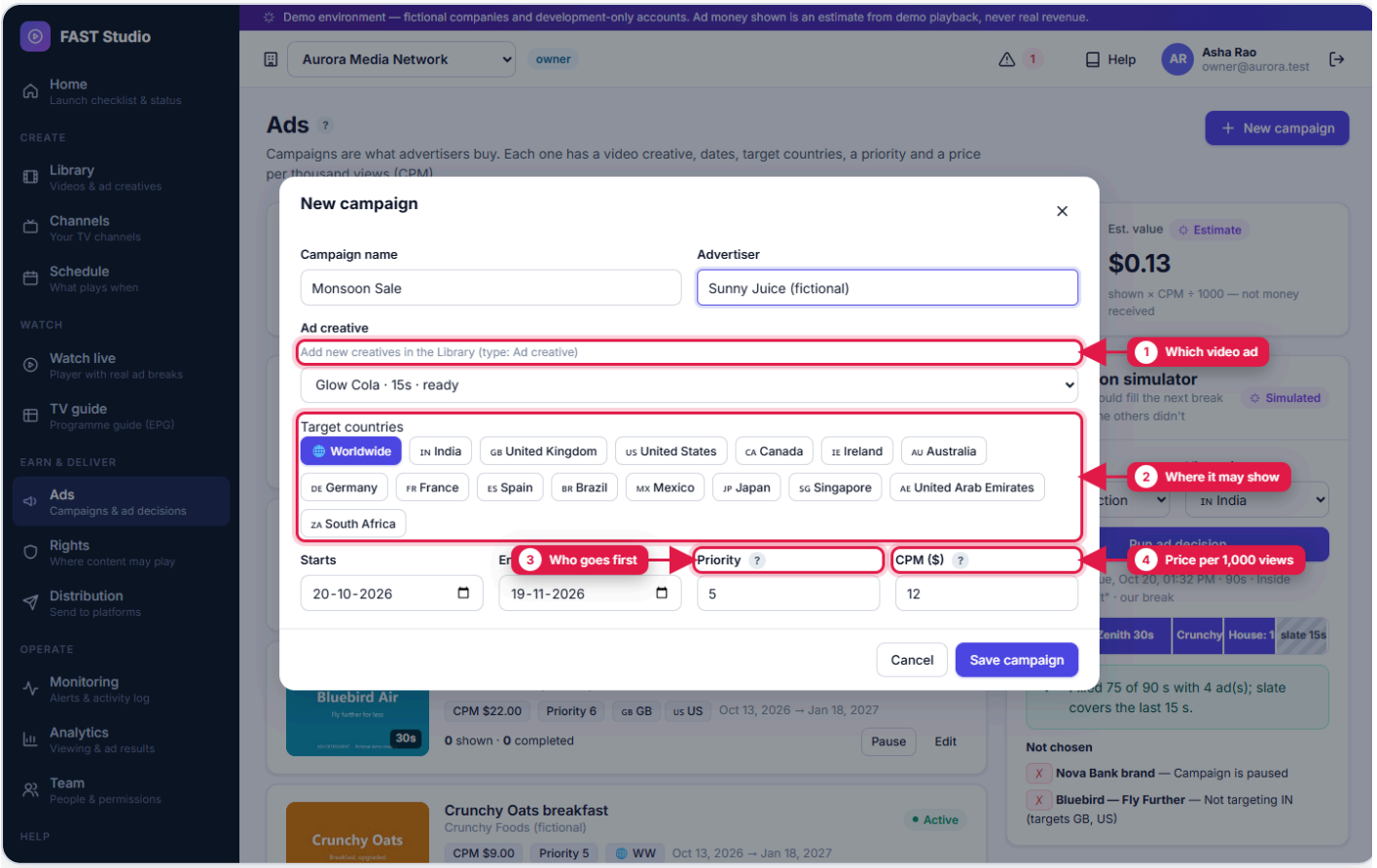
13. Ads



1. **New campaign.**
2. **Estimated value** — ads shown × the price typed into the campaign. Clearly labelled as an *estimate*, never as money received.
3. **Pause / resume** a campaign.
4. A **paused** campaign is never chosen.
5. **Ad decision simulator** — ask "which ads would fill the next break, and why?".



1. **How the break gets filled** — ads in order, plus slate for any leftover time.
2. **Why the others were left out**, one reason each.
3. For example, **"Not targeting IN"**: Bluebird Air only targets the UK and US.



1. **Which ad video** (from the Library).
2. **Target countries.**
3. **Priority** — higher-priority campaigns are placed first. House ads (your own promos) usually sit lowest, to fill leftover time.
4. **CPM** — price per 1,000 ads shown, used for the estimate.

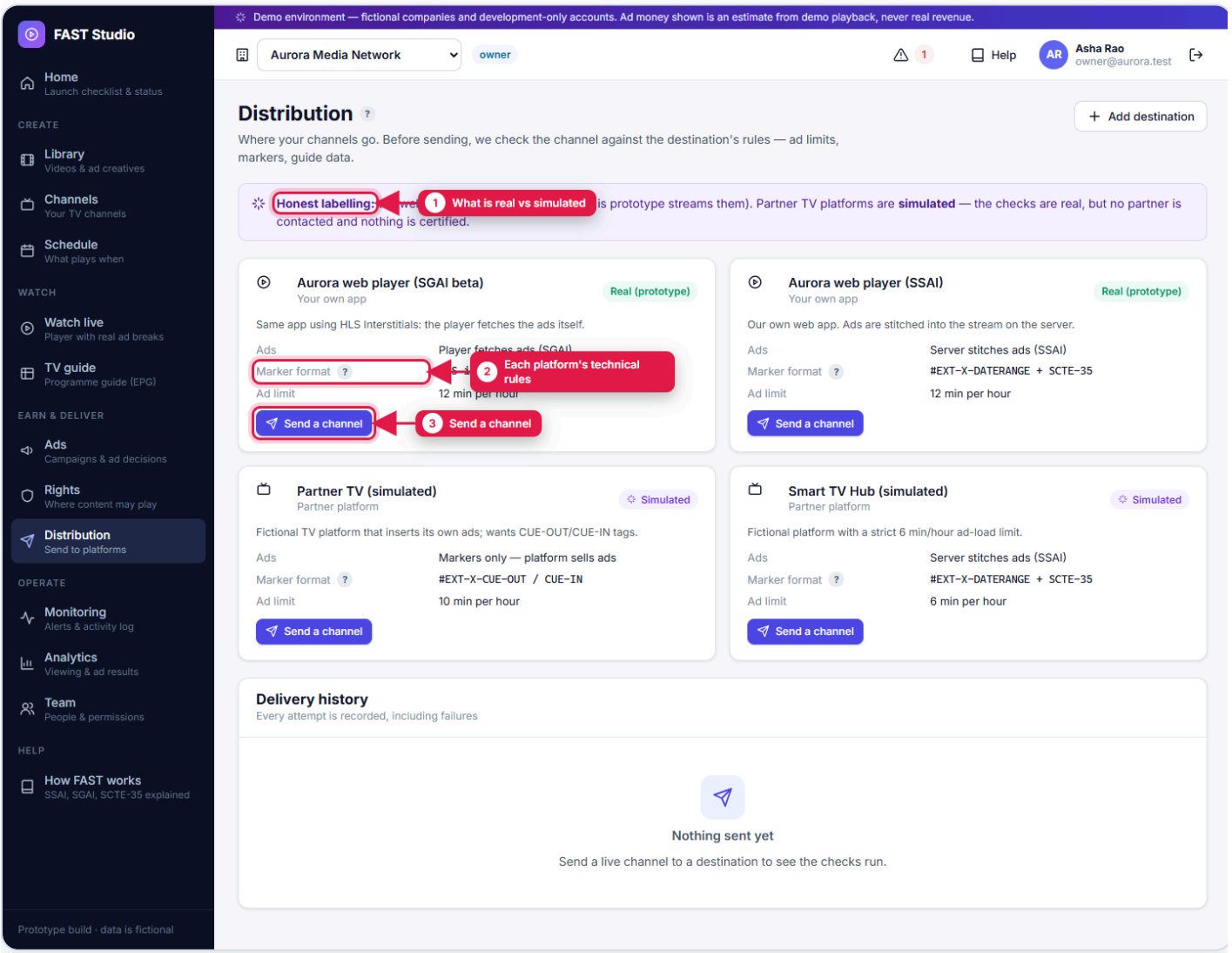
Behind the scenes — how ads are chosen for a break (`AdDecisionService`)

1. Collect all campaigns. Rule out any that are **paused**, **outside their dates**, **not targeting the viewer's country**, or whose **ad video isn't ready**. Each gets a recorded reason.
2. If the programme's licence **forbids ads**, the break stays empty.
3. Sort by **priority**. Among equal priorities, rotate fairly per viewer so the same ad isn't always first.
4. Fill the break: add an ad if it **fits the time left** and its **advertiser isn't already in this break**. Two competing brands from one advertiser don't run back to back (*competitive separation*).
5. Leftover seconds become **slate**. The break never runs over its length.

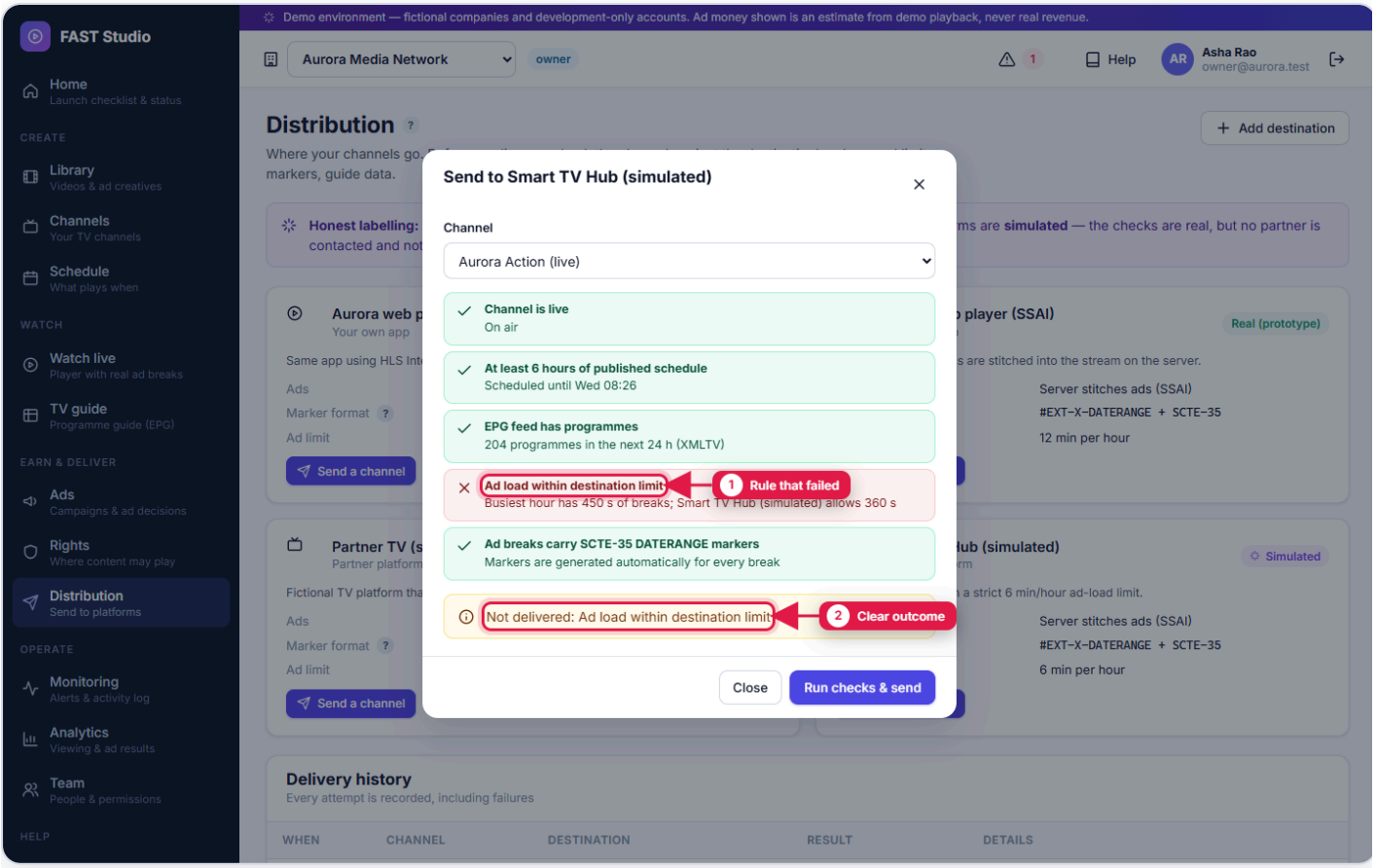
In production this step is a request to a real ad server (in a standard format called VAST). Everything around it — markers, stitching, interstitials, reporting — stays the same.

---

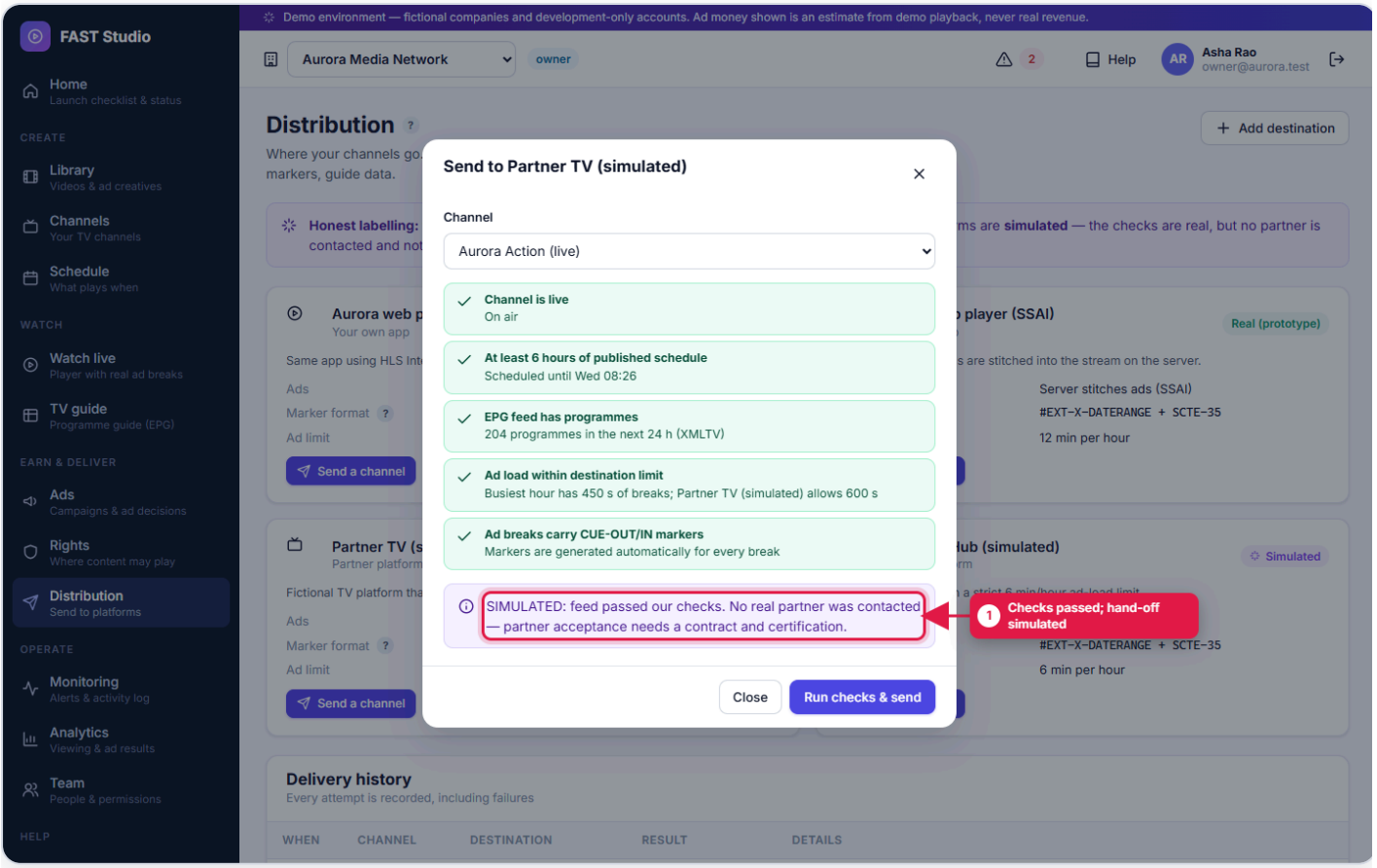
14. Distribution



1. **What is real vs simulated:** web player destinations are real (this prototype streams them). Partner TV platforms are simulated. To let a platform that is itself on FAST Studio read your guide, stream set-up and ad breaks, use **Partners** (chapter 24).
2. **Each platform's technical rules:** ad method, marker format, maximum ad time per hour.
3. **Send a channel** to a destination.



1. **The rule that failed:** Aurora Action's busiest hour has 450 s of ads, but *Smart TV Hub* allows only 360 s.
2. A clear **outcome**. An alert is raised in Monitoring automatically.

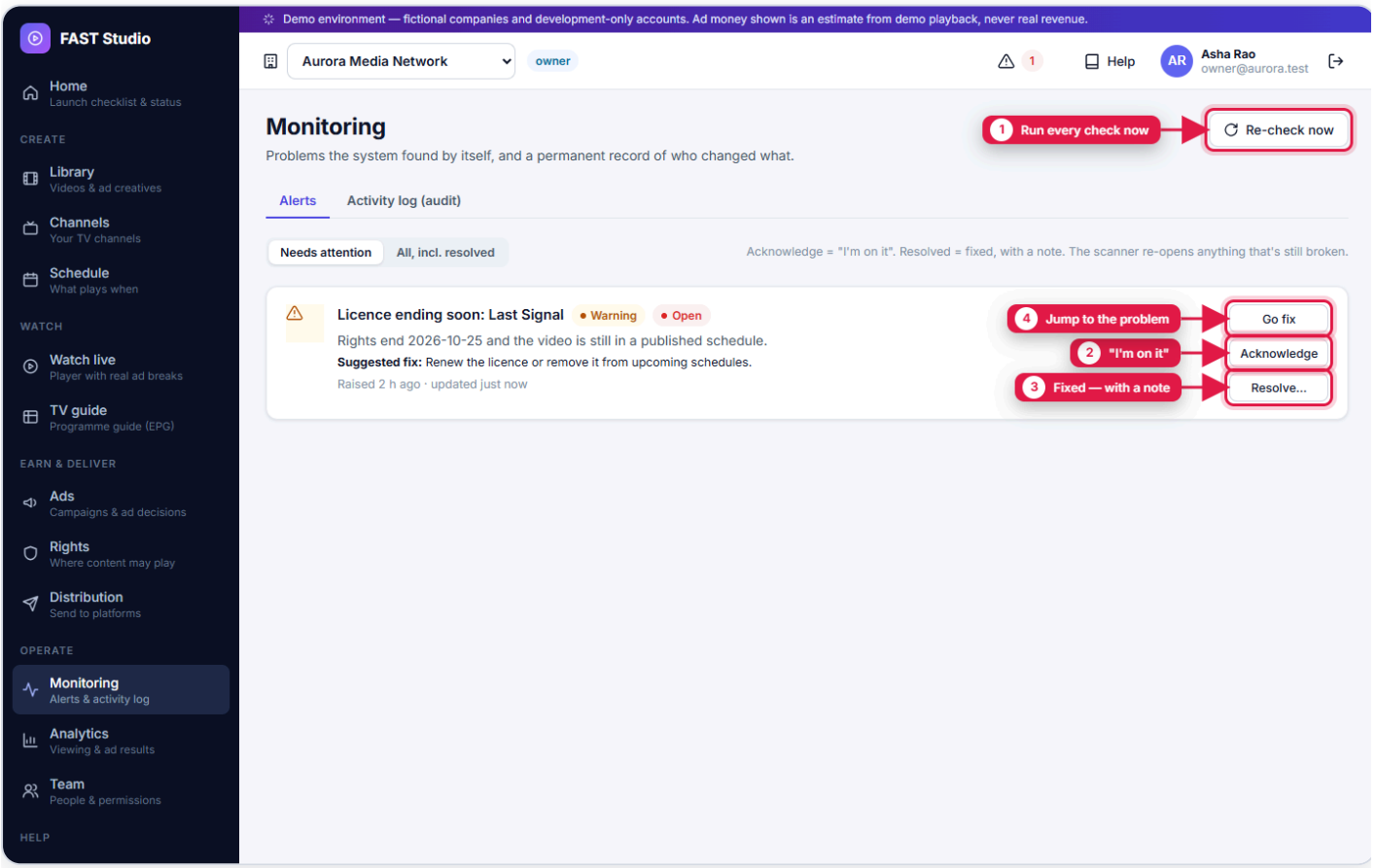


1. **All checks passed.** The hand-off itself is **simulated**: no real partner is contacted, and partner acceptance would need a contract and certification.

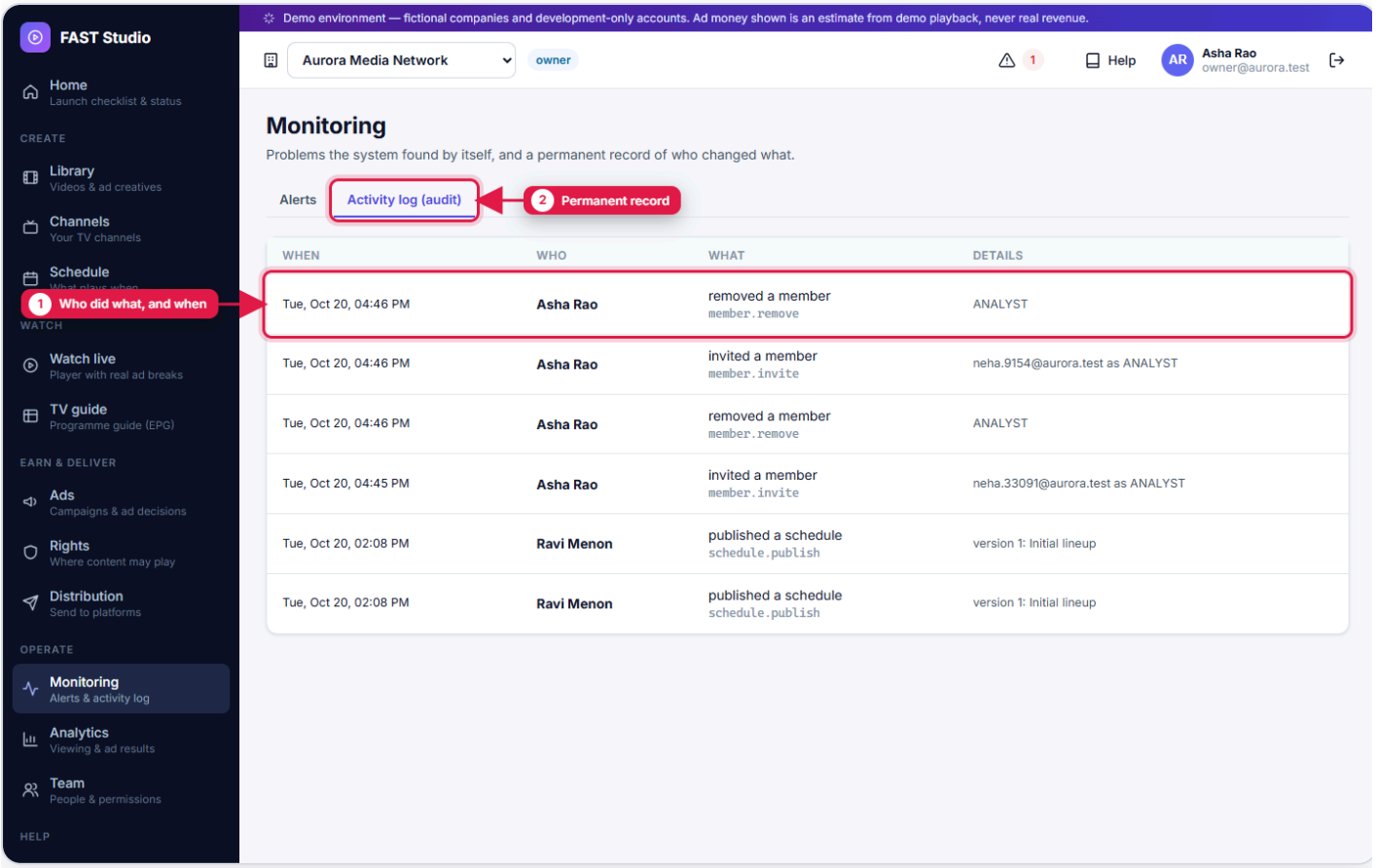
> **Behind the scenes — the checks run before sending:**
>
> - channel is live;
> - at least 6 hours published;
> - the guide has programmes;
> - the busiest hour's ad time is within the destination's limit;
> - breaks carry the marker type the destination needs.
>
> Every attempt, pass or fail, is recorded in the history.

---

15. Monitoring and audit



1. **Re-check now** — run every automatic check immediately. They also run every minute.
2. **Acknowledge** — "I'm on it". The problem is *not* marked fixed.
3. **Resolve** — fixed, with a note of what was done.
4. **Go fix** — jump straight to the page where it's fixed.

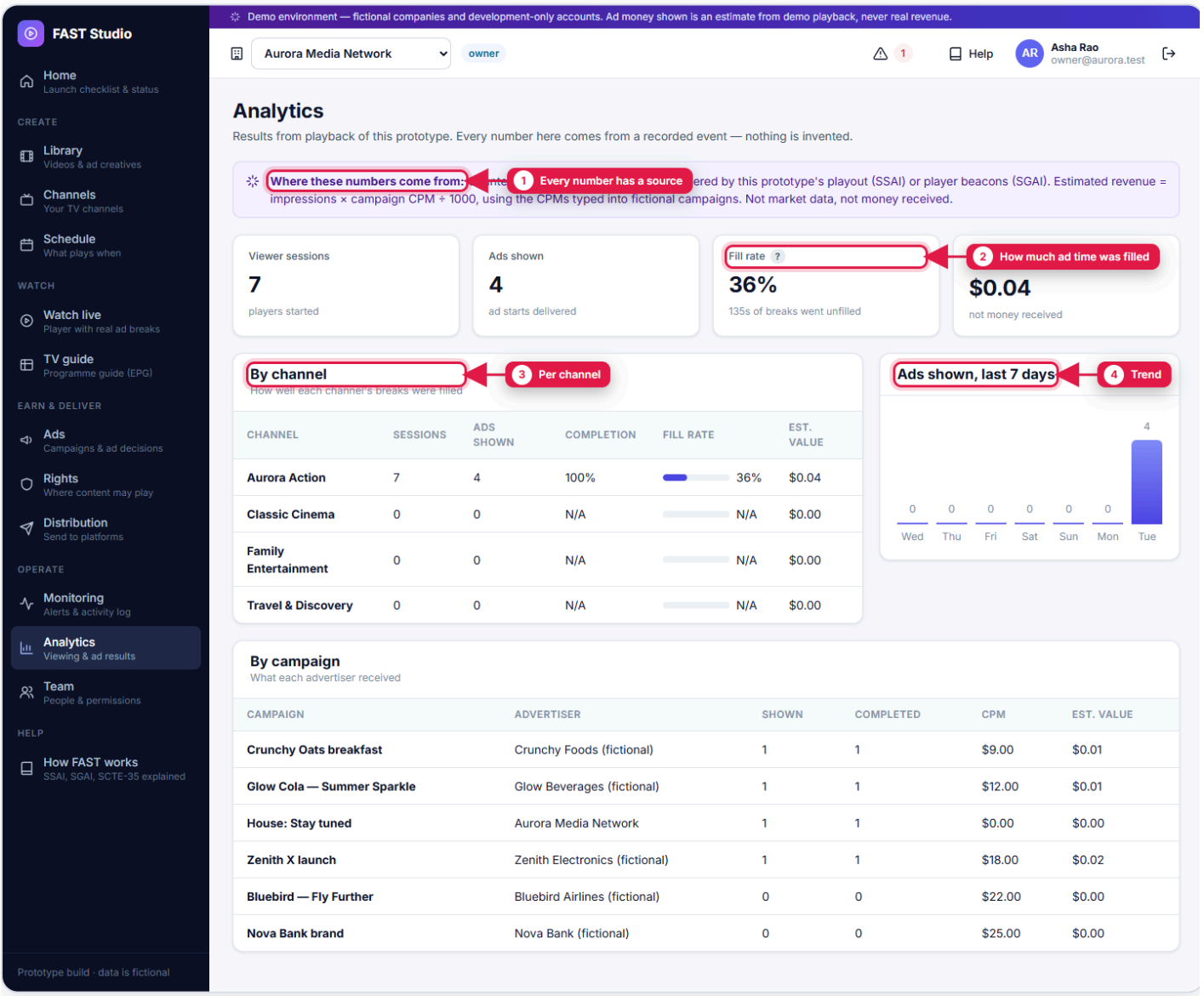


1. **Who did what, and when** — every sign-in, upload, licence change, publish, go-live, campaign change and role change.
2. A **permanent record**. It can't be edited from the product.

> **Behind the scenes — what the monitor checks every minute:**
>
> - live channels with nothing scheduled (critical);
> - schedules running out within 6 hours;
> - videos that failed processing;
> - licences ending within 7 days while still scheduled;
> - failed deliveries.
>
> When a later check finds the problem gone, the alert closes itself with "Cleared automatically". If you mark something resolved but it isn't, the next check raises it again.

---

16. Analytics



1. **Every number has a source.** They are counted from ads this prototype actually delivered (SSAI) or that players reported (SGAI). Nothing is invented.
2. **Fill rate** — how much of *our* ad-break time was filled with ads; the rest was slate.
3. **By channel.**
4. **Trend** over the last 7 days.

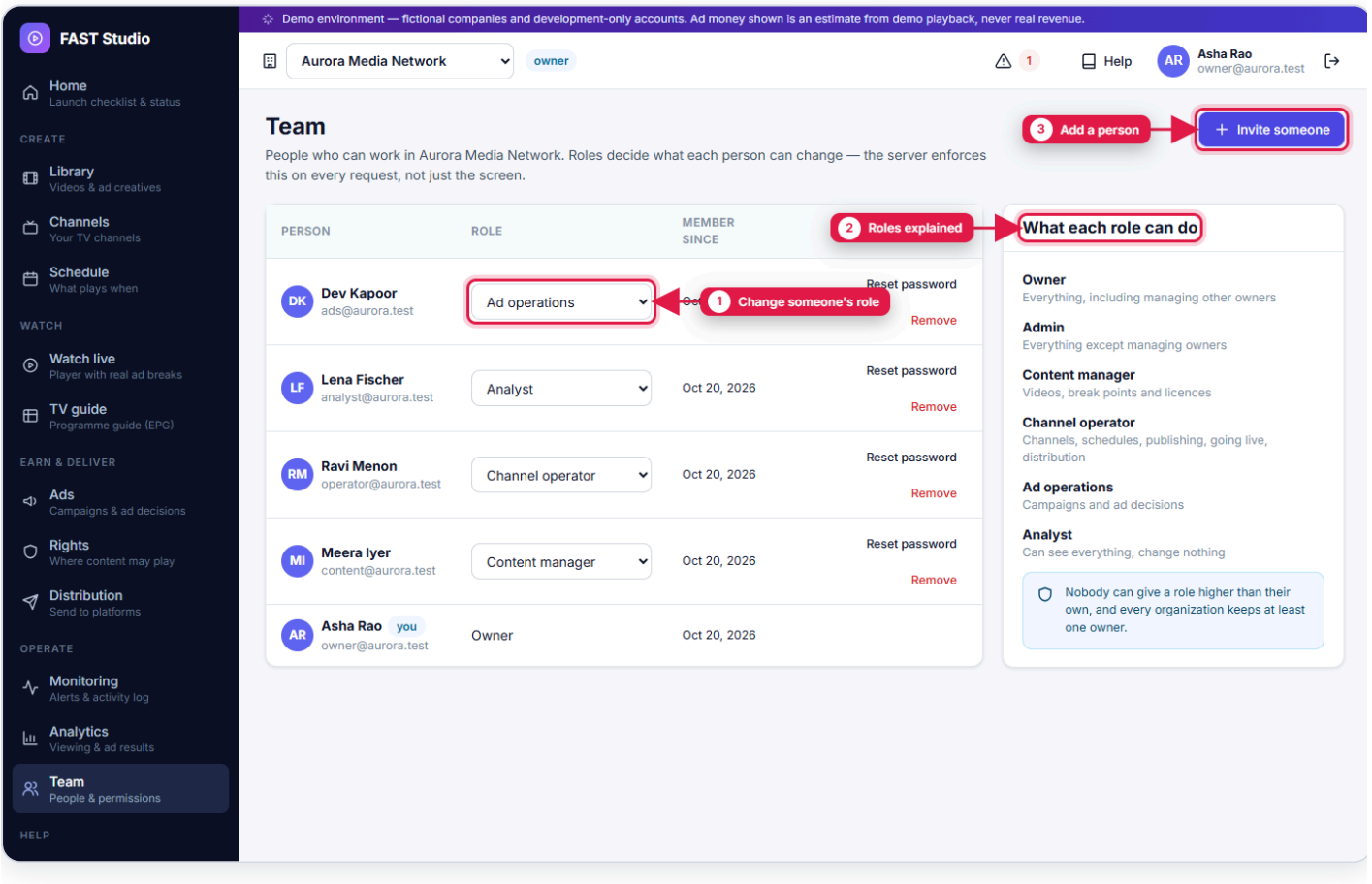
Behind the scenes — formulas:

- **Ads shown** = number of *Started* events.
- **Completion rate** = *Completed* ÷ *Started*.
- **Fill rate** = ad seconds delivered ÷ (ad seconds + unfilled slate seconds) in breaks we're responsible for.
- **Estimated value** = Σ (ads shown × campaign CPM ÷ 1000), computed with exact decimal arithmetic.

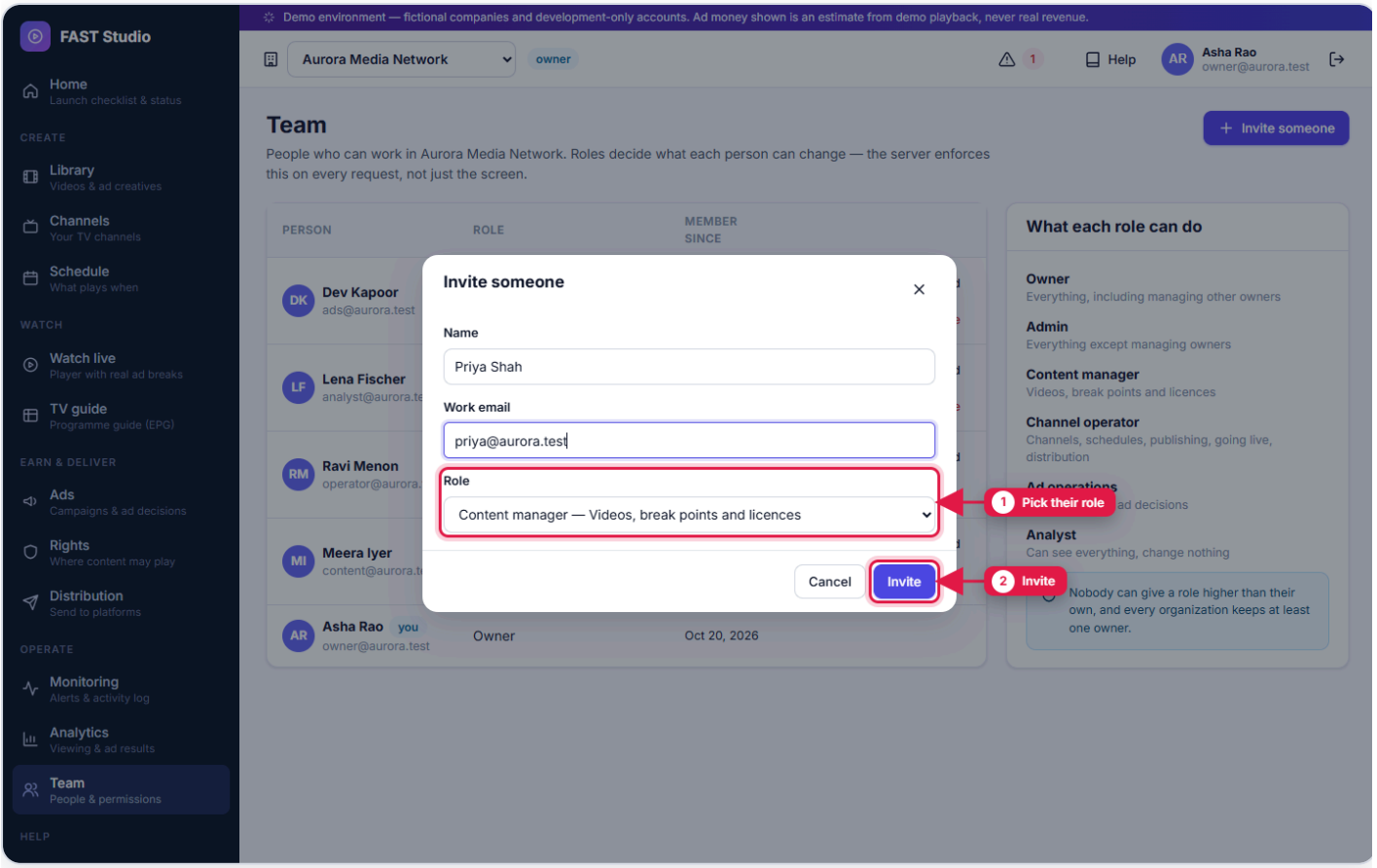
Each event is stored once per viewer, break, ad and step, so a player re-downloading a piece never double-counts. If there's no data yet, the page shows **N/A** rather than a misleading zero.

---

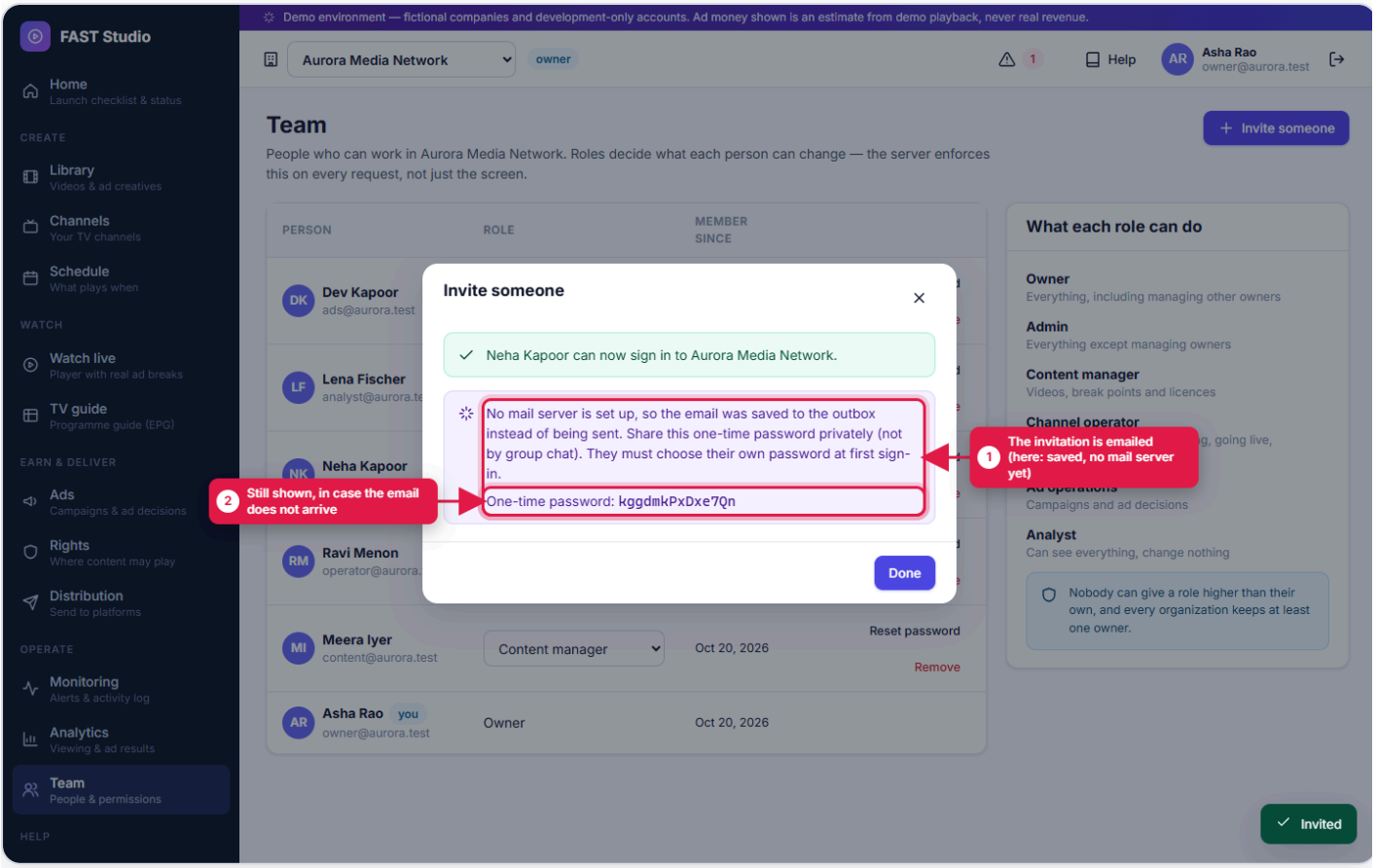
17. Team and permissions



1. **Change someone's role.**
2. **Roles explained**, in plain words.
3. **Invite** a person.



1. **Pick their role.**
2. **Invite.** FAST Studio emails the person an invitation with a one-time password. The new person must replace it at first sign-in. Each person also has a **Reset password** button: see Using it with your own data.



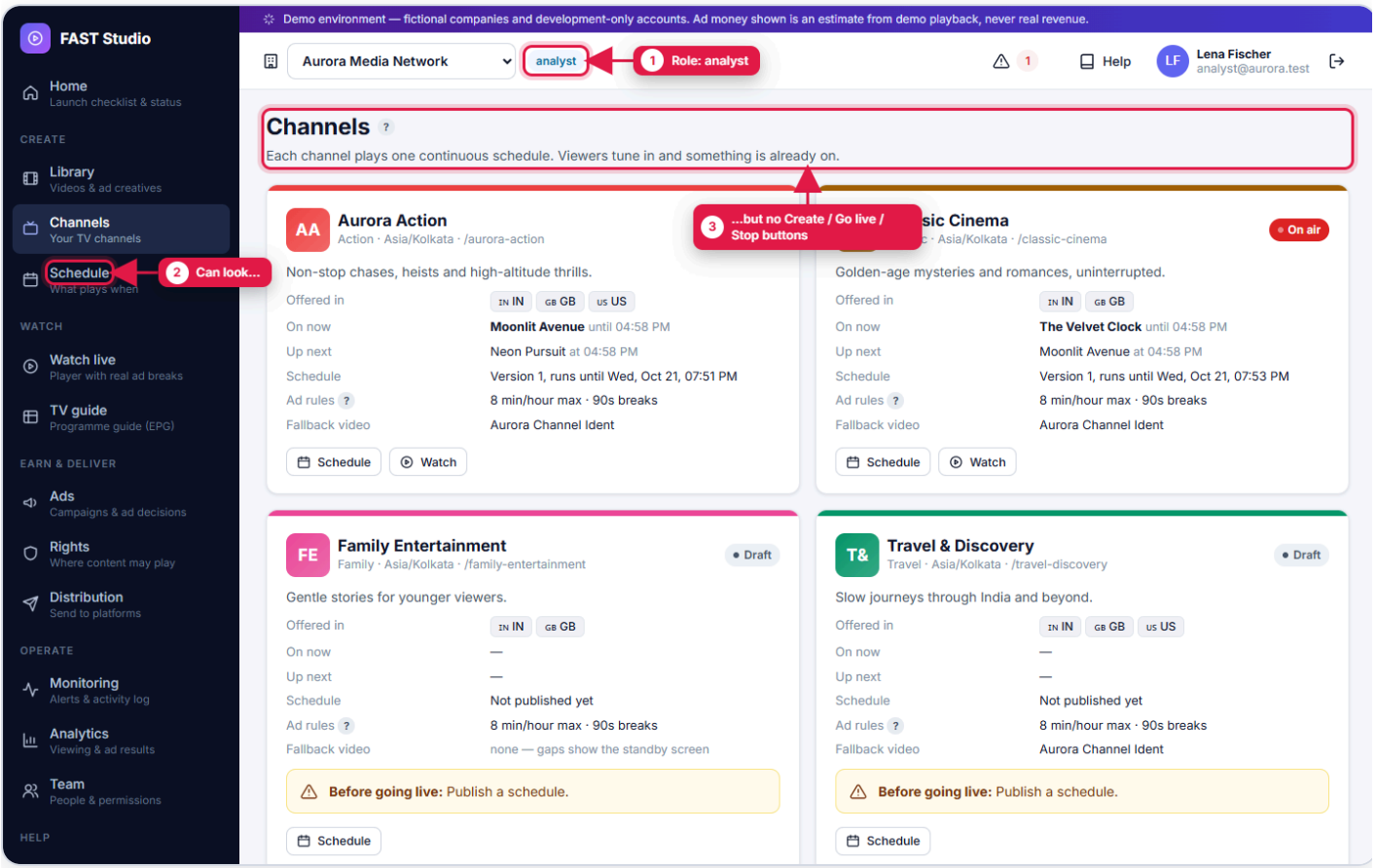
1. **The invitation is emailed.** Here the demo has no mail server, so it was saved to the outbox instead. With a mail server the note says *We've emailed the sign-in details*.
2. **Still shown on screen**, in case the email doesn't arrive.

Role	Can do
Owner	Everything, including managing other owners
Admin	Everything except managing owners
Content manager	Videos, break points, licences
Channel operator	Channels, schedules, publishing, going live, distribution, alerts
Ad operations	Campaigns, ad decisions, alerts
Analyst	See everything, change nothing

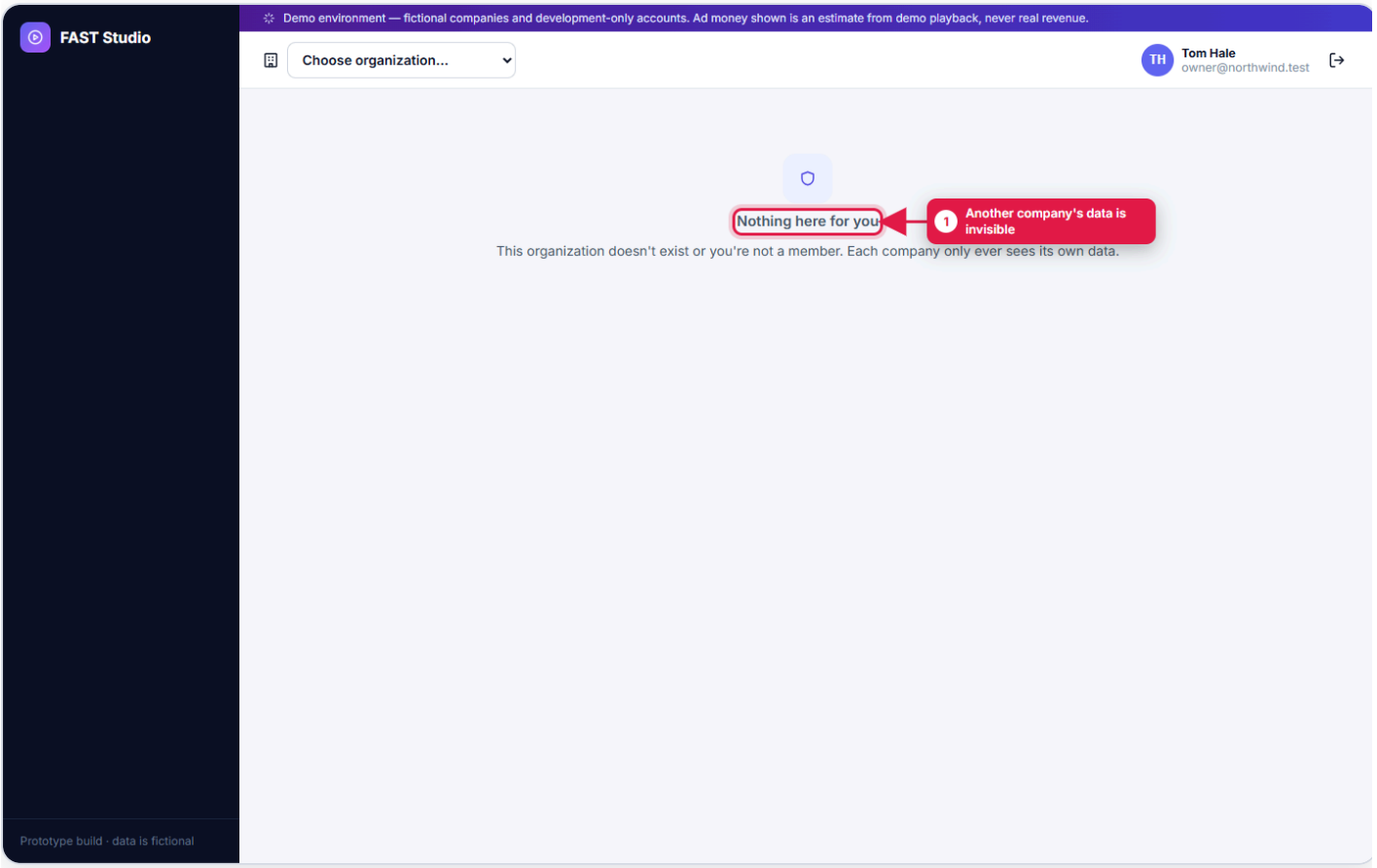
> **Behind the scenes:** permissions are checked **by the server on every request**. The read-only analyst would be refused even by calling the API directly (screenshot 50 shows the screen; the automated test proves the refusal). Nobody can give anyone a role higher than their own. Every company keeps at least one owner. You can't remove yourself.

18. Multitenancy in action

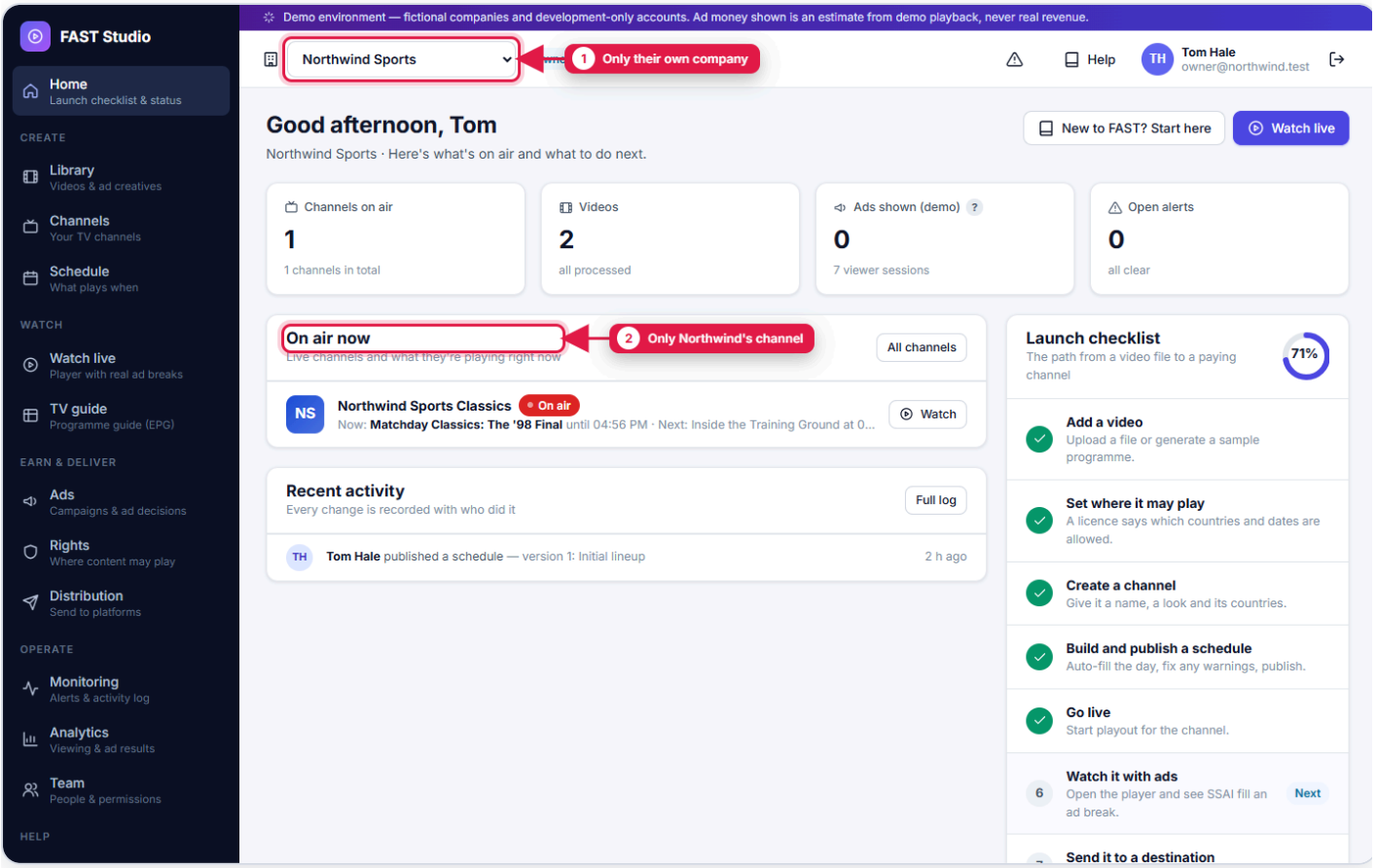
FAST Studio is built for many customer companies ("tenants") on one system, each completely separate from the others.



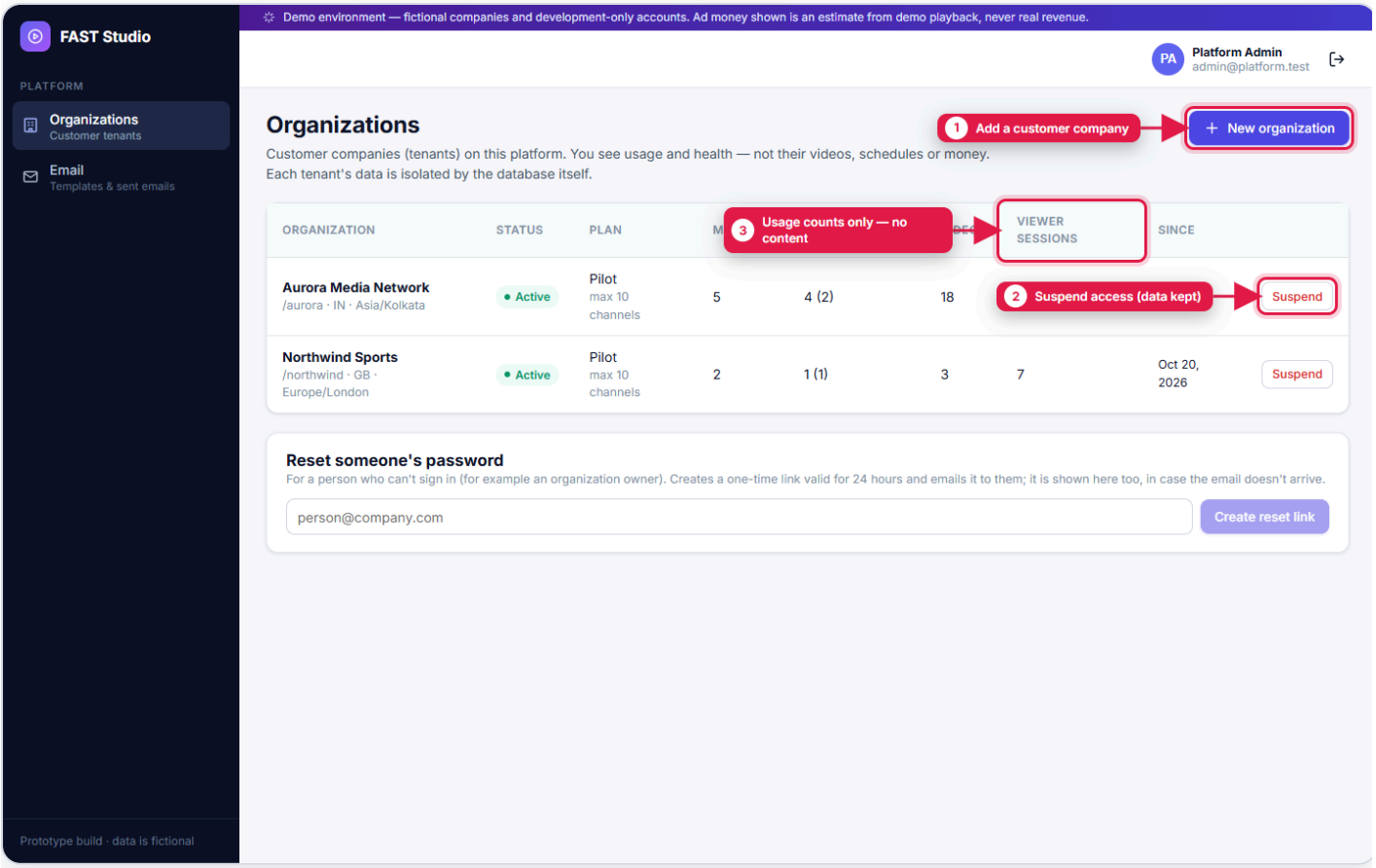
1. **Role: analyst.**
2. Can **look** at channels and schedules…
3. …but there are no Create, Go live or Stop buttons. The server would refuse those actions anyway.



1. Tom (owner of *Northwind Sports*) tries to open Aurora's address. **Nothing is shown — not even whether Aurora exists.**



1. Tom only has **his own company**.
2. Only **Northwind's** channel.



1. **Add a customer company** — creates its workspace and first owner.
2. **Suspend** — the customer's team loses access, but **nothing is deleted and channels on air keep playing**, so viewers aren't cut off.
3. **Usage counts only** — the platform team sees how much each customer uses, not their videos, schedules or money.

Behind the scenes — five layers of isolation (details in **MULTITENANCY.md** (docs/MULTITENANCY.md))

1. **One gatekeeper.** Every company page lives under `/api/orgs/<company>/…`. One component checks that *you* belong to *that* company before anything else runs. If not, you get "not found".
2. **Permission check** on every change.
3. **Every database query is filtered by company.** There is no "fetch anything by its ID" shortcut.
4. **The database itself refuses cross-company links.** Every record carries its company ID, and connections between records must match it. Even a programming mistake can't put one company's video into another company's schedule.
5. **Row-level security inside PostgreSQL.** The app talks to the database as a restricted user, and every connection is stamped with the current company. The database then shows that company's rows only, even if a query forgot its filter, and refuses writes into another company. An automated test probes all 45 company endpoints as the wrong company on every build.

Files are stored in per-company folders. Viewer sessions belong to one company's channel.

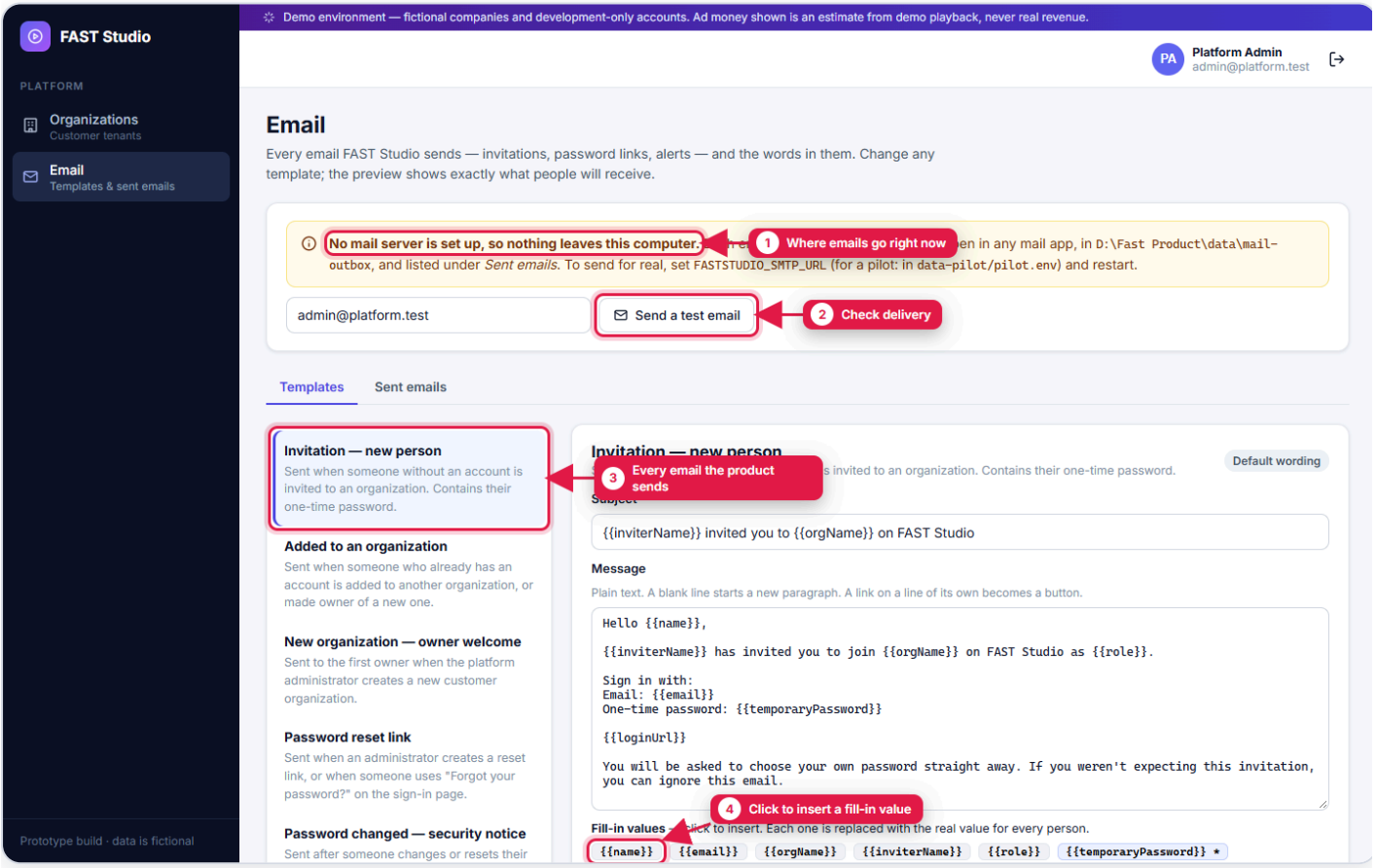
---

19. Email and templates

FAST Studio emails people when something concerns them:

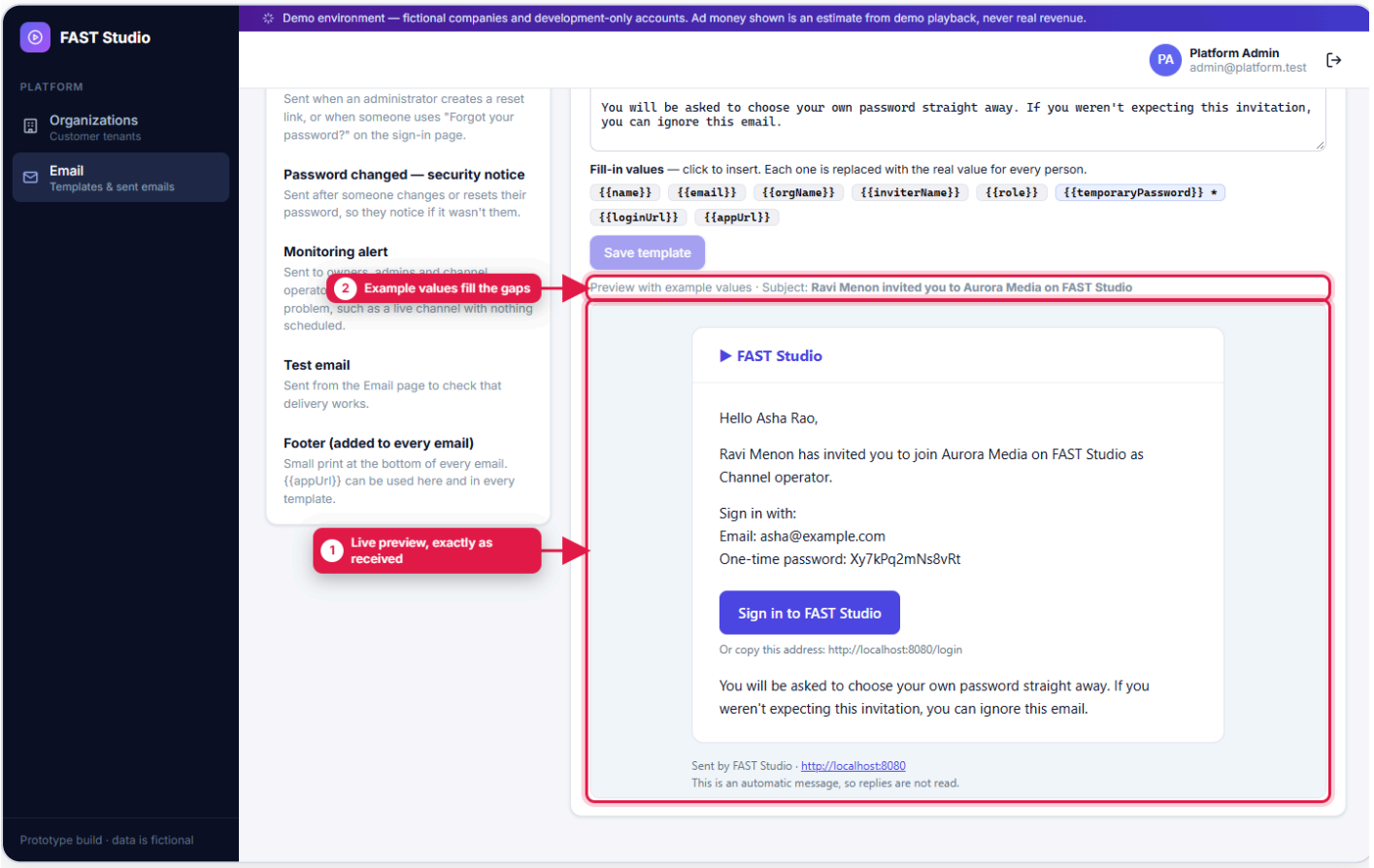
- invitations and new-company welcomes, with a one-time password
- password reset links, whether an administrator created them or the person asked with *Forgot your password?*
- a security notice after a password change
- monitoring alerts to owners, admins and channel operators

The platform administrator decides the wording on the **Email** page (*Platform → Email*).

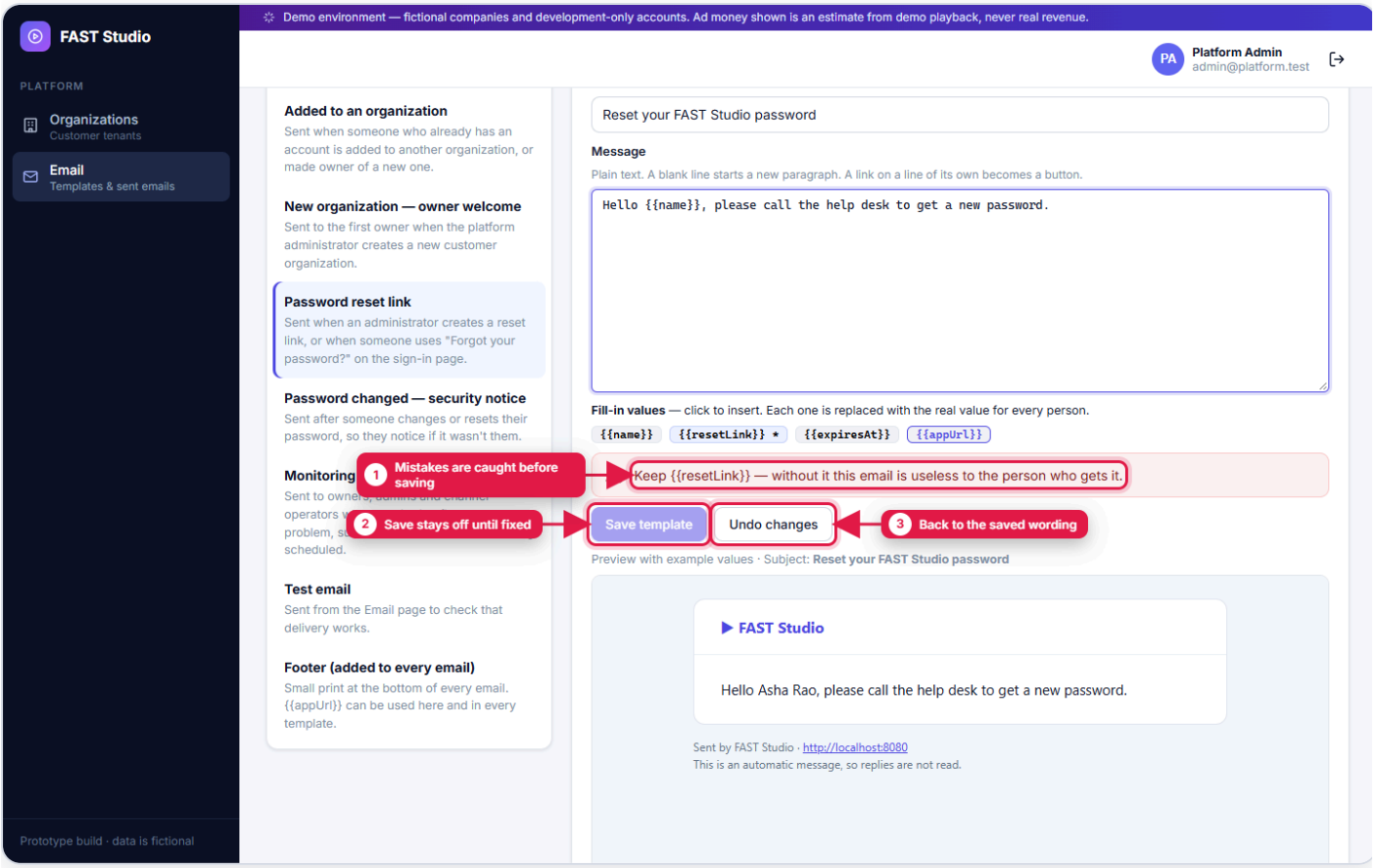


1. **Where emails go right now.** A green note means a mail server is set up and emails are delivered. A yellow note means none is set up yet: each email is written as a file (`.eml` , opens in Outlook or Mail) in the outbox folder and listed under *Sent emails*.
2. **Send a test email** to check delivery after you change settings.
3. **Every email the product sends.** Pick one to edit it. *Customized* marks the ones you changed.
4. **Fill-in values.** Click one to insert it where your cursor is, for example `{{name}}` or `{{orgName}}` . Each is replaced with the real value for every person. A ***** means *required*: the email is useless without it, such as the reset link in a reset email.

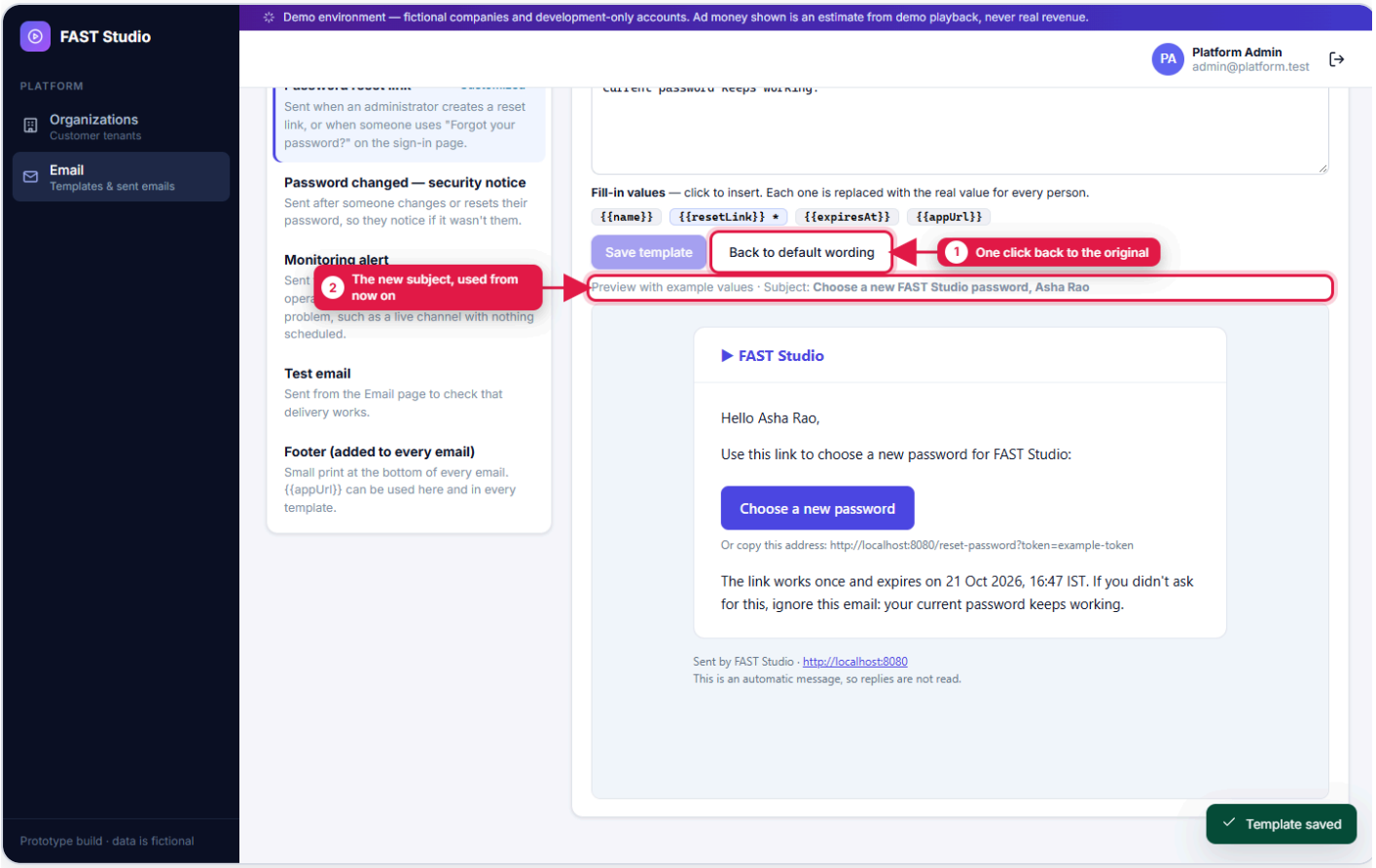
How to write a template: it is **plain text**. A blank line starts a new paragraph. A link on a line of its own (such as `{{loginUrl}}`) becomes a **button**, with the address underneath for mail apps that block buttons. HTML isn't needed, and anything typed is shown exactly as typed, never run as code.



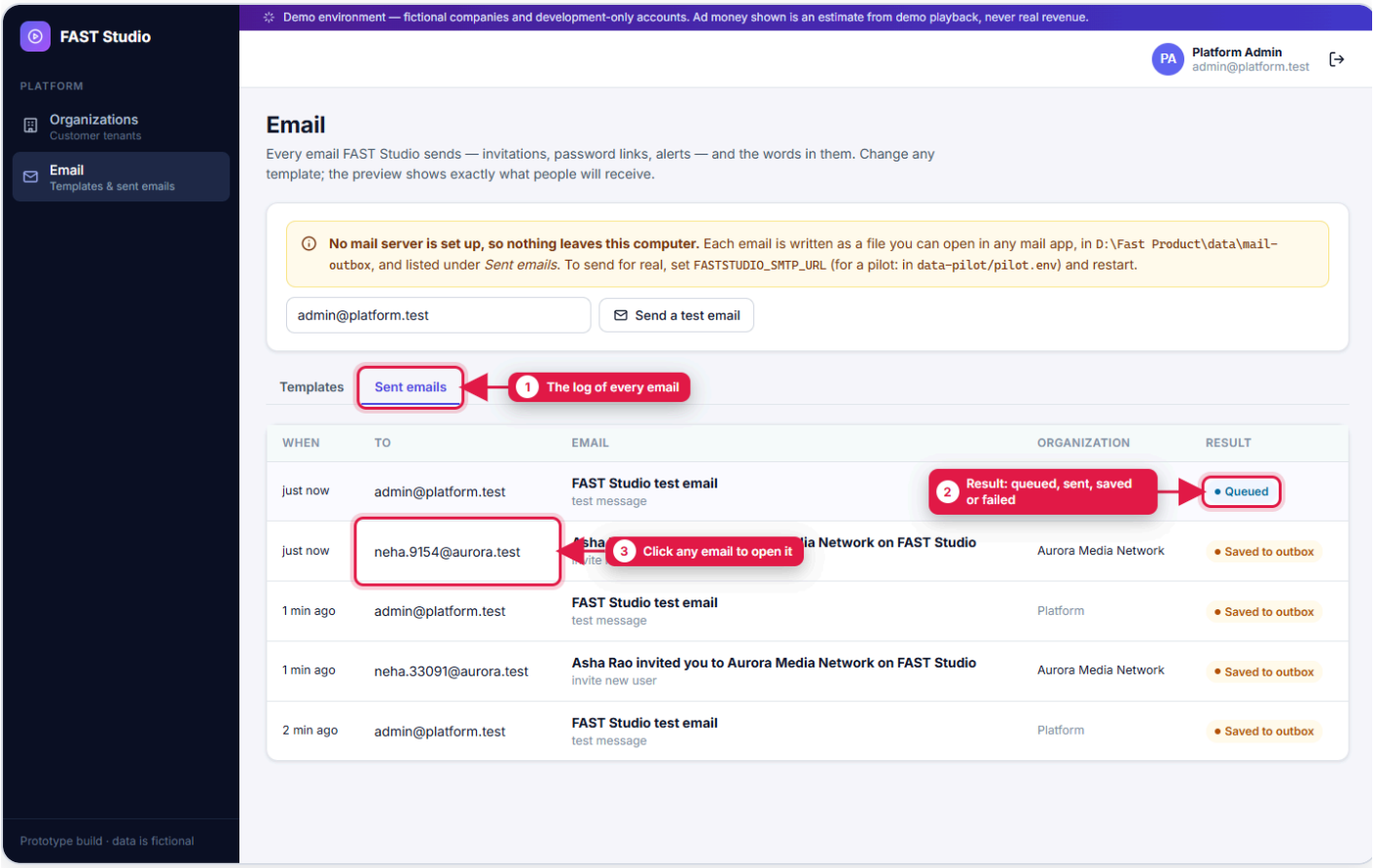
1. **Live preview, exactly as received.** It updates as you type.
2. **Example values fill the gaps**, so you see a realistic email, including the subject line.



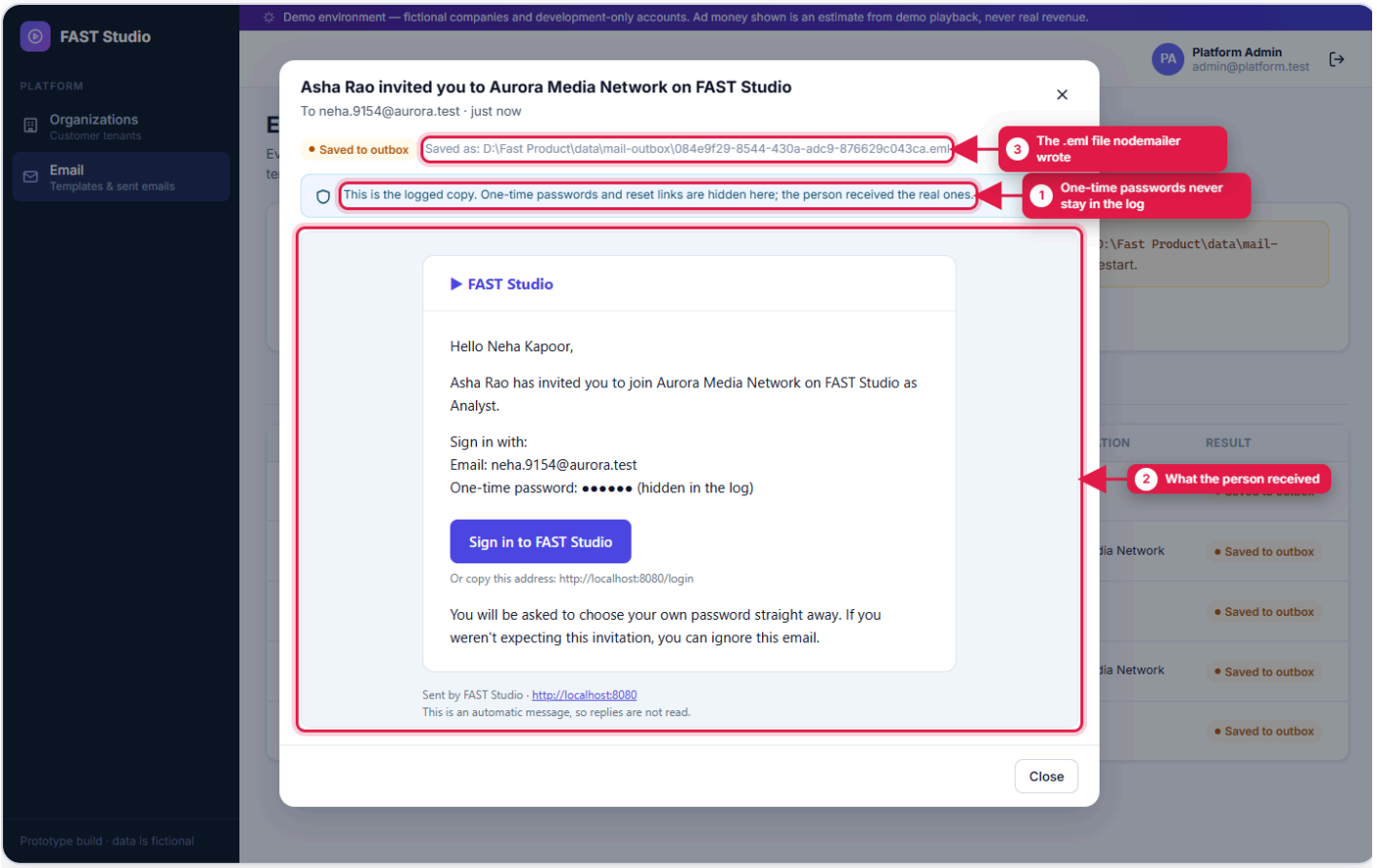
1. **Mistakes are caught before saving.** Examples: removing a required value, using a value that doesn't exist (`{{salary}}`), or an empty subject.
2. **Save stays off** until it is fixed.
3. **Undo changes** brings back the saved wording.



1. **Back to default wording** removes your changes in one click.
2. **The new subject** is used for every email sent from now on.



1. **Sent emails** is the log of every email: newest first, with the company it belongs to.
2. **Result.** *Queued* means waiting its turn (a second or two). *Sent* means accepted by your mail server. *Saved to outbox* means written to a file because no mail server is set up. *Failed* shows the reason, with a **Send again** button.
3. **Click any email** to open it.



1. **One-time passwords and links never stay in the log.** The person received the real ones; the log shows ••••••.
2. **What the person received**, in the same layout.
3. **The .eml file** nodemailer wrote (or, with a mail server, the server's answer).

Behind the scenes — email

- **Outbox first:** every email is stored in the database (`email_message`) in the same step as the action that caused it. If that action fails and is undone, no email goes out. A background worker then hands each email to **nodemailer** (`mailer/send.mjs`), which delivers it through your SMTP server or, without one, writes the .eml file. Emails still waiting survive a restart.
- **Secrets:** the log keeps a copy with one-time passwords and reset links hidden. The real copy is deleted as soon as the email is delivered.
- **Safety of templates:** names and other values are escaped, so a person called `<script>` can't inject code into an email. Template changes are written to the audit log.
- **Who can edit:** platform administrators only. Templates apply to every company; per-company wording is planned.

20. End-to-end checklist

Use this to demonstrate (or test) the whole product in about 15 minutes:

- ☐ Sign in as **Asha** (owner) → Home shows the checklist and channels on air.
- ☐ **Library** → **Add video** → **Upload a file** → watch it go *Processing* → *Ready*. Open it, play it, **add a break point**.
- ☐ **Rights** → give the new video a licence (India, UK, US). Check *Steel Horizon* for a US viewer → blocked, with the reason.
- ☐ **Channels** → **New channel** → 4 steps → draft schedule opens.
- ☐ **Schedule** → add *Mountain Markets* → *Must fix* appears → remove it → **Publish**.
- ☐ **Channels** → **Go live.**
- ☐ **TV guide** → the new channel's programmes appear. Download **XMLTV**.
- ☐ **Watch live** → viewer in **Germany** → region block. Viewer in **India**, **SSAI** → wait for a break → ads play, the delivery log fills → **Under the hood** shows `SCTE35-OUT` and `/ad/` pieces.
- ☐ Switch to **SGAI** → the playlist shows `com.apple.hls.interstitial`, and the player plays the ads itself.
- ☐ Switch to **Partner TV** → `CUE-OUT/CUE-IN` markers, slate in our feed.
- ☐ **Ads** → **simulator** → see why campaigns were left out. Pause one; it disappears from decisions.
- ☐ **Distribution** → *Smart TV Hub* fails on ad load; *Partner TV* passes (simulated) → **Monitoring** shows the alert.
- ☐ **Analytics** → numbers from what you just watched.
- ☐ **Monitoring** → **Activity log** → every step above is recorded.
- ☐ Sign in as **analyst** (no edit buttons) and as **owner@northwind.test** (no Aurora data).
- ☐ **Team** → **Invite someone** → the note says the invitation was emailed (or saved to the outbox).
- ☐ Sign in as **admin@platform.test** → **Email** → change a template, watch the preview, **Send a test email** → **Sent emails** shows it.
- ☐ Sign out → **Forgot your password?** → the same neutral answer for any email.
- ☐ As **Asha**: **Partners** → Partner TV shares 2 channels. As **partner@partnertv.test**: *Shared with us* → open Aurora Action → guide, stream (try as a viewer), ad breaks; *Viewing numbers* is locked. Back as Asha: *Activity log* shows Partner TV's reads.

Automated version: run `node tools/smoke-test.mjs` (27 checks in demo mode, 6–9 in pilot mode) and `cd backend && ./mvnw test` (51 tests, including tenant isolation for companies and partners, sign-in tokens, pilot mode and email).

---

21. Technology reference

Components

Part	Technology	Role
Web app	React 19, TypeScript, Vite, hls.js 1.7	All screens; the video player (supports SGAI interstitials)
Server	Java 21, Spring Boot 4	API, security, playout, SCTE-35, ad decisions, monitoring
Database	PostgreSQL 14 (embedded; external via `DB_URL`) + Flyway migrations	All data; company isolation enforced with keys
Video processing	ffmpeg	Conversion to 3-second HLS pieces, thumbnails, demo videos
Email	nodemailer 10 (Node.js), started by the server for each email	Invitations, reset links, alerts. Uses your SMTP server, or writes .eml files to the outbox.

Code map

Folder (`backend/src/main/java/com/faststudio/`)	What it does
`core/`	Sign-in and tokens (`Tokens.java`, `TokenKeys.java`), security, company gatekeeper (`TenantInterceptor`), shared data access, audit log, embedded database
`media/`	ffmpeg jobs: upload conversion, sample generation, slates
`catalog/`	Library API, licences API
`rights/`	The licence rules engine (`Rights.java`)
`channels/`	Channels, wizard, go-live checks
`schedule/`	Validation and auto-fill (`Planner.java`), draft/publish/rollback, TV guide and XMLTV
`playout/`	Live playlist builder (`PlayoutService.java`), SCTE-35 (`Scte35.java`), public viewer endpoints
`ads/`	Campaigns, ad decisions
`distribution/`	Destinations and delivery checks
`ops/`	Dashboard, alerts and scanner, audit, analytics, SCTE-35 tools
`team/`	Members, roles, platform administration
`partners/`	Partner sharing (`PartnerSharingController`), the partner gatekeeper (`PartnerInterceptor`), the read-only partner API, API clients and `/oauth/token`
`mail/`	Email templates and their checks (`EmailTemplates.java`), outbox queue and nodemailer hand-off (`MailService.java`), Email page API. The sender itself is `mailer/send.mjs`.
`demo/`	Fictional demo data (demo mode only)

Frontend screens live in `frontend/src/pages/`, one file per menu item.

Main data tables

`organization`, `app_user`, `membership` (person ↔ company + role), `asset` (videos, with break points), `rights_policy`, `channel`, `schedule_version` (0 = draft, 1…n = published), `schedule_entry` (programme part or ad break, with owner), `campaign`, `destination`, `publication`, `playback_session`, `ad_event`,

`alert`, `audit_event`, `password_reset` (reset-link fingerprints), `refresh_token` (sign-in renewals, fingerprints only), `partner_link` + `partner_link_channel` (what is shared with which partner), `api_client` (partner machines; secret fingerprints only), `email_template` (your template changes), `email_message` (outbox and sent-email log).

Full API list: **API_SPECIFICATION.md** (docs/API_SPECIFICATION.md). Deep dive on streaming: **HOW_FAST_WORKS.md** (docs/HOW_FAST_WORKS.md).

---

22. Real vs simulated

Capability	Status
Sign-in, roles, company isolation, audit	**Real** (local accounts; production uses single sign-on)
Upload → conversion → preview	**Real** (360p/540p/720p ladder)
Licences enforced in schedule, stream and ads	**Real**
Schedule validation, auto-fill, versions, rollback	**Real**
TV guide and XMLTV	**Real**
Live channel (HLS playout)	**Real**, prototype grade (not a broadcast-grade service)
SCTE-35 markers	**Real**, verified against the standard's sample
SSAI and SGAI	**Real** in the prototype's own stitcher and player
Ad decisions	**Local stand-in** for an ad server; no real advertisers
Viewer location	**Simulated** (you choose it)
Partner platforms	**Simulated** hand-off; the checks are real
Revenue	**Estimate** only
Email (invitations, reset links, alerts)	**Real**: nodemailer through your mail server; without one, saved as .eml files
Sign-in tokens (JWT), organization IDs in every address	**Real**
Partner organizations (sharing, partner portal, partner API, API clients)	**Real**. The partner companies in the demo are fictional.
Billing, payouts, DRM, CDN, single sign-on	Not built yet — see the **roadmap** (docs/IMPLEMENTATION_ROADMAP.md)

Known limits:

- Videos added before the quality ladder existed play in one quality (360p). New uploads and samples get 360p, 540p and 720p.
- Per-viewer playlists even for SGAI (sharing comes with a CDN).
- Ad decisions kept in memory (single server).

- Video links are signed and expire. Pilot mode uses a fixed signing secret; a CDN would add its own token checks.
- Email templates are the same for every company (per-company wording later). Sign-in uses local accounts with sign-in tokens, not a company's single sign-on yet. Partner organizations are planned (chapter 23).
- One email is sent at a time. That is fine for account and alert emails, but not for newsletters.
- Backups are cold copies, so the app pauses for about 30 seconds while one is made.

---

23. Organization IDs, sign-in tokens and partners

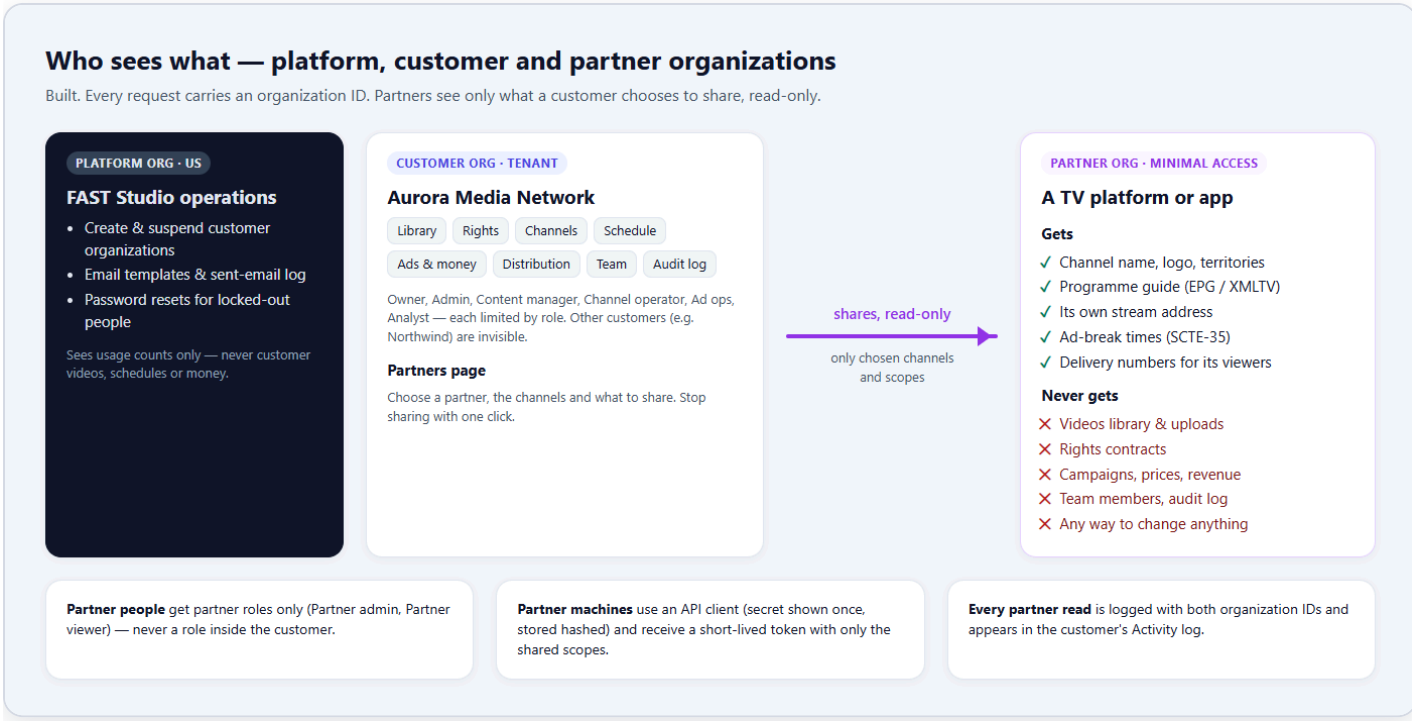
Status: organization IDs and sign-in tokens are **built** (phases P1–P2). Partner organizations are **planned** (P3–P4). The full design is in **PARTNER_ACCESS_AND_TOKENS.md** (docs/PARTNER_ACCESS_AND_TOKENS.md).

Does every API carry an organization ID?

Kind of request	Count	Organization in the request?
Company work: library, rights, channels, schedule, ads, distribution, monitoring, team	46	**Yes.** It is in the address, `/api/orgs/{org}/…`, and is checked against your membership and by the database on every request.
Platform administration: organizations, email, resets	13	Not as an *acting* company. This is cross-company **by design** and shows counts only. Plan: platform staff act as the **platform organization**.
Your account: sign-in, password, forgot password	8	No. This is about a *person*, and `/api/auth/me` lists all their companies.
Viewer playback and signed video pieces	9	**Yes.** A viewer session is tied to one company, and video addresses are signed per company.
SCTE-35 tools	2	No. They read no company data.

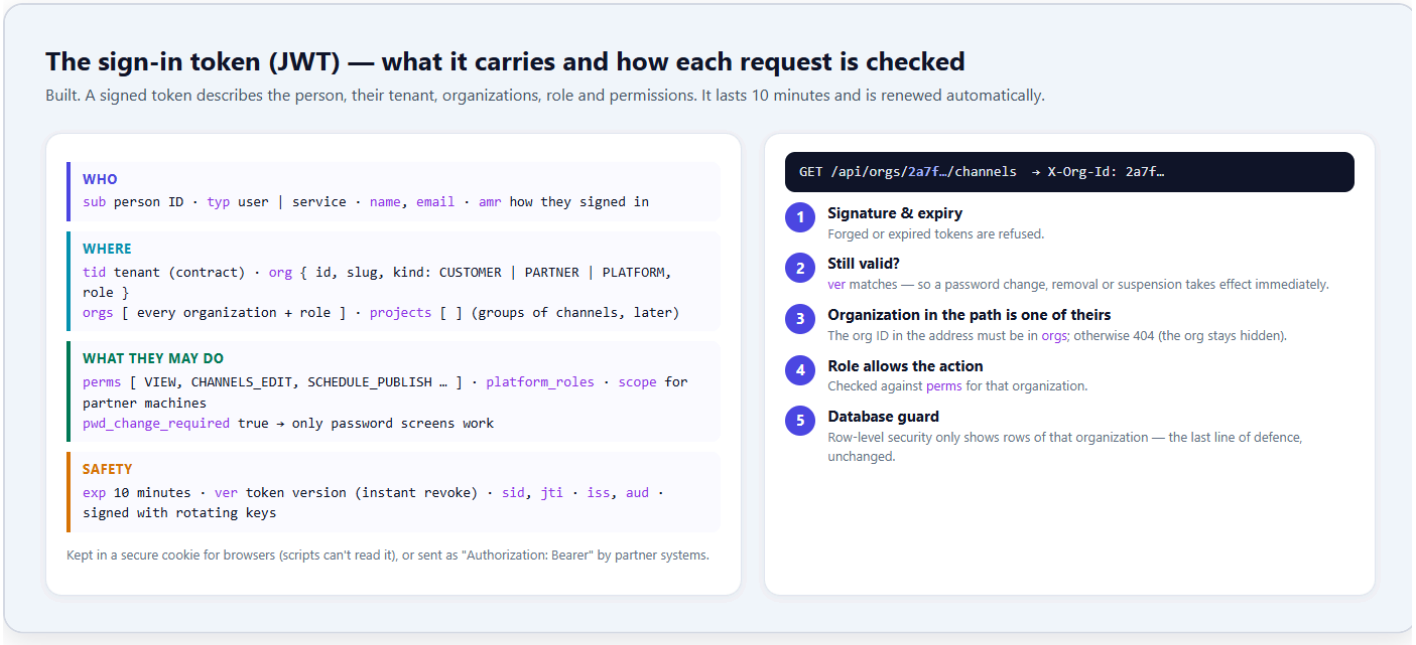
Everything that touches company data is scoped to one organization. **Now built:** every company address also accepts the organization's **stable ID** (its UUID, which never changes) besides the short name. Every answer carries an `X-Org-Id` header, and every log line of a request names the organization. Platform administration is recorded as the **platform organization**, so every audit entry of company or platform work names its organization.

Partner organizations with minimal access — built



A kind of organization, **Partner**, is for TV platforms, apps and ad partners. A partner starts with **no access at all**. On the **Partners** page, a customer chooses which channels to share and what to share: channel details, programme guide, the partner's own stream set-up, ad-break times, viewing numbers. Everything is read-only and can be stopped with one click. Partners never see videos, rights, campaigns, money, team or audit. Every partner read is logged with both organization IDs. The full walkthrough and an example are in chapter 24.

Sign-in tokens (JWT) — built



Since this version, signing in gives you a signed token that describes you:

- **Who:** their ID, name and email
- **Tenant:** the contract
- **Organizations:** the active one and all others, each with its kind (customer, partner or platform) and the person's role
- **Projects:** an empty list for now (groups of channels with their own roles come later)
- **Permissions:** what the role allows
- **Safety details:** 10-minute expiry and a version number, so a password change or removal takes effect immediately

The browser keeps it in a secure cookie that page scripts can't read, next to a **refresh token** that renews it every 10 minutes without you noticing. Scripts sign in with `POST /api/auth/token` and send `Authorization: Bearer`. When you pick another company in the menu, the token switches to it. The address still decides the organization and the token must agree; the database guard stays as the last line of defence.

What happens when something changes:

Change	Effect
Your role changes, or you are removed from a company	Your current token stops working at once; the renewed one has the new role or no longer lists the company
A company is suspended	All its members' tokens are re-checked at once; the company answers *suspended*
You change your password	Your other devices are signed out at once; this one continues
A copied refresh token is used again	That whole sign-in is closed for safety
You choose "sign out everywhere" (`POST /api/auth/logout-all`)	Every device is signed out

Phase	What	Size
P1 ✅ built	Organization UUID in addresses, `X-Org-Id` , org ID in every log line, organization kinds	2–3 days
P2 ✅ built	Sign-in tokens (access + refresh), switch organization, sign out everywhere	4–5 days
P3 ✅ built	Partner organizations: sharing, read-only partner API, Partners page, partner portal	5–7 days
P4 ✅ built	Partner API clients (machine access), rate limits, OpenAPI	2–3 days
P5	Projects, company single sign-on, per-company email wording	later

24. Partners: sharing channels with TV platforms

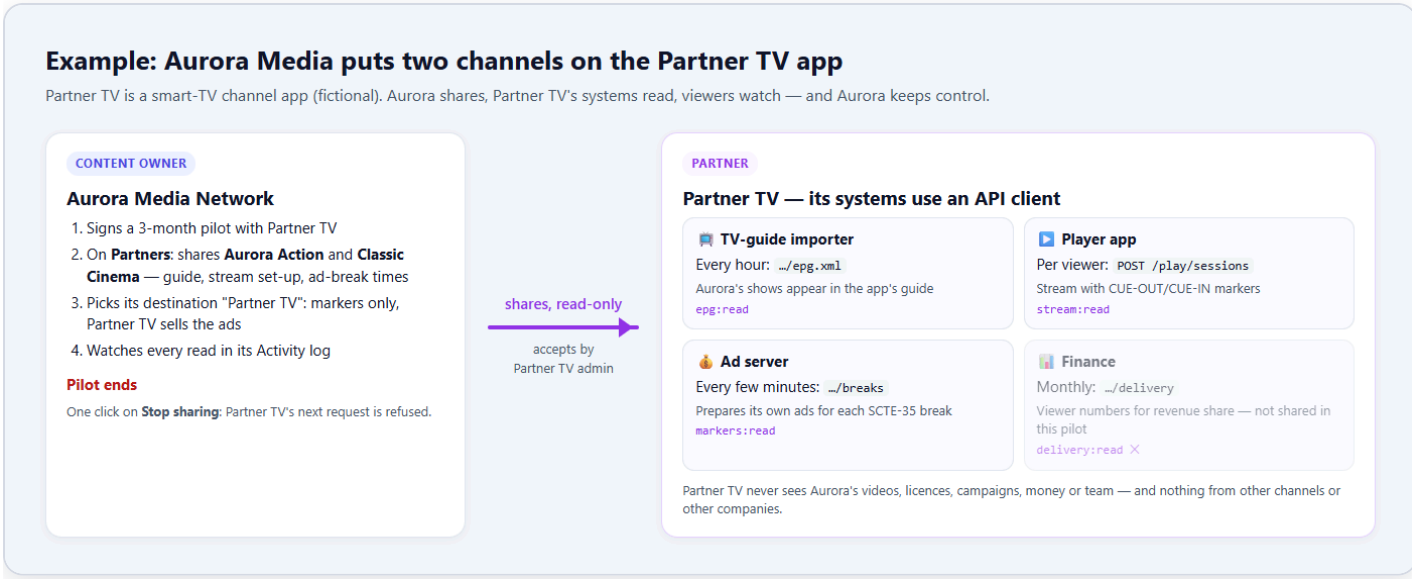
What it is for

A FAST channel earns money when many people watch it, and most viewers find FAST channels inside **other companies' apps**: smart-TV channel guides, streaming services, car and airline screens. Those companies, the **partners**, need four things from you, and they need them **automatically**:

They need	Why	In FAST Studio
Your **programme guide**	To show "what's on" in their TV guide	*Programme guide* (EPG / XMLTV)
A way to **start the stream** for each viewer	With their own ad set-up and the markers they understand	*Stream set-up*
Your **ad-break times**	If they sell the ads, their ad system must know when breaks come (SCTE-35)	*Ad-break times*
Viewing numbers	To agree on revenue share at the end of the month	*Viewing numbers* (their platform only)

Without this feature you would email spreadsheets, hand out passwords to your own account, or build a custom feed per partner. With it, the partner gets its **own small account** (a *partner organization*). You share exactly what the contract says, they read it themselves (people in the partner portal, machines through the partner API), and you can see every read and stop at any time.

Example: Aurora Media puts two channels on the Partner TV app

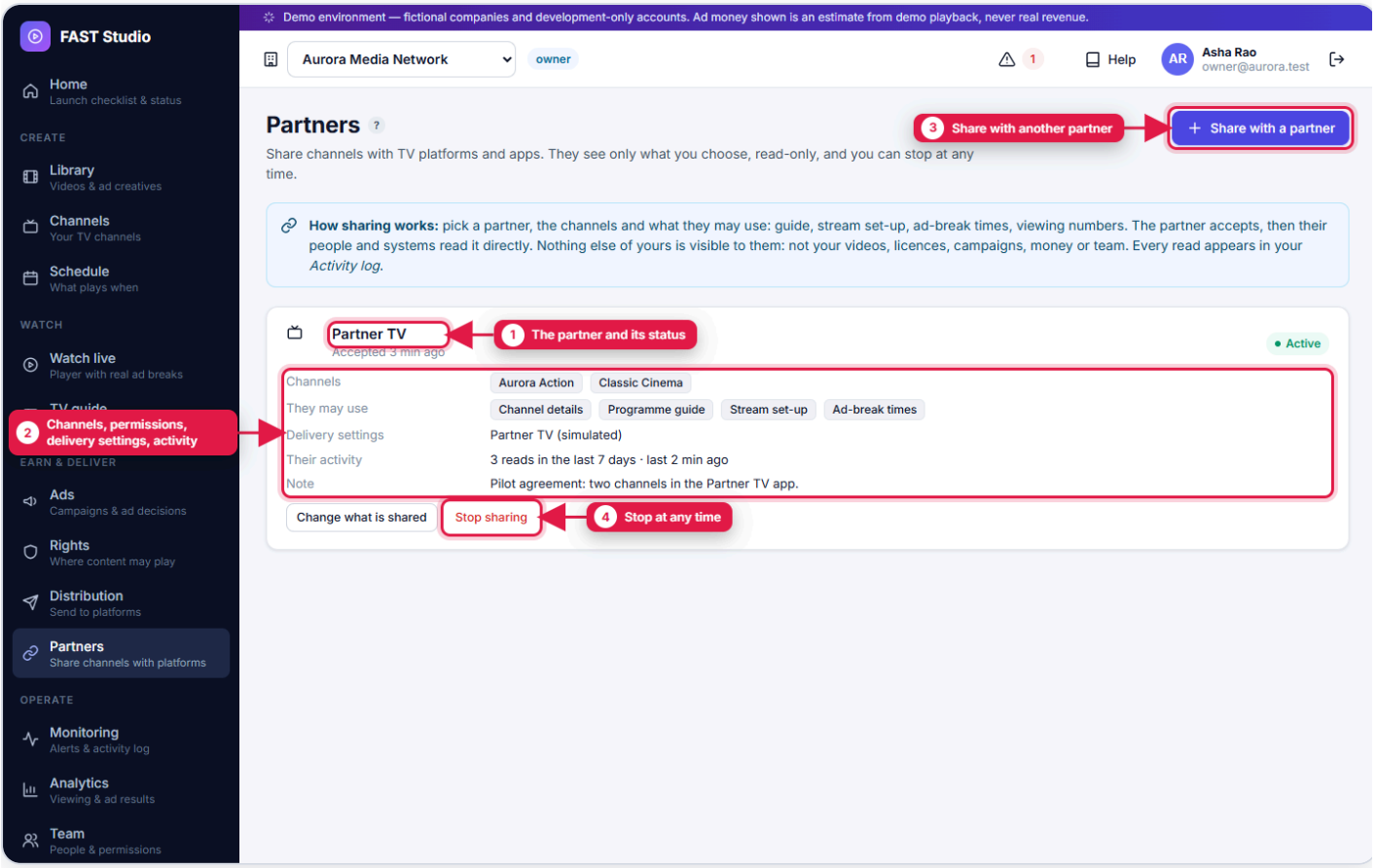


The demo contains exactly this story, so you can follow it with the demo accounts:

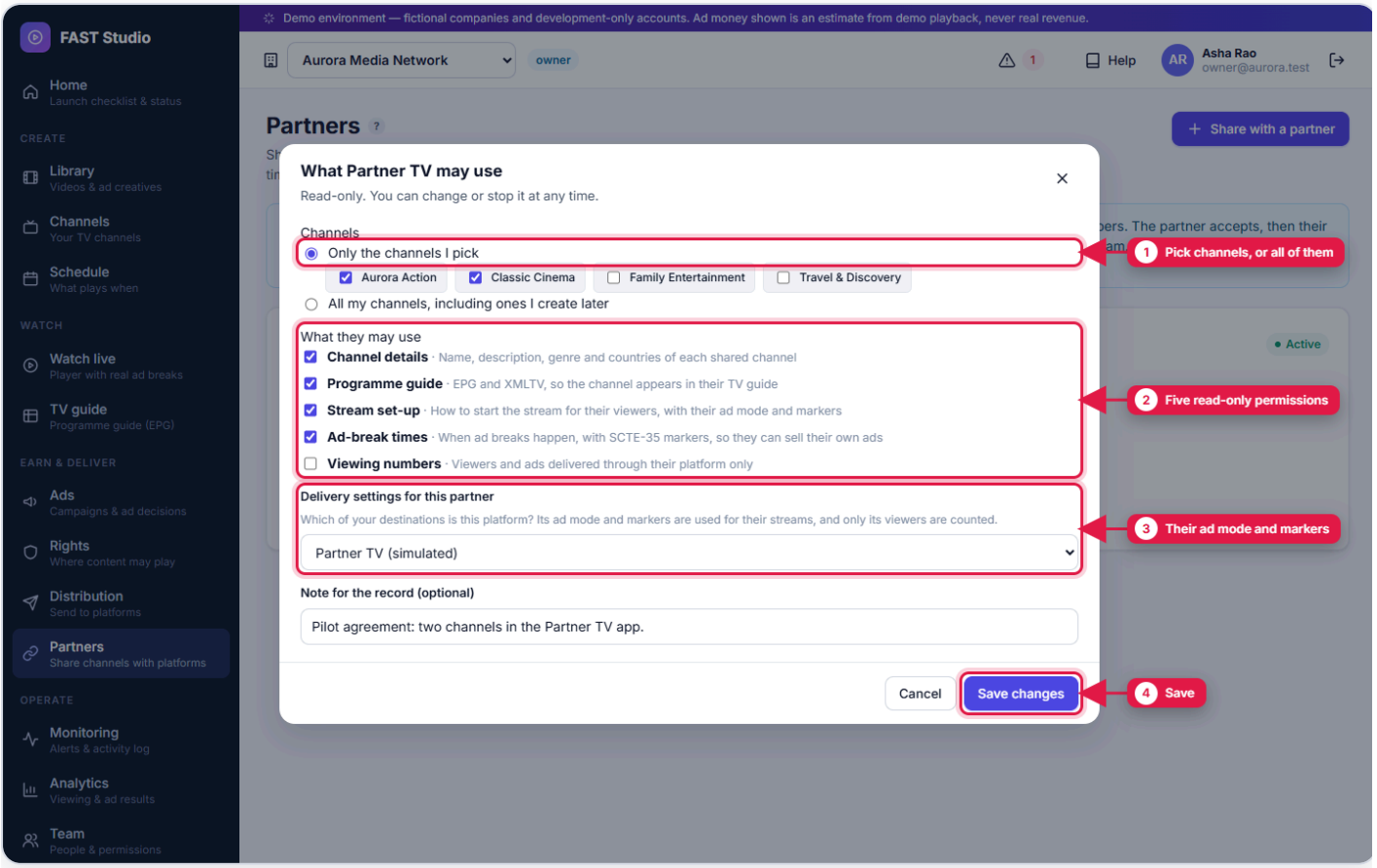
1. **The deal.** Aurora Media signs a 3-month pilot with *Partner TV*, a smart-TV channel app. Partner TV wants *Aurora Action* and *Classic Cinema*. It will sell the ads itself, so it only needs markers in the stream.
2. **Partner TV gets an account.** The platform administrator creates *Partner TV* as a **partner** organization (screenshot 80). Its admin, Sam Lee, gets the welcome email.
3. **Aurora shares.** On **Partners**, Asha shares the two channels with *Channel details, Programme guide, Stream set-up, Ad-break times*. She leaves *Viewing numbers* off for the pilot, and picks the destination *Partner TV (simulated)*: ads are sold by the partner, markers are CUE-OUT/CUE-IN. Sam gets an email.
4. **Partner TV accepts and connects.** Sam presses **Accept**. Partner TV's developer creates an API client called *Guide importer*. Their systems then:
   - pull the **XMLTV guide every hour**, so Aurora's shows appear in the Partner TV guide;
   - start one **playback session per viewer**, so viewers watch with CUE-OUT/CUE-IN markers;
   - read **upcoming ad breaks** with SCTE-35 markers, so Partner TV's own ad server prepares its ads.
5. **Aurora keeps an eye on it.** Every read appears in Aurora's *Activity log* as *Sam Lee (Partner TV)* or *Partner TV (API)*, and the Partners page counts reads per week.
6. **Scope changes.** For the full contract, Aurora ticks *Viewing numbers*. Partner TV's finance team can then see viewers and ads **through Partner TV only**, never other platforms' numbers.
7. **The pilot ends.** Aurora presses **Stop sharing**. Partner TV's next request is refused: no guide, no new streams, no numbers. Viewers already watching are not cut off mid-programme.

What Partner TV **never** sees, at any step: Aurora's video library, licences, campaigns and prices, revenue, team, audit log, other channels, or anything of any other company.

For the content owner: the Partners page



1. **The partner and its status.** *Waiting for partner* until they accept, then *Active*, then *Stopped* when you stop.
2. **Channels, permissions, delivery settings and activity**, all in one place. The activity line counts reads in the last 7 days.
3. **Share with a partner** starts a new sharing.
4. **Stop sharing** takes effect on the partner's very next request.

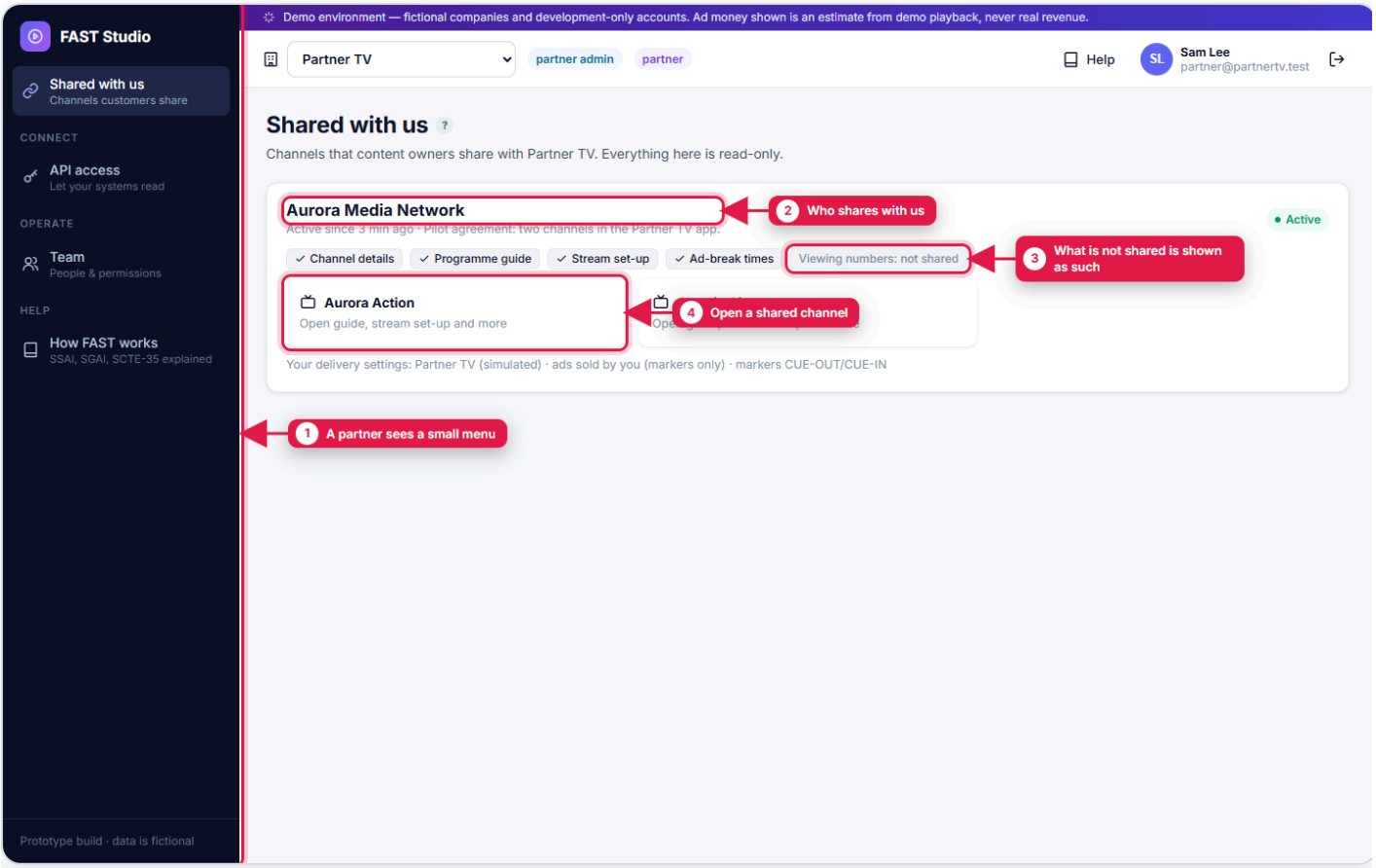


1. **Pick channels, or all of them.** "All my channels" includes channels you create later.
2. **Five read-only permissions**, each explained in plain words. There is no permission to change anything.
3. **Delivery settings: which of your destinations this partner is.** Its ad mode (SSAI, SGAI or markers only) and marker format are used for the partner's streams, and only viewers who came through it are counted.
4. **Save.** Changes apply to the partner at once, with no new invitation.

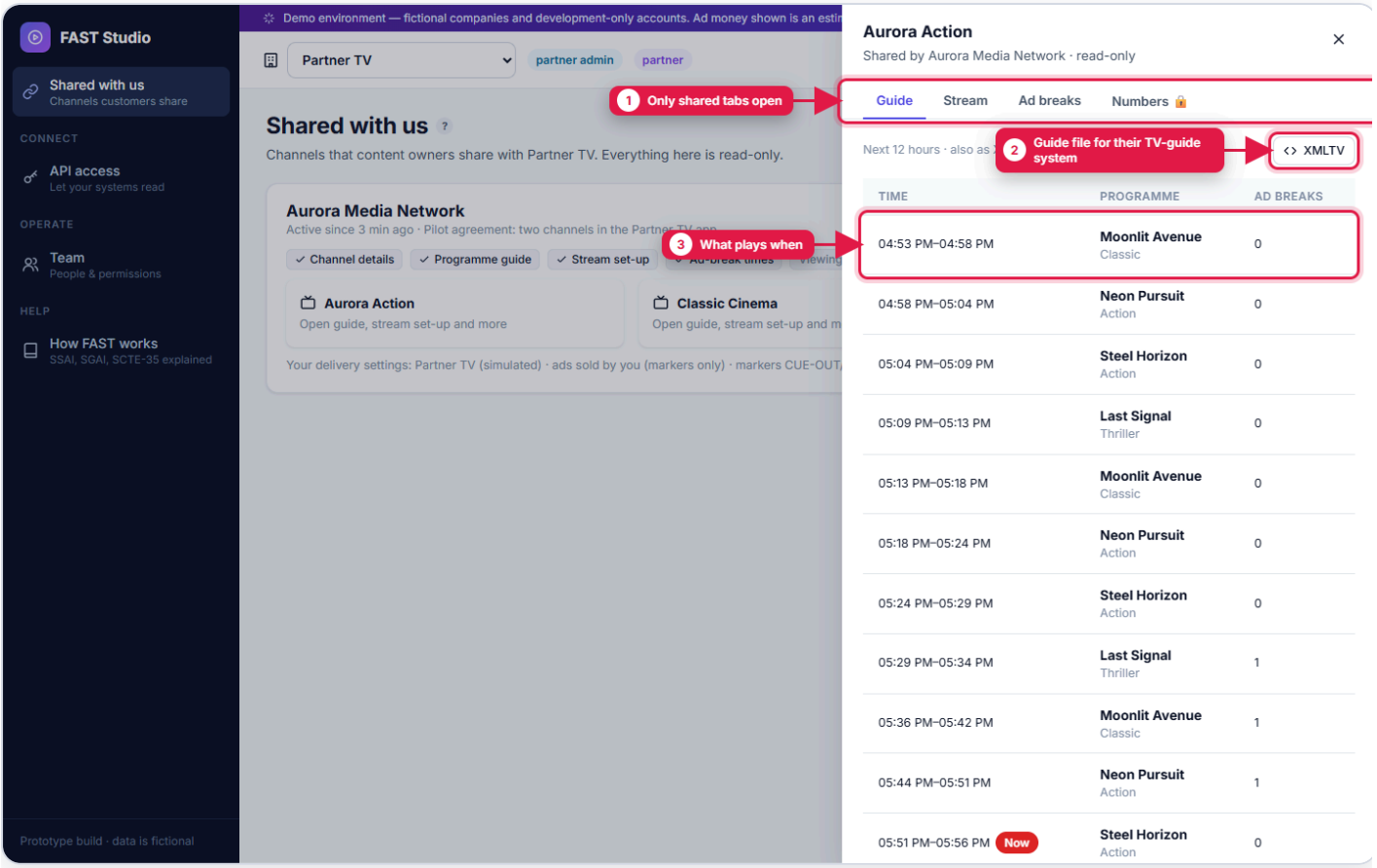
Who can share: owners, admins and channel operators (the same people who manage Distribution).

For the partner: the partner portal

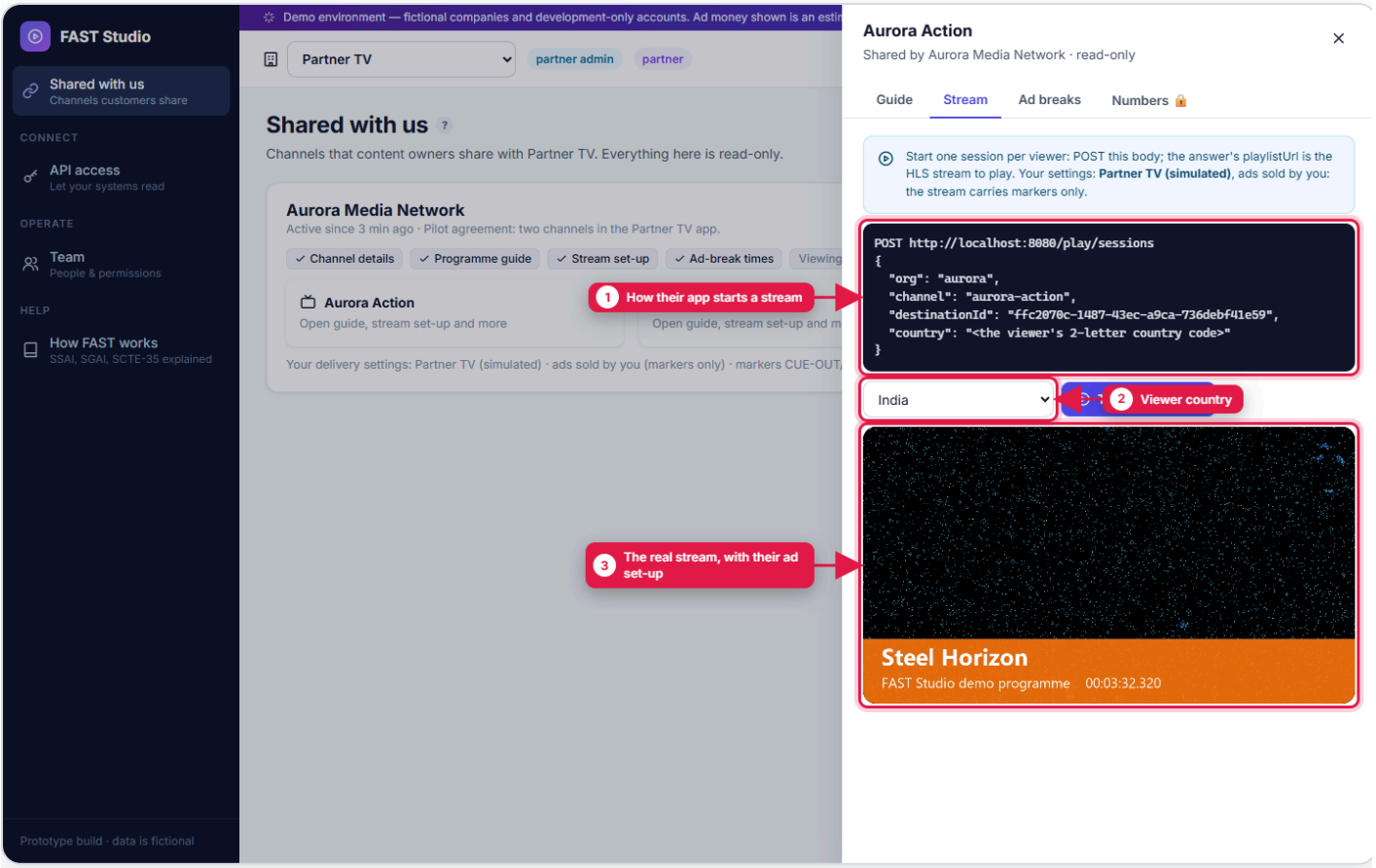
Partner people sign in like anyone else and see a smaller menu: *Shared with us*, *API access*, *Team*.



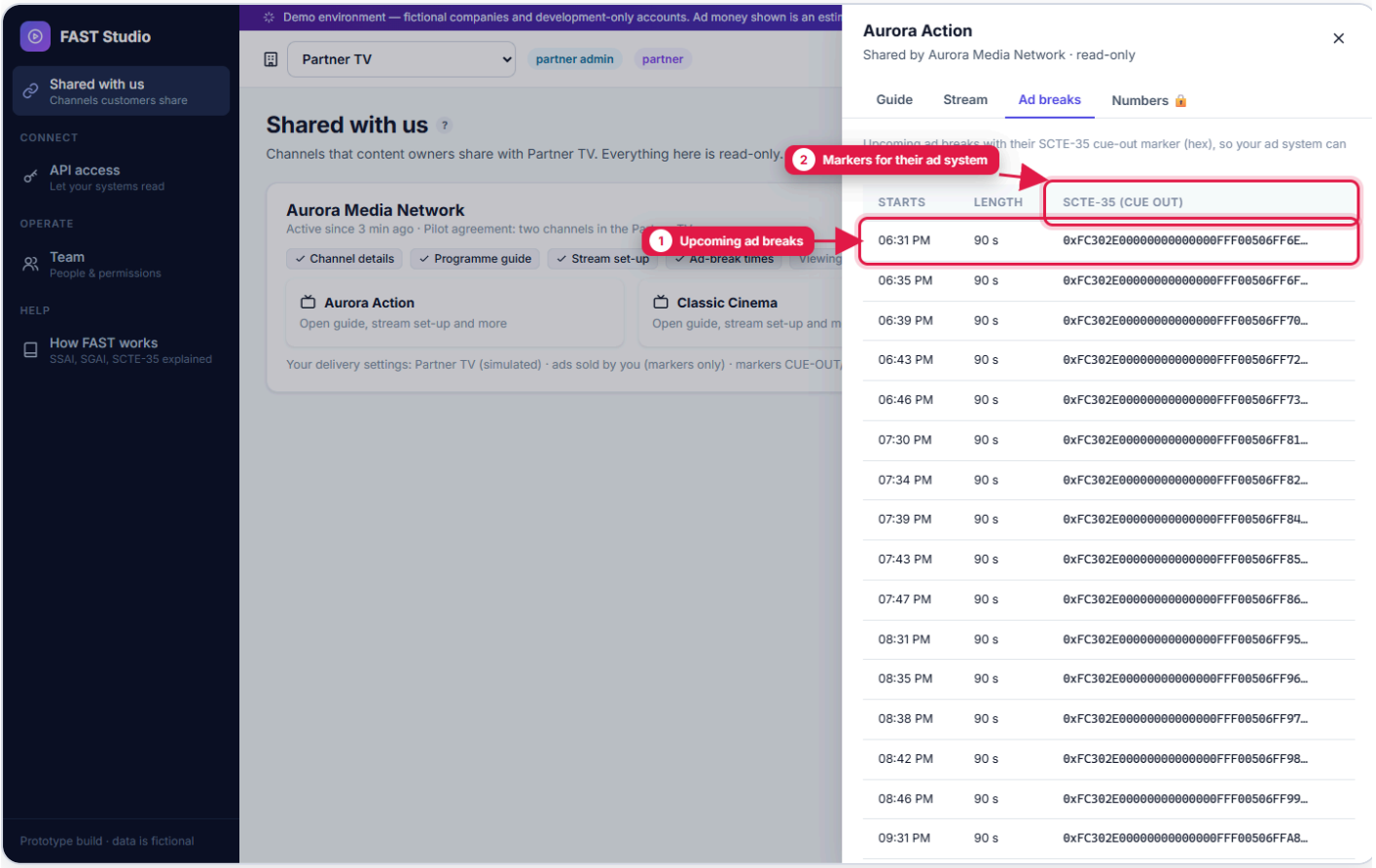
1. **A partner sees a small menu**, with no library, schedule or ads: a partner organization owns no content.
2. **Who shares with us**, with the note from the content owner.
3. **What is not shared is shown as such** (here: *Viewing numbers*), so nobody wonders whether it is missing.
4. **Open a shared channel.** A new sharing shows an **Accept** button instead; only a *partner admin* can accept.



1. **Only shared tabs open.** *Numbers* is locked here.
2. **XMLTV**: the guide file their TV-guide system imports.
3. **What plays when**, with the number of ad breaks per programme.



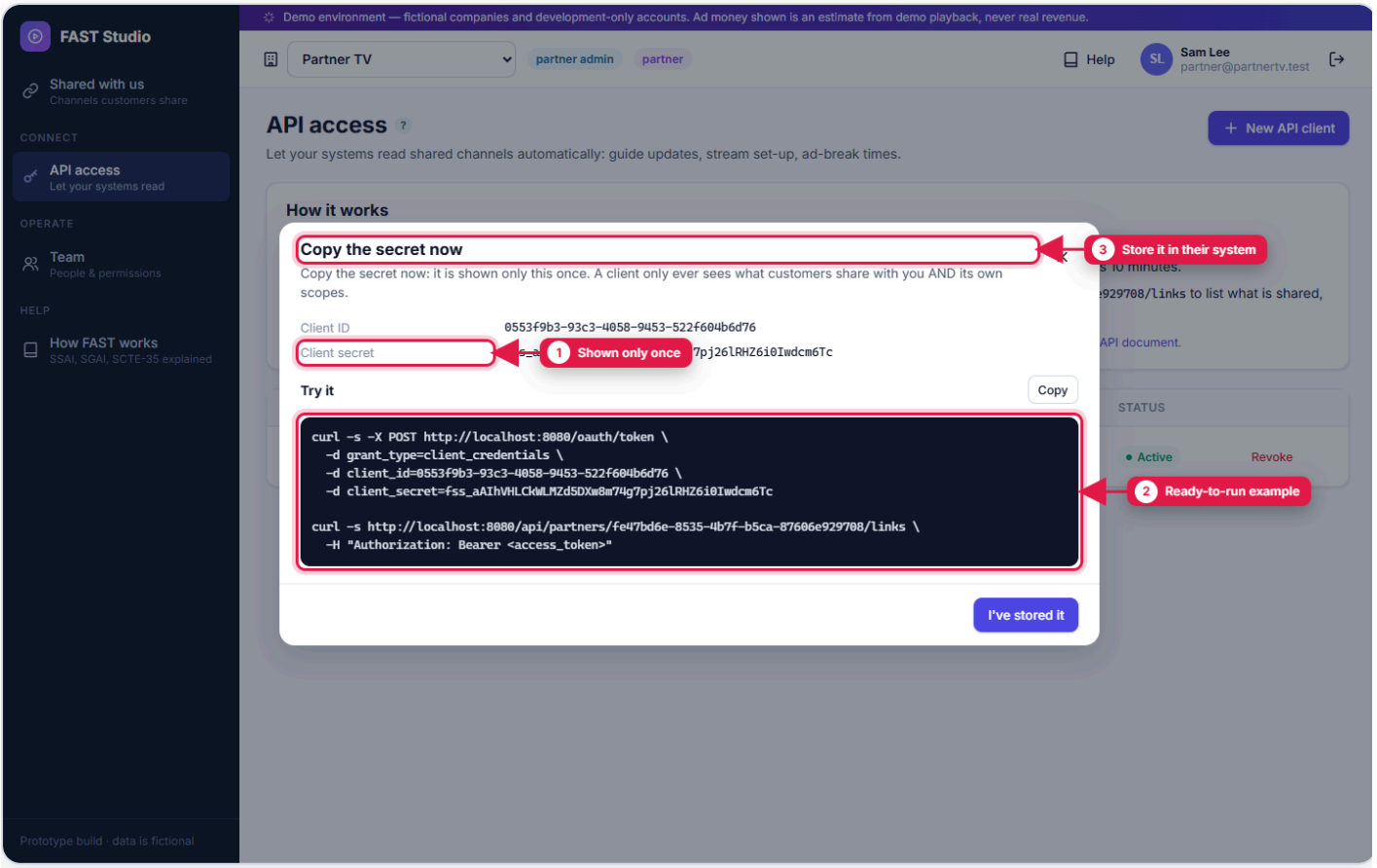
1. **How their app starts a stream**: one request per viewer, ready to copy.
2. **Viewer country**, because rights and ads depend on where the viewer is.
3. **Try it as a viewer** plays the real stream with the partner's own ad set-up, here markers only.



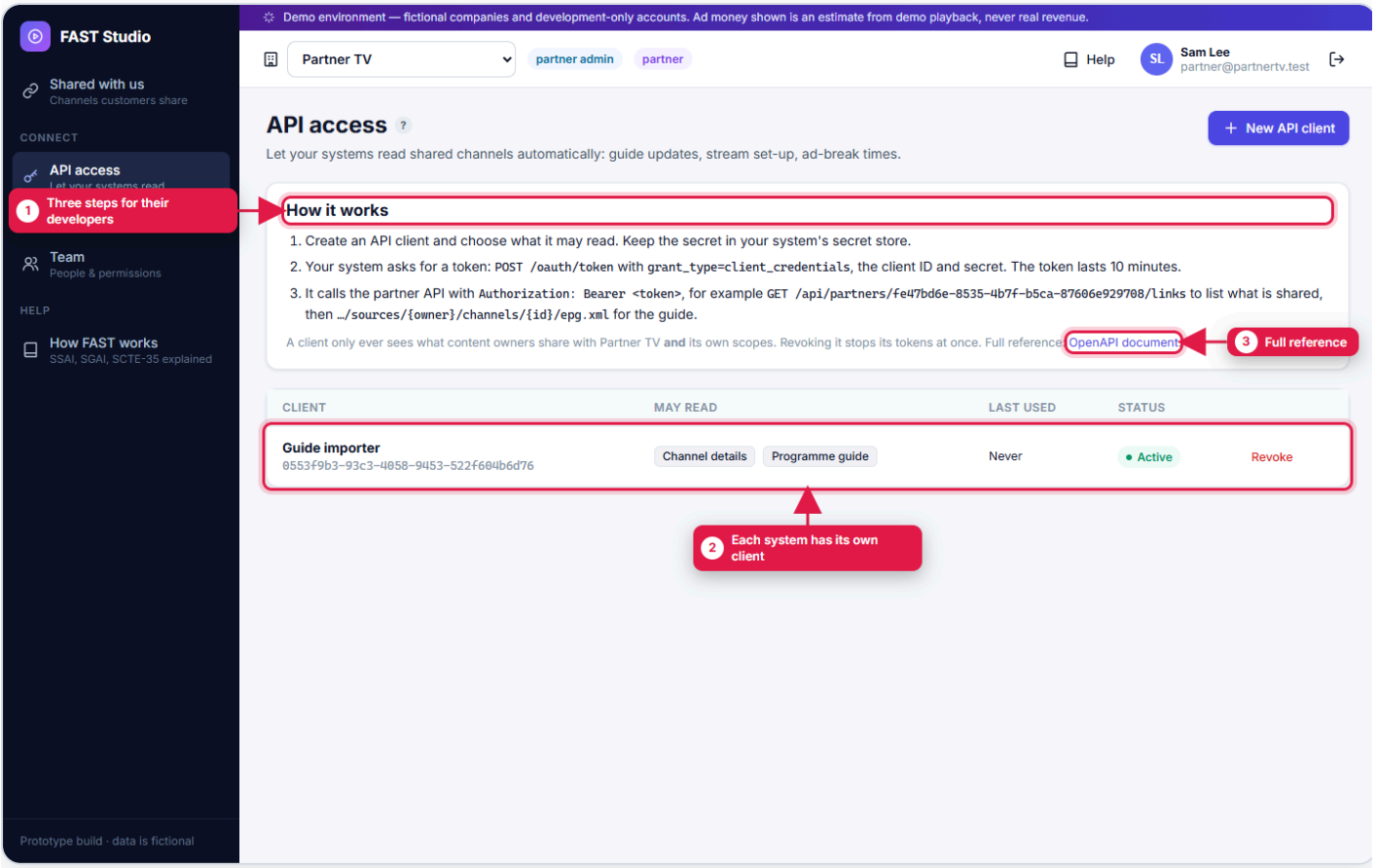
1. **Upcoming ad breaks**: start time and length.
2. **SCTE-35 markers** in hex, the same ones that are in the stream, for their ad system.

Partner roles: **Partner admin** accepts sharings, manages API clients and people. **Partner viewer** can look at everything shared and change nothing.

For the partner's developers: API access



1. **The secret is shown only once.** FAST Studio stores only a fingerprint of it.
2. **A ready-to-run example**: get a token, then list what is shared.
3. **Store it in their system's secret store**, not in an email.

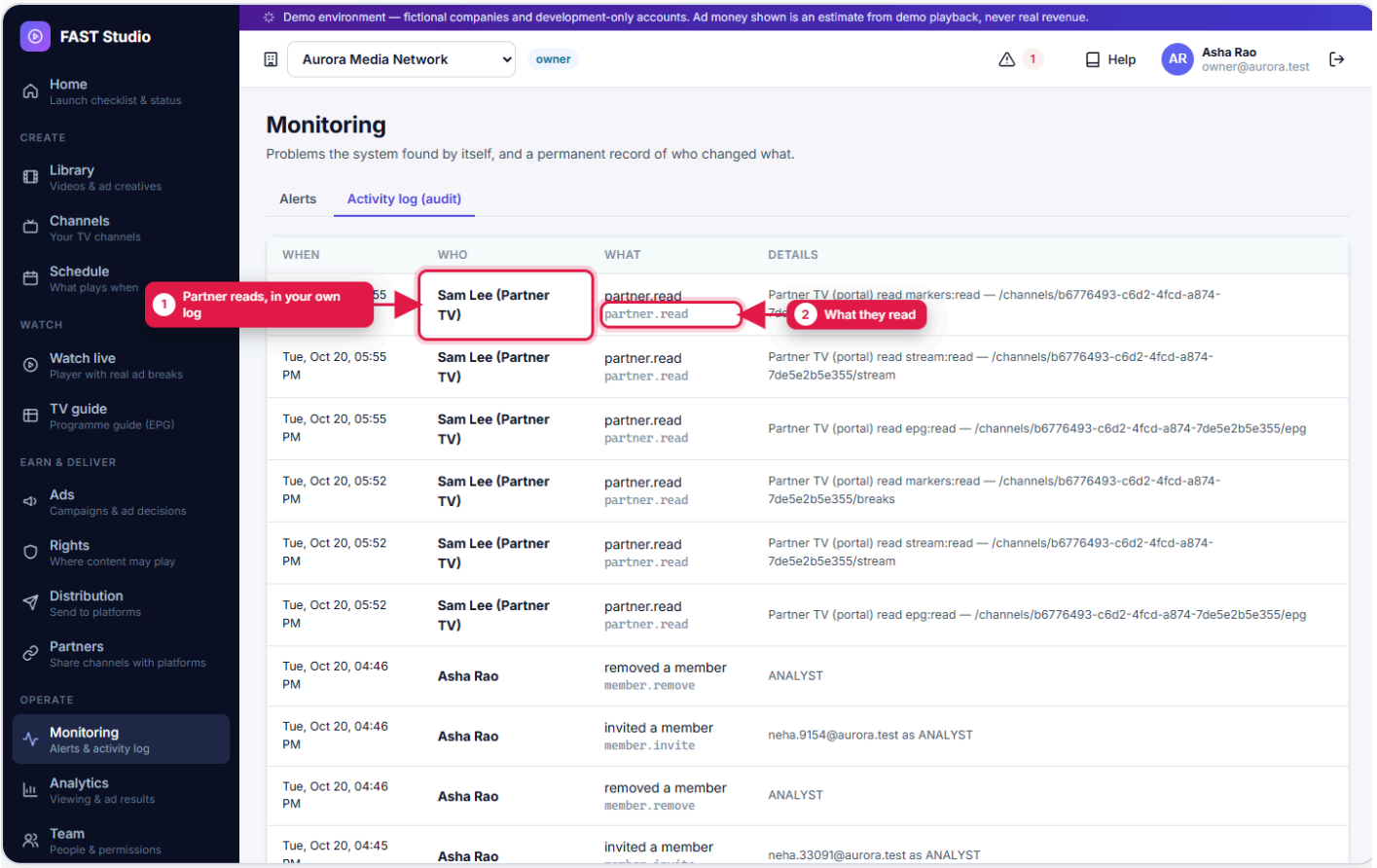


1. **Three steps** for developers: create a client, get a 10-minute token at `POST /oauth/token` (OAuth 2.0 *client credentials*), and call the partner API with `Authorization: Bearer`.
2. **Each system has its own client** with its own permissions. A client sees only what is shared with the partner **and** its own permissions. Revoking it stops its tokens at once.
3. **Full reference**: the OpenAPI document at `/partner-api.openapi.json`.

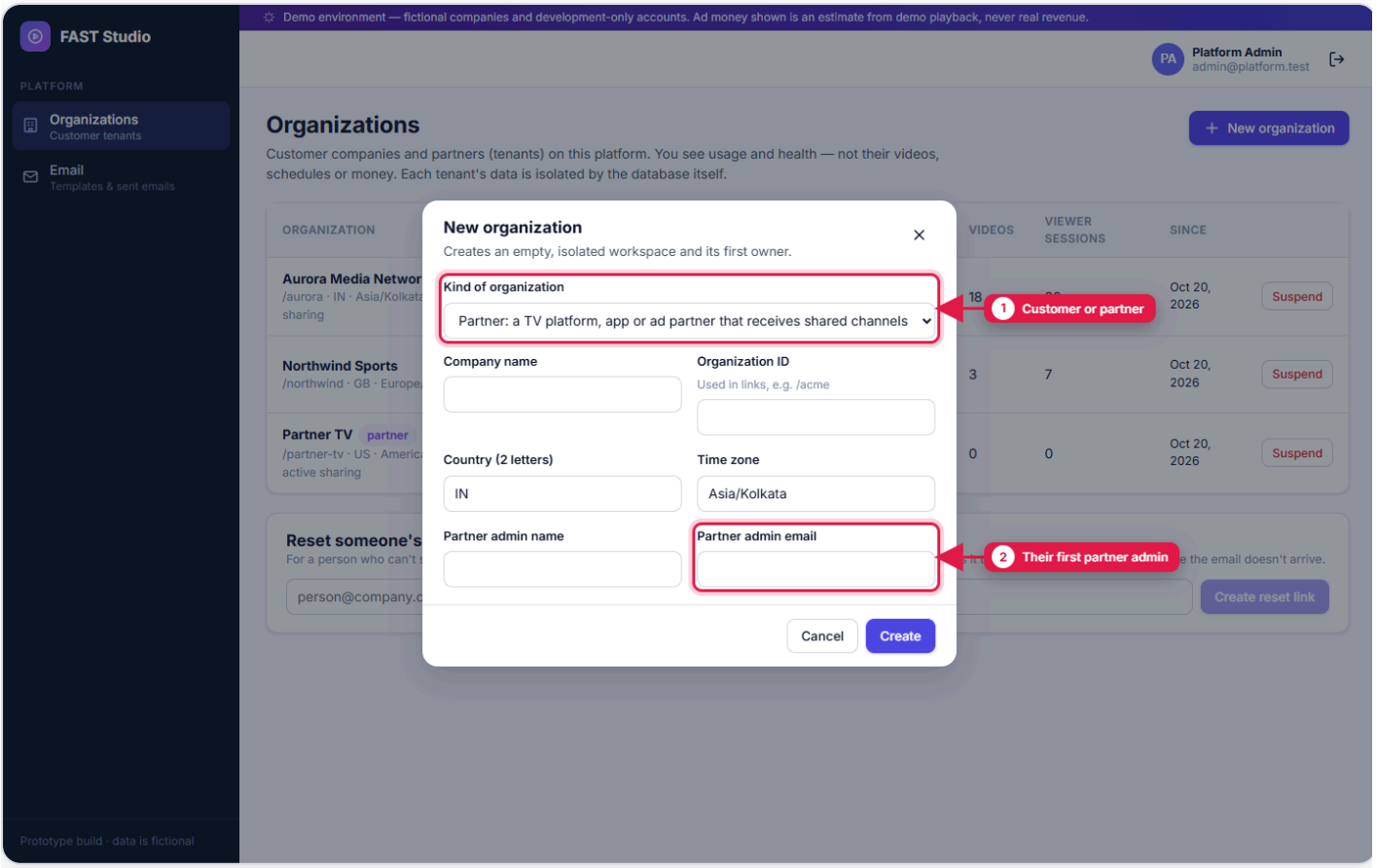
For example, a guide importer that runs every hour:

```
TOKEN=$(curl -s -X POST https://studio.example.com/oauth/token \
  -d grant_type=client_credentials -d client_id=$CLIENT_ID -d client_secret=$CLIENT_SECRET | jq -
curl -s -H "Authorization: Bearer $TOKEN" \
  https://studio.example.com/api/partners/partner-tv/sources/aurora/channels/$CHANNEL_ID/epg.xml
```

Staying in control



1. **Partner reads appear in your own Activity log**, by person, e.g. *Sam Lee (Partner TV)*, or by system, *Partner TV (API)*.
2. **What they read**: which permission and which channel.



1. **Customer or partner.** Only the platform administrator creates organizations, and chooses the kind.
2. **Their first partner admin** gets the welcome email and must choose a password.

Behind the scenes — partners

- **One gatekeeper** (`PartnerInterceptor`) checks every partner request. The caller must belong to the partner organization (or be one of its API clients). The content owner's sharing must be *active*, unexpired, and include both the permission and the channel. Anything else answers **404**, so a partner can't even find out what exists but isn't shared.
- **Read as the owner, read-only.** After the checks, the request runs as the content owner under the database's row-level security, inside a **read-only transaction**. The answers are fixed shapes, never internal records, so a new internal field can't leak by accident.
- **Partner organizations own nothing.** Of the company pages they can only use *Team*; every other company page answers 404 for them.
- **Machines:** API client secrets are 256 random bits and stored only as a fingerprint. `/oauth/token` gives a 10-minute token of type *service* that names the partner and the client's permissions. That token is refused everywhere except the partner API, and stops at once when the client is revoked.
- **Rate limits:** each partner organization may make 120 requests a minute (`faststudio.partners.rate-limit-per-minute`). More gets *429 Too many requests*.
- **Audit with both organizations:** each read is written to the owner's log with the partner organization alongside (`via_org_id`).
- **Tested:** for every partner endpoint, automated tests check that another company's data, an unshared channel or an unshared permission is refused. They also check that partner people can't use company pages, that stopping and narrowing apply at once, and the API client and token rules.